

UNIVERSIDADE FEDERAL FLUMINENSE

PEDRO CORTEZ FETTER LOPES

**FEM-based Permeability Computation in  
GPU for Large-Scale Voxelized Porous  
Microstructures**

Niterói

2023

UNIVERSIDADE FEDERAL FLUMINENSE

PEDRO CORTEZ FETTER LOPES

# FEM-based Permeability Computation in GPU for Large-Scale Voxelized Porous Microstructures

Tese de Doutorado apresentada ao Programa de Pós-Graduação em Computação da Universidade Federal Fluminense como requisito parcial para a obtenção do Grau de Doutor em Computação. Área de concentração: Ciência da Computação.

Orientador:

André Maués Brabo Pereira

Coorientador:

Ricardo Leiderman

Niterói

2023

Ficha catalográfica automática - SDC/BEE  
Gerada com informações fornecidas pelo autor

L864f    Lopes, Pedro Cortez Fetter  
          FEM-based Permeability Computation in GPU for Large-Scale  
          Voxelized Porous Microstructures / Pedro Cortez Fetter Lopes. -  
          2023.  
          116 f.: il.

          Orientador: André Maués Brabo Pereira.  
          Coorientador: Ricardo Leiderman.  
          Tese (doutorado)-Universidade Federal Fluminense, Instituto  
          de Computação, Niterói, 2023.

          1. Computação Paralela. 2. Método dos Elementos Finitos.  
          3. Mecânica dos Fluidos. 4. Meios Porosos. 5. Produção  
          intelectual. I. Pereira, André Maués Brabo, orientador. II.  
          Leiderman, Ricardo, coorientador. III. Universidade Federal  
          Fluminense. Instituto de Computação.IV. Título.

CDD - XXX

# PEDRO CORTEZ FETTER LOPES

## FEM-based Permeability Computation in GPU for Large-Scale Voxelized Porous Microstructures

Tese de Doutorado apresentada ao Programa de Pós-Graduação em Computação da Universidade Federal Fluminense como requisito parcial para a obtenção do Grau de Doutor em Computação. Área de concentração: Ciência da Computação.

Aprovada em Junho de 2023.

### BANCA EXAMINADORA

*André Maués Brabo Pereira*

---

Prof. Dr. André Maués Brabo Pereira - Orientador, UFF

*Ricardo L*

---

Prof. Dr. Ricardo Leiderman - Co-orientador, UFF

*Marcos de Oliveira Lage Ferreira*

---

Prof. Dr. Marcos de Oliveira Lage Ferreira, UFF

*Daniel*

---

Prof. Dr. Daniel Cardoso Moraes de Oliveira, UFF

*Luís Antônio Guimarães Bitencourt Jr.*

---

Prof. Dr. Luís Antônio Guimarães Bitencourt Júnior, USP

*Luiz Fernando Campos Ramos Martha*

---

Prof. Dr. Luiz Fernando Campos Ramos Martha, PUC-Rio

*Rodolfo A. Victor*

---

Dr. Rodolfo Araújo Victor, Petróleo Brasileiro S.A.

Niterói

2023

*“This cosmic sadness is just here to remind you that, without love,  
breathing is just the ticking of (an unwinding clock)”*

*Chris Hannah*

# Agradecimentos

Agradeço primeiramente à minha namorada, Ana Carolina, por me acompanhar e me apoiar durante toda a pós-graduação. Ana, sem você, eu não haveria chegado até aqui. Muito obrigado por toda a ajuda, por ser quem você é, e por estar ao meu lado. Espero sempre poder te retribuir à altura.

Agradeço à minha família, pelo incentivo e pelas oportunidades que me proporcionaram ao longo da vida para que eu pudesse me dedicar aos estudos. Em especial, agradeço aos meus pais, Marcella e Marcos, e à minha irmã, Isabel.

Sou grato aos meus orientadores, André e Ricardo, por toda a contribuição a este trabalho e à minha formação como pesquisador, de forma geral. Vocês me guiaram desde quando eu era um aluno recém saído da graduação até este momento em que concluo meu Doutorado. Hoje me sinto capaz de trabalhar e contribuir no meio acadêmico, e muito disto se deve às suas mentorias. Muito obrigado.

Agradeço aos colegas de trabalho no LCC. Não cito nomes para não deixar alguém de fora, sou grato a todos. Um doutorando não é uma ilha, e a convivência diária no laboratório é refletida no produto final. Fazem parte do processo: debates sobre questões teóricas, conversas bem-humoradas no almoço, forças-tarefa *impromptu* para encontrar bugs em códigos, etc. Todos foram importantes para mim.

Sou grato a todos os meus amigos, da época de escola à faculdade. Em especial, ao Bernardo e ao Patrick, que me aturam desde a infância. Vocês certamente tem uma parcela de contribuição na minha formação como pessoa, e sempre os agradecerei por isso.

Por fim, mas não menos importante, agradeço a todos os professores com os quais tive contato no Instituto de Computação. Todos os cursos que fiz aqui durante o Mestrado e Doutorado somaram à minha formação, e tenho apenas boas lembranças de todos. Estendo o agradecimento à toda a equipe de funcionários que mantém o Instituto funcionando e sua estrutura bem cuidada. Não há trabalho de Doutorado que possa ser feito sem limpeza, segurança, manutenção das instalações, e tantos mais outros serviços essenciais que não podem passar despercebidos. Muito obrigado a todos.

# Resumo

O escoamento de fluidos em meios porosos é um fenômeno físico de interesse especial para áreas como a Petrofísica Digital e a Ciência de Materiais, onde se busca estimar, por meio de simulações computacionais, a permeabilidade de amostras de rocha e de materiais porosos, respectivamente. Estas amostras costumam ser imageadas, dentre outras possibilidades, por equipamentos de microtomografia computadorizada ( $\mu$ CT). Desta forma, há um campo de estudo aberto por soluções numéricas das equações diferenciais que governam este problema físico em malhas baseadas em imagens (*voxel-based*). Os problemas desta natureza costumam ser de larga escala, lidando com centenas de milhões de variáveis a serem computadas, o que leva à necessidade de técnicas de implementação eficientes em memória, e que explorem paralelismo.

Nesta Tese, considera-se o escoamento monofásico em meios porosos, que é um problema regido pelo fluxo de Stokes. Busca-se resolver as equações diferenciais parciais em questão pelo Método dos Elementos Finitos (MEF), o que não é corriqueiro na literatura dedicada ao estudo de permeabilidade em imagens de  $\mu$ CT. Usualmente, outros autores preferem métodos como simulações baseadas em partículas ou em redes de poros, sob o argumento de que simulações de escoamento de fluido pelo MEF em grandes imagens seriam problemas inviáveis em termos de memória. No entanto, a adoção de esquemas “sem matriz” (ou “sem montagem”) mostrou-se bastante promissora nos últimos anos para a implementação do MEF *voxel-based* em grandes malhas estruturadas para problemas como condução de calor e elasticidade. O objetivo deste trabalho é a expansão destas técnicas para a simulação do escoamento de fluidos em meios porosos, visando a obtenção da permeabilidade de materiais imageados por  $\mu$ CT em computadores pessoais. É apresentada uma implementação em placas gráficas (GPUs) para modelos baseados em imagens. Mostra-se como a estratégia nó-a-nó (NBN) pode ser adaptada para problemas onde apenas os poros fazem parte da malha. Ademais, uma extensão desta para um esquema estêncil-a-estêncil (SBS) é apresentada. *Solvers* iterativos baseados no Método dos Gradientes Conjugados (CG) e no Método do Resíduo Mínimo (MINRES) foram implementados para simulações em 2D e 3D. São desenvolvidas versões alternativas destes que alocam apenas 3 e 2 vetores de variáveis na memória global da GPU.

Resultados são apresentados para a verificação e validação das implementações propostas. Modelos bidimensionais e tridimensionais de permeabilidade analítica ou semi-analítica conhecida são analisados para que os resultados computacionais sejam comparados com valores de referência. Em seguida, um modelo de fibra e uma amostra de arenito são estudados, a fim de demonstrar a capacidade do programa em lidar com amostras digitais obtidas por varreduras de  $\mu$ CT. Finalmente, um lote de amostras de rocha reservatório é estudado com os *solvers* desenvolvidos, para mostrar sua adequação para estudos de Petrofísica Digital em larga escala.

Todas as simulações relatadas nesta Tese foram realizadas em placas gráficas de rela-

tivo baixo custo. Análises de amostras baseadas em imagens com  $600^3$  *voxels* e até  $1000^3$  *voxels* são conduzidas com alocação de memória limitada, abaixo de 8 GB ou 12 GB, e dentro de prazos razoáveis, ou seja, dezenas de minutos, ou, no máximo, menos de 2 horas por simulação. As métricas obtidas contrastam com as reportadas sobre implementações em CPU encontradas na literatura, que comumente requerem dezenas de *gigabytes* e múltiplas horas para simulações de tais dimensões.

**Palavras-chave:** Permeabilidade, Stokes, microtomografia computadorizada, MEF, implementação sem matriz, GPU.

# Abstract

Fluid flow in porous media is a physical phenomenon of special interest to areas such as Digital Petrophysics and Materials Science, where one seeks to estimate, through computer simulations, the permeability of rock samples and porous materials, respectively. These samples are usually imaged, among other possibilities, by micro-computed tomography ( $\mu$ CT) scanners. In that sense, there is an open field of study for numerical solutions of the differential equations that govern this physical problem in image-based (or voxel-based) meshes. Problems of this nature tend to be large-scale, dealing with hundreds of millions of variables to be computed, which leads to the need for memory-efficient implementation techniques that explore parallelism.

In this Thesis, the problem of Stokes flow through porous media is tackled. It is sought to solve the partial differential equations in question using the Finite Element Method (FEM), which is not commonplace in the dedicated literature. Usually, researchers prefer other methods such as simulations based on finite differences, Lattice-Boltzmann methods, or pore networks, on the grounds that simulations of fluid flow by the FEM in large images would be an unfeasible computational problem in terms of memory. However, the adoption of matrix-free (or assembly-free) schemes has shown to be very promising in recent years for the implementation of voxel-based FEM for large structured meshes, considering elliptic problems such as heat transfer and elasticity. The objective of this Thesis is the expansion of these techniques for the simulation of fluid flow in porous media, aiming at computing the permeability of materials imaged by  $\mu$ CT scanning in personal computers. An implementation on graphics processing units (GPUs) for image-based models is presented. It is shown how the node-by-node (NBN) strategy can be adapted to handle problems where only the pores are part of the mesh. Further, an extension of it for stencil-by-stencil schemes is presented. Iterative solvers based on the Conjugate Gradient Method (CG) and the Minimal Residual Method (MINRES) were implemented for 2D and 3D simulations. Alternative versions of these solvers that require just 3 and 2 arrays of variables to be allocated in the GPU are devised.

Results are presented for the verification and validation of the proposed implementations. Two- and three-dimensional voxel-based models of known analytical or semi-analytical permeability are analyzed to have their computational results checked against reference values. Then, a fiber model and a sandstone rock sample are studied, in order to demonstrate the capability of the program to handle digital samples obtained via  $\mu$ CT scans. Finally, a batch of reservoir rock samples is studied with the developed solvers, to showcase their fitness for large-scale Digital Petrophysics studies.

All of the simulations reported in this Thesis were conducted in consumer-grade gaming-dedicated graphics cards. Analyses of image-based samples with  $600^3$  voxels and up to  $1000^3$  voxels are conducted with limited memory allocation, under 8 GB or 12 GB, and within sensible time frames, i.e. tens of minutes, or, at most, less than 2 hours per

simulation. The obtained metrics stand in contrast with those from CPU implementations found in the literature, that commonly require tens of GB's and multiple hours for simulations of such dimensions.

**Keywords:** Permeability, Stokes flow, micro-computed tomography, FEM, matrix-free, GPU.

# List of Figures

|     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |    |
|-----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1.1 | A sandstone sample. To the left, two-dimensional slices of the data obtained as a $\mu$ CT scan. To the right, voxel-based depictions of segmented subvolumes extracted from the $\mu$ CT scan. In grey, the solid phase, in blue, the pore space – this is where fluid flow simulations take place. . . . .                                                                                                                                                                                                                                                                                | 5  |
| 3.1 | An unbounded homogeneous macroscale domain $Y$ with its material phase being depicted by a heterogeneous microscale periodic unit cell $\Omega$ . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                   | 21 |
| 4.1 | Difference between an unstructured mesh (to the left) and a voxel-based mesh (to the right). . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | 32 |
| 4.2 | Illustration of the access patterns for a node in a NBN scheme, for a two-dimensional problem. On the left, a depiction of the order of access of neighboring elements: counterclockwise. On the right, the arrows indicate memory accesses to data associated with each neighboring node, and floating point multiplications. Notice that each node has 3 DOFs in 2D (or 4 DOFs, in 3D). The arrows highlighted in red depict such operations with the current node. The notion carries to three-dimensional problems, where there are 8 voxels, arranged in “layers” of 4 voxels. . . . . | 43 |
| 4.3 | Illustration of the access patterns for a node in a SBS scheme, for a two-dimensional problem. The arrows indicate memory accesses and floating point multiplications. As for NBN, the notion carries to three-dimensional problems. . . . .                                                                                                                                                                                                                                                                                                                                                | 45 |
| 4.4 | Image-based pore meshing strategy. To the left: Definition of internal pore nodes (green), within $\Omega_p$ , and border nodes (red), at $\Gamma$ . Solid phase identified by white voxels and pore space represented by black voxels. To the right: The application of the proposed node numbering rules to this mesh. Notice that the periodic boundary conditions at $\partial\Omega$ lead to matching indexes in opposite faces. . . . .                                                                                                                                               | 47 |

|     |                                                                                                                                                                                                                                                                                                                                                                                              |    |
|-----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 4.5 | Periodic vertex numbering scheme in the monolithic grid that characterizes an image. Notice that the operations to compute the vicinity of any vertex ( $\{\text{Up,Down,Right,Left,Near,Far}\}$ ) can be straightforwardly implemented via arithmetic expressions with modulus operators, as indicated in Table 4.1. . . . .                                                                | 49 |
| 4.6 | Illustration of the 8-bit keys employed to map the solid-pore interface, for a two-dimensional problem. In three-dimensional problems, the first 4 bits would have been used for the other “layer” of voxels. . . . .                                                                                                                                                                        | 50 |
| 5.1 | CPU vs. GPU architecture. Based on a figure seen in Kirk and Hwu [69]. .                                                                                                                                                                                                                                                                                                                     | 52 |
| 5.2 | Memory hierarchy of a CUDA-enabled device. Based on a Figure seen in Gupta [50]. . . . .                                                                                                                                                                                                                                                                                                     | 53 |
| 5.3 | Flowchart schematics to illustrate the functioning of the developed program.                                                                                                                                                                                                                                                                                                                 | 57 |
| 6.1 | Two-dimensional medium for which there are analytical solutions, represented by raster images with increasing pixel count. Solid phase in white, pore space in black. . . . .                                                                                                                                                                                                                | 72 |
| 6.2 | Imposed pressure field due to the effective periodic gradient $\overline{p}_i$ applied for analyses in direction $x_1$ (horizontal) of the samples shown in Figure 6.1. Notice that fluid tends to flow against the direction of this pressure gradient, i.e. from higher to lower pressure. . . . .                                                                                         | 72 |
| 6.3 | Porosity (left) and number of DOFs (right) for each of the considered samples. These are plots that illustrate the data shown in Table 6.2. . . . .                                                                                                                                                                                                                                          | 73 |
| 6.4 | Convergence of the FEM-based permeability analysis for the model illustrated in Figure 6.1. To the left, the results for each image dimension. To the right, the numerical error on a logarithmic scale. $k$ is the permeability obtained by the simulation, $k_{DT} = 2.0434\text{e}+04$ D is the permeability given by the series expansion solution from Drummond and Tahir [32]. . . . . | 73 |
| 6.5 | Metrics in relation to the number of DOFs of each analysis. To the left, the number of iterations for the residual to converge within the numerical tolerance. To the right, the average time per iteration. Both of these growths impact the overall elapsed time. . . . .                                                                                                                  | 74 |
| 6.6 | Velocity field magnitude = $\ u_i\ $ [mm/s] (top) and pressure field = $\overline{p}_i x_i + p^*$ [Pa] (bottom) for the coarser and finer samples. . . . .                                                                                                                                                                                                                                   | 75 |

|      |                                                                                                                                                                                                                                                                                                                                                                                                                               |    |
|------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 6.7  | Convergence of the normalized residual by the $L2$ norm criterion for the coarsest and finest samples shown in Figure 6.1. Comparison between CG and MINRES solvers. Note that the curves for the $2048^2$ pixels model have been sampled at each 10 iterations. . . . .                                                                                                                                                      | 76 |
| 6.8  | Periodic array of cylindrical pipes. Fluid flows solely within and along the pipes, the space between them can be understood as solid. . . . .                                                                                                                                                                                                                                                                                | 78 |
| 6.9  | Periodic cell of an array of cylindrical pipes, with dimensions $1\text{mm} \times 1\text{mm}$ in the $x_1x_3$ cross-section. Three pipe diameters are considered: in black, $0.125\text{mm}$ ; in darker grey, $0.25\text{mm}$ ; in lighter grey, $0.5\text{mm}$ . Notice that this is a three-dimensional model, with a “thickness” of 10 voxels. Fluid flow is simulated in direction $x_2$ . . . . .                      | 78 |
| 6.10 | Convergence of the FEM-based permeability analysis for the model illustrated in Figure 6.9. To the left, the results for each image dimension, and for each radius of the pipes (from top to bottom). To the right, the numerical error on a logarithmic scale. $k$ is the permeability obtained by the simulation, $k_{HP}$ is the permeability given by the Hagen-Poiseuille equation. . . . .                              | 79 |
| 6.11 | Memory (top) and time (bottom) metrics for the permeability analyses of the cylindrical pipes model. . . . .                                                                                                                                                                                                                                                                                                                  | 81 |
| 6.12 | A periodic square packing of solid spheres. . . . .                                                                                                                                                                                                                                                                                                                                                                           | 83 |
| 6.13 | Periodic cell of a packed bed of spheres, a $1\text{mm} \times 1\text{mm} \times 1\text{mm}$ volume. The sphere is solid and has a diameter of $1\text{mm}$ . The empty volume around the sphere comprehends the pore space. . . . .                                                                                                                                                                                          | 83 |
| 6.14 | Convergence of the FEM-based permeability analysis for the model illustrated in Figure 6.13. To the left, the results for each image dimension. To the right, the numerical error on a logarithmic scale. $k$ is the permeability obtained by the simulation, $k_{ZH} = 2.5203\text{e}+03$ D is the permeability given by the semi-analytical solution based on the drag force coefficient from Zick and Homsy [154]. . . . . | 84 |
| 6.15 | Memory (left) and time (right) metrics for the permeability analyses of the sphere pack model. Notice that the simulations with the 3 and 2 vectors solvers were highlighted in the plots. . . . .                                                                                                                                                                                                                            | 85 |

|      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |    |
|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 6.16 | Evolution of permeability (left) and residual metrics (right) in regards to a normalized iteration count. $ \kappa^{(it)} - \kappa /\kappa$ represents the relative difference of the permeability at a given iteration from the final value, when the solver converged. Notice that the MINRES2 solver is considered for all of these curves, and the plot on the right depicts the behavior for the $600^3$ voxels sample only. Similar patterns were seen for the other samples. . . . . | 86 |
| 6.17 | $\mu$ CT scan of a Boise Idaho Brown sandstone. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                     | 87 |
| 6.18 | $\mu$ CT scan of a fibrous material. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                | 88 |
| 6.19 | Convergence of the normalized residual by the $L2$ norm criterion for the analyses in direction $x_1$ of the sandstone sample shown in Figure 6.17. Comparison between PCG, PMINRES and MINRES solvers. . . . .                                                                                                                                                                                                                                                                             | 90 |
| 6.20 | Magnitude of the velocity field $\ u_i\ $ [m/s] obtained via simulations on the sandstone sample. Results from the simulations in directions in $x_1$ on the left, and in $x_3$ on the right. . . . .                                                                                                                                                                                                                                                                                       | 90 |
| 6.21 | Convergence of the normalized residual by the $L2$ norm criterion for the analyses in direction $x_1$ of the fiber sample shown in Figure 6.18. Comparison between PCG, PMINRES and MINRES solvers. . . . .                                                                                                                                                                                                                                                                                 | 92 |
| 6.22 | Magnitude of the velocity field $\ u_i\ $ [m/s] obtained via simulations on the fiber sample. Results from the simulations in directions in $x_1$ on the left, and in $x_3$ on the right. . . . .                                                                                                                                                                                                                                                                                           | 92 |
| 6.23 | Two-dimensional slices of the $600^3$ voxels samples detailed in Table 6.9. Pore space in black, solid phase in white. . . . .                                                                                                                                                                                                                                                                                                                                                              | 94 |
| 6.24 | Comparison of permeability values obtained via simulation against laboratory measures . . . . .                                                                                                                                                                                                                                                                                                                                                                                             | 95 |
| 6.25 | Permeability $\times$ porosity plot for the batch of reservoir rock samples . . . .                                                                                                                                                                                                                                                                                                                                                                                                         | 97 |
| 6.26 | Average elapsed time per simulation $\times$ number of DOFs plot for the batch of reservoir rock samples. All models have $600^3$ voxels. . . . .                                                                                                                                                                                                                                                                                                                                           | 98 |
| 6.27 | Peak memory allocation in GPU per simulation $\times$ number of DOFs plot for the batch of reservoir rock samples. All models have $600^3$ voxels. Notice that different trends of allocation were highlighted for the PCG and PMINRES2 solvers. . . . .                                                                                                                                                                                                                                    | 99 |

- 
- 6.28 Peak memory allocation per simulation  $\times$  number of pore voxels plot for the batch of reservoir rock samples. Notice that memory metrics reported in Yang et al. [151] were plotted as well. The porosity of their samples was written next to their respective markers, for quick reference and comparison. 100
- 6.29  $964^3$  voxels image constructed from an Austin Chalk (AC) carbonate sample. The original  $600^3$  voxels volume from  $\mu$ CT scans was mirrored to achieve this image. . . . . 101
- 6.30  $1000^3$  voxels image constructed from a Bandera Brown (BB) sandstone sample. The original  $600^3$  voxels volume from  $\mu$ CT scans was mirrored to achieve this image. . . . . 102

# List of Tables

|     |                                                                                                                                                                                                                                                                                                                                                                                            |    |
|-----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 4.1 | Computational functions that allow straightforward vicinity (connectivity) characterization in a periodic structured grid, following a column-major numbering scheme. Note that $(R, C, L)$ are the numbers of rows, columns and layers in the grid, respectively, and $v_{id}$ is used to depict a vertex's index.                                                                        | 49 |
| 5.1 | Summary of the implemented solvers. From left to right, the columns after the solver identifier depict: the number of vectors of variables stored in memory, the amount of matrix-vector products, the amount of <b>daxpy</b> operations, the amount of <b>reduce</b> - or <b>dotprod</b> -like operations, and the amount of preconditioner-vector operations, all per iteration. . . . . | 69 |
| 6.1 | Hardware specifications for the simulations reported in Sections 6.1 and 6.2.                                                                                                                                                                                                                                                                                                              | 70 |
| 6.2 | Image-based models with different resolutions for the reproduction of the analytical benchmark (Figure 6.1). . . . .                                                                                                                                                                                                                                                                       | 72 |
| 6.3 | Image-based models with different resolutions for the reproduction of the cylindrical pipes (Figures 6.8 and 6.9). Notice that $\phi$ was used to represent the porosity of each sample, for conciseness. . . . .                                                                                                                                                                          | 78 |
| 6.4 | Image-based models with different resolutions for the reproduction of the packed bed of spheres (Figure 6.13). . . . .                                                                                                                                                                                                                                                                     | 83 |
| 6.5 | $\mu$ CT scans of real material samples. . . . .                                                                                                                                                                                                                                                                                                                                           | 87 |
| 6.6 | Effective absolute permeability and computational metrics for the analysis of the sandstone sample. . . . .                                                                                                                                                                                                                                                                                | 88 |
| 6.7 | Effective permeability and computational metrics for the fiber sample. . . .                                                                                                                                                                                                                                                                                                               | 91 |
| 6.8 | Hardware specifications for the simulations reported in this Section. . . .                                                                                                                                                                                                                                                                                                                | 93 |
| 6.9 | Sandstone and carbonate reservoir rocks studied as further validation of the implementations. All of these have experimental values for absolute permeability. Each of the considered microstructural images had $600^3$ voxels. Slices of each of these rocks are illustrated in Figure 6.23. . . . .                                                                                     | 94 |

---

|      |                                                                                                                              |     |
|------|------------------------------------------------------------------------------------------------------------------------------|-----|
| 6.10 | Batch of image-based models from 17 reservoir rock samples. . . . .                                                          | 96  |
| 6.11 | Characteristics of the gigavoxel samples. . . . .                                                                            | 101 |
| 6.12 | Permeability results, time and memory metrics for the analyses of the gi-<br>gavoxel samples. PMINRES2 was employed. . . . . | 102 |

# List of Abbreviations and Acronyms

|         |                                              |
|---------|----------------------------------------------|
| API     | : Application Programming Interface          |
| CG      | : Conjugate Gradient (method)                |
| CPU     | : Central Processing Unit                    |
| CUDA    | : Compute Unified Device Architecture        |
| DOF     | : Degree-of-freedom                          |
| EBE     | : Element-by-element                         |
| FDM     | : Finite Difference Method                   |
| FEM     | : Finite Element Method                      |
| FFT     | : Fast Fourier Transform                     |
| FVM     | : Finite Volume Method                       |
| GPU     | : Graphics Processing Unit                   |
| HPC     | : High-Performance Computing                 |
| LBM     | : Lattice-Boltzmann Method                   |
| μCT     | : Micro-Computed Tomography                  |
| MINRES  | : Minimal Residual (method)                  |
| NBN     | : Node-by-node                               |
| PCG     | : Preconditioned Conjugate Gradient (method) |
| PDE     | : Partial Differential Equation              |
| PMINRES | : Preconditioned Minimal Residual (method)   |
| RAM     | : Random Access Memory                       |
| SBS     | : Stencil-by-stencil                         |
| SIMD    | : Single Instruction Multiple Data           |
| SIMT    | : Single Instruction Multiple Thread         |
| SM      | : Streaming Multiprocessor                   |

# Contents

|          |                                                               |           |
|----------|---------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                           | <b>1</b>  |
| 1.1      | Motivation . . . . .                                          | 4         |
| 1.2      | Goals . . . . .                                               | 5         |
| 1.3      | Main expected contributions . . . . .                         | 6         |
| 1.4      | Outline . . . . .                                             | 7         |
| <b>2</b> | <b>Literature Review</b>                                      | <b>9</b>  |
| 2.1      | Computational homogenization . . . . .                        | 9         |
| 2.2      | Image-based permeability estimation in Petrophysics . . . . . | 12        |
| 2.3      | FEM simulations of Stokes flow in porous media . . . . .      | 14        |
| <b>3</b> | <b>Mathematical Modeling</b>                                  | <b>17</b> |
| 3.1      | A quick note on notation . . . . .                            | 17        |
| 3.2      | Homogenization . . . . .                                      | 19        |
| 3.3      | Effective permeability of porous microstructures . . . . .    | 24        |
| <b>4</b> | <b>Numerical Simulations for Permeability Estimation</b>      | <b>29</b> |
| 4.1      | Finite Element Method . . . . .                               | 31        |
| 4.2      | Iterative solvers . . . . .                                   | 36        |
| 4.3      | Matrix-free operations . . . . .                              | 41        |
| 4.3.1    | Node-by-node . . . . .                                        | 42        |
| 4.3.2    | Stencil-by-stencil . . . . .                                  | 44        |
| 4.4      | Image-based pore space meshing . . . . .                      | 46        |

|          |                                                                            |            |
|----------|----------------------------------------------------------------------------|------------|
| <b>5</b> | <b>Proposed implementation</b>                                             | <b>51</b>  |
| 5.1      | Massively parallel GPU programming . . . . .                               | 51         |
| 5.1.1    | A brief introduction . . . . .                                             | 51         |
| 5.1.2    | Memory . . . . .                                                           | 52         |
| 5.1.3    | The CUDA framework . . . . .                                               | 54         |
| 5.2      | An outlook of the developed code . . . . .                                 | 55         |
| 5.3      | Kernels for matrix-free operations . . . . .                               | 58         |
| 5.4      | CG and MINRES for Stokes flow in CUDA . . . . .                            | 62         |
| 5.5      | 3 and 2 vector alternatives . . . . .                                      | 64         |
| 5.5.1    | PCG3 and PMINRES3 . . . . .                                                | 64         |
| 5.5.2    | PCG2 and PMINRES2 . . . . .                                                | 66         |
| 5.6      | Summary of the implemented solvers . . . . .                               | 68         |
| <b>6</b> | <b>Experiments</b>                                                         | <b>70</b>  |
| 6.1      | Verification tests . . . . .                                               | 71         |
| 6.1.1    | Two-dimensional semi-analytical benchmark . . . . .                        | 71         |
| 6.1.2    | Three-dimensional analytical benchmark . . . . .                           | 77         |
| 6.1.3    | Three-dimensional packed bed of spheres . . . . .                          | 82         |
| 6.2      | Validation with $\mu$ CT scans . . . . .                                   | 86         |
| 6.3      | Permeability characterization of a batch of digital rock samples . . . . . | 93         |
| 6.3.1    | Gigavoxel analyses . . . . .                                               | 101        |
| <b>7</b> | <b>Final remarks</b>                                                       | <b>104</b> |
| 7.1      | Future work . . . . .                                                      | 106        |
|          | <b>References</b>                                                          | <b>108</b> |

# Chapter 1

## Introduction

Transport phenomena in composite media are a recurring matter of study in Materials Science and particularly in Petrophysics. In this context, an especially prominent topic is the characterization of transport properties of macroscopically homogeneous (or quasi-homogeneous) materials via homogenization analysis. To this end, in general, governing partial differential equations have to be solved at a microscopic domain to capture the effects of localized heterogeneities, so that effective constitutive tensors can be computed for the medium in question. As imaginable, this process usually relies on numerical (computational) simulations. There is widespread applicability for this sort of method, both in terms of physics phenomena to be studied, e.g. thermal conductivity [76, 81, 148], electrical resistivity [13, 37, 128], or permeability [3, 4, 128, 144]; and fields to benefit from it, such as Engineering [110, 145], Geology and Petroleum Sciences (Oil and Gas) [7, 8, 19, 66], and Biomechanics [10, 58]. This work is focused in the absolute permeability – single-phase fluid – of reservoir rocks, composites and fibrous materials. To put it simply, permeability is a linear measure that relates the flow rate through a porous medium to an applied pressure gradient or body force, i.e. it quantifies the flow due to something that “pushes” a fluid through the material.

Physical properties of material samples, including permeability, can be studied with practical experiments in a laboratory. The digital testing approach considered here, which is based on numerical homogenization, is a complementary alternative to study composite materials and porous media without a strong reliance on practical laboratory work or oversimplified empirical/analytical models. Also, simulations may aid in interpreting results and extrapolating loading and boundary conditions that would not be feasible in a laboratory. Physical testing is often expensive and time-consuming, due to difficult sample preparation, complex non-repeatable procedures, and the destructive nature of

various experiments. Computational studies, on the other hand, are automated and fully repeatable once a proper digital model has been acquired, and key information such as simulation logs and input/output data are properly tracked. In this regard, the advent of high-resolution imaging techniques has played a great role in pushing Materials Science forward, with resources as *micro-computed tomography* ( $\mu$ CT) [25, 26] being increasingly employed in “image and compute” works [7, 8, 12, 35, 109, 129, 130]. As put by Dvorkin [35], computational material characterization went from “do it because we can” to “do it because it’s needed”. However, there are challenges specific to digital testing, the main one being the sheer size of the required simulations. When dealing with  $\mu$ m resolution, it is common that representative volumes might demand clusters or supercomputers for image-based homogenization analyses to be conducted. In light of this, there is an open horizon for studies that focus on lightweight computational solvers and strategies, e.g. [6, 41, 81, 151], especially for permeability estimation.

The study of permeability in porous media can be seen as an important branch of homogenization that has its own peculiarities, issues, and applications. The fluid flow simulations within a porous voxelized microstructure are modeled via Stokes equations, a linearized form of the classical Navier-Stokes equations for laminar flow. In Petrophysics, this sort of modeling is frequently employed [3, 8, 19, 23, 62, 75, 97, 102, 128]. These governing equations yield saddle point problems, in contrast to elliptic equations that appear for other phenomena, which means that instability is often an issue in numerical simulations. Further, no-slip boundary conditions at the interface between pore and solid phases must be properly established, which is not always a simple task. Both of these points, among others, commonly lead researchers to work with pore-network- [18, 103, 150] or particle-based [19, 36] simulations for fluids, which is not so usual for heat transfer or elasticity, for instance. In general, in comparison to other phenomena, fluid flow simulations in porous media are significantly more demanding in terms of computational resources.

In light of the computational challenges associated with the required large-scale fluid flow simulations, attempts at memory-efficient strategies are common in the literature, often at the cost of increased processing time or possible compromise of representability with simplified numerical models. The Lattice-Boltzmann Method (LBM) is recurrently employed in Digital Petrophysics [6, 19, 36, 66, 114, 126], which is usually justified by its inherent parallelizable nature, and the relative ease it provides in applying boundary conditions and in exploring the voxel-based nature of the discrete domain to construct a particle grid [36]. It is also common to see works that employ the Finite Difference Method

(FDM) in this context [1, 97, 101, 102, 126, 144], and there have been works exploring the Semi-Implicit Method for Pressure-Linked Equations (SIMPLE) algorithm together with the Finite Volume Method [19, 131]. In a more general numerical homogenization scenario, considering other physical phenomena, Fast Fourier Transform (FFT)-based solvers are also prominently used [30, 70, 93, 127], and it appears in permeability analyses as well [83, 92, 96]. A common thread in some texts regarding these approaches is a mention that these are feasible alternatives to the Finite Element Method (FEM), under the assumption that the FEM would require excessive memory allocation. However, FEM-based solutions have been proposed for permeability [3, 4, 9, 75, 121, 151] and have been a staple in image-based homogenization of other physical properties, such as elasticity [45]. Further, as explored in recent work [10, 68, 81, 85, 116] for elasticity and heat transfer, the image-based FEM allows for extremely memory-efficient matrix-free implementations that are fit for massive parallelization in Graphics Processing Units (GPU), leveraging the hardware’s high throughput to compensate computational cost trade-offs. Bearing this in mind, there is a clear step to be taken: image-based matrix-free FEM in GPU for permeability estimation. This is the scope of this Thesis.

An important aspect of these Stokes flow simulations is that, ideally, only the pore space should be meshed, although there are works [9] that adopt numerical penalties to model the solid phase as a “very viscous” fluid, a strategy that will not be explored here, since it generates unnecessarily large meshes for low porosity samples, as is the case for most reservoir rocks. Having to extract a pore “skeleton” within an image-based mesh can be troublesome in regards to lightweight implementation approaches, as the tasks of storing and managing a large unstructured mesh often impose a heavy toll on memory. However, the seemingly natural impulse to store connectivity maps and sparse global matrices can be avoided, which aids in allowing large-scale analyses to be conducted in personal-use computers equipped with gaming-dedicated GPUs, for example. In light of this, alongside acceleration via parallelism, a careful assessment of the data structures to be stored, beyond conventional practices for the FEM, is essential.

Graphics cards are increasingly being employed in Scientific Computing, mostly as a mean to accelerate “embarrassingly parallel” tasks. The *Single Instruction Multiple Thread* (SIMT) paradigm, inherent to GPUs, favors intensive computational algebra operations such as vector scaling, vector-vector products, and sparse matrix-vector products, thus being applicable to numerical simulations in discrete spatial domains, especially if a fixed-grid discretization is possible. Note that this sort of hardware can be applied for unstructured meshes as well, although additional care is often required, for example,

to avoid possible data races and unbalanced workloads among different threads. Single- and multi-GPU applications for numerical simulation can be seen in academic works from the past decade to the present day, in various fields of computational science, e.g. [6, 10, 33, 41, 55, 68, 72, 81, 85, 95, 116, 118, 119], to cite a few. Even so, memory availability is still an issue, even for high-end consumer-grade graphics cards. Nowadays, relatively accessible GPUs – pricewise – that are significantly powerful in terms of throughput and performance seldom exceed 12 GB of available Random Access Memory (RAM). On the other hand, high-performance-dedicated cards, such as the ones within a Nvidia DGX<sup>TM</sup> Station, can surpass this threshold, but at a considerably higher cost – not meant for personal use. In the context of a digital material laboratory, or a digital Petrophysics laboratory, there is much potential to explore from personal-use – cheaper – GPUs to perform large-scale parallelizable simulations. In other words, to a certain extent, GPUs can serve as an alternative to clusters or supercomputers. To that end, to make use of the rich data that comes from imaging resources via  $\mu$ CT, memory allocation has to be thoroughly scrutinized and even some trade-offs with performance need to be reassessed, as in recomputing data when feasible. In that sense, matrix-free FEM implementations are well-suited, if not ideal, for this purpose.

## 1.1 Motivation

This work follows up on some of the author’s recent developments [81], where massively parallel matrix-free Conjugate Gradient (CG) solvers for homogenization via image-based FEM were explored. In the scope of past work, physical phenomena governed by elliptic equations were considered – heat transfer and linear elasticity – and the whole voxel-based regular grid was taken as the mesh. Two approaches were considered for the matrix-free operations, element-by-element (EBE) and node-by-node (NBN), and it was observed that NBN tends to be faster for structured meshes. Now, the idea is to expand on that previously developed expertise to simulate fluid flow in porous media, so that the absolute permeability of samples imaged via  $\mu$ CT, as shown in Figure 1.1, can be estimated. It is an author’s expectation, with this work, to show that representative permeability analyses, in terms of image dimension and resolution, can be conducted with limited hardware – personal computers equipped with relatively accessible GPUs – by employing a memory-efficient matrix-free FEM implementation. Graphics cards are used to ensure that this can be done in sensible time frames, i.e. within minutes or a few hours at most, instead of multiple hours or even days. It is interesting to notice that, even though the FEM

is not the most usual method of choice in the context of Petrophysics for permeability estimation, this work is in accordance with a prediction stated in 2013 by Blunt et al. [19], where it was mentioned that GPU methods were likely to flourish in pore-scale modeling in the years to come.

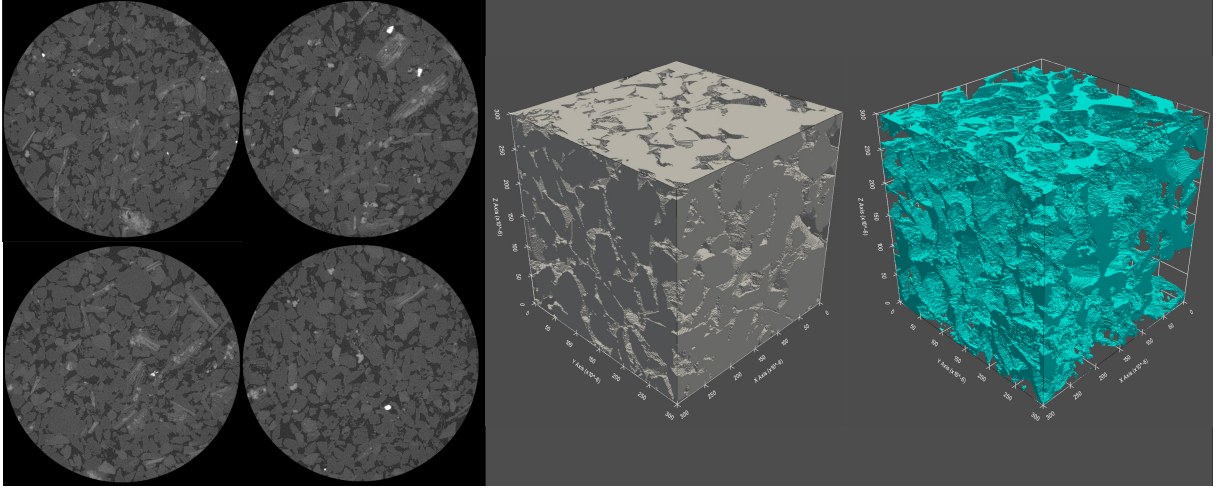


Figure 1.1: A sandstone sample. To the left, two-dimensional slices of the data obtained as a  $\mu$ CT scan. To the right, voxel-based depictions of segmented subvolumes extracted from the  $\mu$ CT scan. In grey, the solid phase, in blue, the pore space – this is where fluid flow simulations take place.

An important additional component is the intended application of the solvers to be developed here. In the scope of a Petrophysics research project, there is interest in associating computational results for Nuclear Magnetic Resonance (NMR) simulations, as shown in [17, 119], to the absolute permeability of reservoir rock samples imaged via  $\mu$ CT. NMR relaxation times are obtainable in an operational well, while permeability often is not. It is expected that Artificial Intelligence (AI) and/or Data Mining techniques may be employed to large datasets containing images and simulation responses, following approaches similar to those seen in papers as [31, 48], so that NMR data may provide additional insight into permeability estimation, especially for carbonates. To build these datasets, multiple large-scale image-based permeability analyses will be required, so a fast lightweight solver is vital.

## 1.2 Goals

The main objective of this work is the development of efficient – especially in terms of memory – massively-parallel algorithms for the computation of the absolute permeability of large porous voxelized microstructures, aiming at the characterization of material

samples imaged via  $\mu$ CT scans. The idea is to employ graphics cards to tackle “time vs. memory” trade-offs in matrix-free schemes by exploring their enhanced throughput for algebraic operations. The proposed lightweight simulations are to be carried out in consumer-grade hardware in reasonable time, i.e. no more than a few hours for the largest possible analyses. As per the Author’s prior experience in developing GPU solvers for the computation of thermal conductivity and elasticity of  $\mu$ CT data, it is assumed that a voxel-based FEM approach is suitable for permeability as well.

In order to meet this main objective, three key goals were devised:

1. Development of massively parallel matrix-free CG and MINRES solvers in GPU for image-based finite element analyses of Stokes flow in porous voxelized microstructures, following a *node-by-node* (NBN) approach. This involves studying the mathematical modeling of Stokes flow, understanding in detail and validating the numerical simulations, and iterating extensively in a process of implementing and optimizing the code, to be written “from scratch” in C and CUDA C++.
2. Verifying whether the FEM can in fact be realistically employed for absolute permeability estimation of large three-dimensional  $\mu$ CT scans with personal hardware in sensible timeframes, i.e. minutes or few hours at most. In practice, the idea is to assess time and memory metrics to validate the hypothesis.
3. Estimating the permeability of  $1000^3$  voxels images of sandstone and carbonate samples allocating less than 12 GB in RAM, which is the memory limit of the best GPU considered in this work. This is meant to serve as a milestone of the proposed implementation.

## 1.3 Main expected contributions

The knowledge, methods, and code developed in the scope of this Thesis are meant to convey to communities of digital (computational) materials scientists an efficient alternative approach for the characterization of absolute permeability from  $\mu$ CT scans in personal-use machines, like desktop computers. Bear in mind that, nowadays, the reality of the field is reliance on clusters, supercomputers, and clouds for large representative three-dimensional analyses of this kind – hundreds of millions of voxels or more. Indeed, there are various sorts of sophisticated numerical simulations to be made in this scope that do require expensive computational resources, but we aim to show that absolute permeability

computation is not necessarily one of them. By employing a not so common FEM-based strategy, an extremely lightweight (low on memory allocation) solution can be achieved, exploring GPUs for acceleration. Implementations that require only 3 or even 2 vectors of variables to be allocated in global memory are presented, and thus the usual 4 or 5 vector toll imposed by CG or MINRES solvers, respectively, is roughly halved. Note that, as per the code optimization nature of this endeavor, the devised algorithms are expected to be applicable to HPC environments as well, so that even larger analyses may be conducted in distributed frameworks.

The matrix-free implementation proposed in this Thesis expands on the EBE and NBN approaches seen in the literature on voxel-based FEM simulations in GPU. From what we could gather, this is the first work that details a massively parallel NBN implementation for Stokes flow in graphics cards, considering only the pore space within a structured grid. A stencil-by-stencil (SBS) extension of such NBN scheme is also proposed. Further, to keep track of the pores within an image, a straightforward data structure is devised to serve as a map from image to mesh indexes and vice-versa, so that arithmetic operations to “walk” on a periodic grid may be employed to recompute node connectivity on demand. This also allows for the identification of pore nodes and border nodes via simple numbering rules, which in turn allows for no-slip behavior at solid-pore interfaces to be separated seamlessly in code from the behavior within the pores, i.e. non-null velocity. It is stressed that the strategy devised for phase mapping in a grid can be applied for other physical phenomena as well, e.g. wave propagation simulations via FEM or FDM considering just the solid phase.

## 1.4 Outline

This text is organized in seven chapters. Chapter 1 comprehends this introduction. Chapter 2 depicts an academic background to this work, pinpointing its main contributions and where they are expected to fit in the dedicated literature. Related works and important sources that contributed to the author’s understanding of the field are properly referenced. Chapter 3 brings a detailed description of the adopted mathematical modeling, from a more generic homogenization problem to the homogenization of absolute permeability of porous media. In Chapter 4, the proposed numerical simulations are outlined. A step-by-step development of the adopted numerical modeling for approximate solutions of the governing partial differential equations via the FEM is presented. To this end, weak-form expressions are constructed by employing the weighted residual method.

In this Chapter, it is also shown how Krylov subspace solvers, CG and MINRES, are employed for the large sparse systems of algebraic equations that arise from the FEM, also detailing how different matrix-free approaches can be embedded in these. Furthermore, the adopted image-based pore meshing strategy is presented. In Chapter 5, the proposed implementation is discussed in detail. The idea is to present the reasoning behind the key decisions to the actual development of the code. It is shown how the CPU/GPU heterogeneous computing model is explored for the massive parallelization of the required matrix-free algebraic operations. Further, it is shown how alternative implementations of the CG and MINRES solvers can be made to allocate fewer vectors in the global memory. In doing so, 3 and 2 vector solutions are proposed. Chapter 6 showcases computational experiments with the implemented solvers, its results and metrics. In this Chapter, verification exercises are shown, where numerical results are compared against analytical and semi-analytical predictions for benchmark models. Then, validation experiments are detailed, where effective absolute permeability tensors are obtained for  $\mu$ CT scans of a fibrous composite and of a sandstone sample, having the results checked with reference values from the literature. Finally, results and metrics are reported for the permeability analyses of a batch of large-scale  $\mu$ CT samples of reservoir rocks, including sandstones and carbonates, and two simulations with gigavoxel samples are presented. At last, Chapter 7 presents some final concluding comments and reflections on possible future developments.

# Chapter 2

## Literature Review

### 2.1 Computational homogenization

The characterization of microscopically heterogeneous materials via equivalent homogeneous media is a staple of Materials Science. Ancient and widespread materials, such as concrete and cast iron, are often perceived as homogeneous macroscopically, even though they have intricate heterogeneous microstructural arrangements that affect their physical properties. The mathematical modeling and procedures to calculate *homogenized* properties are encompassed in a topic referred as *homogenization*. Since the 1960's there have been research papers on systematic, mathematical, approaches to model homogenization problems for mechanical properties, which can be seen in classical work by authors such as Hill [57] and Hashin and Shtrikman [53]. In the following decades, the field evolved with works that focused on the mathematical modeling of various physical phenomena, as thermal and electrical conductivity, e.g. Perrins (1979) [111], and permeability, e.g. Drummond and Tahir (1984) [32] and Cioranescu et al. (1996) [23]. Also on special considerations regarding the domain, such as the presence of pores, as in Cioranescu and Paulin (1979) [24]. The goal often was to search for analytical solutions for relatively “well-behaved” domains that could be used to approximately model the microstructure of real materials. These studies were essential to solidify the mathematical basis of homogenization, supporting the later numerical developments, and provided valuable benchmarks and rules-of-mixture [2, 52] that can be used to validate computational results and provide quick and fairly accurate insight into the physical properties of composite or porous materials (or upper and lower bounds, at least).

An important limitation remained from the more theoretical works: the representation of complex microstructures. From the 1990's onward, the advent of imaging technologies

allowed the digital representation of material samples with microscopic resolution (voxel size in  $\mu\text{m}$ ), leading to the search for computational – numerical – approaches of homogenization to handle the naturally discrete image-based domains. The application of  $\mu\text{CT}$  imaging as a tool in Materials Science has been thoroughly explored in the work of Cnudde et al. [25, 26]. Regarding the numerical simulations, Guedes and Kikuchi (1990) [49] and Brockenbrough et al. (1991) [20] were some early works to explore the employment of the FEM in homogenization analyses for the mechanical properties of composites. In 1994, Schwartz et al. [128] detailed a numerical modeling for the characterization of transport properties, including permeability, of pixel-based samples taken from  $\mu\text{CT}$  scans. In 1995, Garboczi and Day [45] presented an image-based FEM solver for the effective elasticity of voxelized domains. In 1999, Alcocer et al. [4] proposed a FEM-based numerical approach to compute the permeability of periodic porous media. From the 2000's to the present day, the literature on image-based computational homogenization is extensive, with works exploring the FEM [9, 29, 37, 76, 77, 80, 93, 100, 109, 139, 148], the FDM [1, 13, 97, 101, 102, 126, 144], the LBM [6, 62, 66, 114, 126], and the FVM [19, 39, 40, 129, 130, 131] as means to solve the governing partial differential equations in the microstructural domain. The computational challenge that arises from the required large-scale simulations is often addressed in the literature, and memory-efficient strategies of implementation are certainly explored, but the common solution to both memory and performance thresholds usually aims at distributed high-performance computing in clusters or supercomputers. The matter of performing analyses of large images –  $400^3$  voxels or more – with personal-use hardware has been neglected more often than not, largely due to technological limitations that are being tackled recently. It was predicted in the 2010's that heterogeneous computing with GPUs was likely to gain space in this context, as stated in Blunt et al. (2013) [19].

A particularly recurrent method that was not mentioned in the previous paragraphs is the FFT-based solution proposed by Moulinec and Suquet [98], later formalized in Michel et al. [93]. It has been employed in the context of permeability analyses [83, 92, 96], although not as often as in other phenomena, as it can be seen in review texts on the subject, e.g. [127]. In this method, the problem at hand is rewritten as a set of Lippmann-Schwinger equations, allowing the microscale fields to be obtained via a convolution integral in the microscale domain – algebraic operations in transformed space, and thus it requires a naturally matrix-free scheme. In contrast, the FEM, for example, relies on large sparse matrices that may not be stored in memory if their coefficients are recomputed on demand. The possibility to employ efficient and widespread FFT

implementations and the lack of large matrices are usual arguments in favour of the Moulinec-Suquet approach [83, 93]. This is a well-suited methodology for voxel-based domains, even though it may face numerical trouble and Gibbs phenomenon due to high phase contrast, an issue that is tackled in papers that focus on this strategy, as in [64, 83]. The FFT-based scheme was proposed as a feasible alternative in terms of memory requirements, as opposed to other – more conventional – methods. However, with the improvement of computational hardware and increased parallelism capabilities, matrix-free strategies for other numerical methods are being gradually more explored in the scope of numerical homogenization nowadays [6, 76, 77, 151].

The matrix-free FEM has been around since the 1980's, as it can be seen in the early work of Hughes et al. (1983) [61], where the *element-by-element* (EBE) approach was proposed. In the following years, the works of Carey and Jiang (1986) [21] and Erhel et al. (1991) [38] expanded on the matter. In the last decade, within a scenario of heterogeneous computing with GPUs and high-performance computing in clusters, EBE FEM solvers resurfaced in the literature [67, 68, 85, 86], mostly in papers regarding topology optimization [33, 87, 116] and biomechanics simulations [10]. Fixed-grid, or voxel-based, meshes were shown to benefit largely from this sort of approach [10, 67, 82, 85]. In this context, the idea of working with *node-by-node* (NBN) [68, 85] approaches gained some traction. In the scope of image-based homogenization, there are works such as Pereira et al. [109], Liu et al. (2019) [76, 77], and Yang et al. (2019) [151] that documented the employment of matrix-free FEM strategies for material characterization, employing parallelism techniques in CPU and/or distributed hardware. In 2022, the author of this Thesis presented EBE and NBN Preconditioned Conjugate Gradient (PCG) solvers in GPU for the homogenization of thermal conductivity and linear elasticity of voxelized microstructures via the image-based FEM, in Lopes et al. [81]. The goal of the present work, recall, is to expand on that memory-efficient and fast methodology for the characterization of absolute permeability.

It is also worth mentioning that other methods such as the Transformation Field Analysis (TFA), by Dvorak (1992) [34], and Nonuniform Transformation Field Analysis (NTFA), by Michel and Suquet (2003) [94], have been employed in homogenization scenarios where relatively complex constitutive mechanical properties are of interest. For example, there are works focused on TFA or NTFA where viscoelasticity is admitted concurrently with nonlinearity due to plasticity or damage, as in [14, 84, 120]. Herein, the presented developments apply to linear phenomena, although they may be later expanded for nonlinear simulations.

Currently, the state-of-the-art of computational homogenization, in regards to applications, is focused on two major topics: (i) biological media and composite material design and analysis; and (ii) reservoir rock characterization. Both rely on imaging techniques, often  $\mu$ CT, for the digital microstructural modeling of the studied media. In (i), there is a growing interest in fiber-reinforced composites [41, 70, 89, 90], porous fiber and woven materials [80, 105, 106, 107, 129, 130], and biological media such as bone [10, 43, 67]. In (ii), authors are mostly interested in the pore-scale modeling of transport phenomena in  $\mu$ CT scans of rock samples, which can be seen in academic works that usually fall under the umbrella of Digital Petrophysics [7, 8, 19, 25, 35, 123, 126, 124, 125, 137] – all published since the last decade. It is commonly stated in works like these that the image-based analyses of representative (large) samples are delegated to clusters. Here, all the computations are performed in personal computers or workstations equipped with a single GPU. The developments presented in this Thesis are primarily intended to be applied in (ii), but they are applicable to (i) as well.

## 2.2 Image-based permeability estimation in Petrophysics

The capacity of a material to allow fluids to permeate through its porous microstructure has been a matter of study in Soil Mechanics for centuries. In 1856, Darcy proposed a linear constitutive law, based on empiricism, to quantify the flow rate through porous media given a pressure drop. This mathematical model is well known as Darcy’s law. It was later rigorously derived from the Navier-Stokes equations admitting laminar flow, i.e. Stokes flow, in 1986 by Whitaker [146], who aided in formalizing the notion that permeability is described by a constitutive tensor. Due to the famous complexity in finding solutions for the governing differential equations of fluid flow, researchers have historically searched for simplified modeling strategies that would allow for fair approximations to be obtained in some cases of interest. As early as in the 1920’s and 30’s there were texts regarding the fluid flow in, and thus the permeability of, domains composed of packed spherical solids, as in the seminal articles by Kozeny [73] and Carman [22]. These laid the foundations for the famous semi-empirical Kozeny-Carman equation, which is still used as a practical expression for the permeability of sandstone samples, for example. There were later works that expanded on this topic, such as Drummond and Tahir (1984) [32] that described the laminar flow in porous media composed of a periodic arrangement of parallel cylinders. Even so, such approaches are not generic and may not provide acceptable

results for all porous media. The application of numerical homogenization procedures to estimate the permeability of real microstructures remains a challenge to this day, largely because of the high computational cost of the required simulations. The “image and compute” paradigm has grown in this context since the 2000’s [7, 8, 19, 35, 36, 102], but the recurrent FFT- and FEM-based approaches seen in the homogenization of elasticity, for instance, were not so prominently employed in the homogenization of permeability.

The field of Digital Petrophysics pushed forward on alternative methods to explore imaging technologies for permeability estimation. Measures of absolute<sup>1</sup> and relative<sup>2</sup> permeability of reservoir rocks are essential for oil and gas operations, and so, since the dawn of image-based material characterization in the 1990’s, there was interest in making feasible the estimation of these properties from  $\mu$ CT scans despite the limitations of the available hardware and methods. A strategy based in pore-networks gained popularity in the 1990’s and 2000’s, as it can be seen in articles by Øren et al. (1998) [103] and Blunt et al. (2001) [18]. For further reading, an in-depth review on the topic was made in 2016 by Xiong et al. [150]. The core idea behind this method is simplifying the physics simulations by substituting the porous microstructure in question by an equivalent network of spherical pores connected by pipes, where fluid flow can be straightforwardly modeled. Then, an important computational challenge is finding such network, which can be seen as a directed graph. Multiphase oil-water scenarios can also be described by pore-networks with wettability-based simplified models [18].

From the early 2000’s onward, particle-based simulations have been quite common in the dedicated literature [6, 62, 66, 114, 126], with the LBM as the most employed method. In works that review the state-of-the-art of image-based permeability estimation from the recent decades, it was recurrently mentioned as one of the most appropriate numerical methods for such analyses, as in Dvorkin et al. (2009) [36]. In this approach, the digital microstructure can be considered “as is”, without the construction of simplified models. It is interesting to notice that the choice for LBM is often justified by its supposed efficiency when dealing with the pore space within a voxel-based mesh, even though there can be very efficient FDM or even FEM implementations in this context as well. The FDM itself has been employed for permeability estimation for decades, as it can be seen in the 1990 paper by Adler et al. [1], and in more recent works such as Gerke et al. [47], Mostaghimi et al. [97], and Victor et al. [144], to cite a few. Throughout the years since the 2000’s, the method has continuously been explored, with authors often bringing up

---

<sup>1</sup>single-phase fluid flow

<sup>2</sup>multi-phase fluid flow

a similar argument that the fixed-grid nature of voxel-based meshes fits naturally with the method, making it a realistic alternative to the LBM. Once more, it stands out that the FEM, less explored in this context, can also be efficiently implemented to consider such regular structured meshes, as it will be shown in this Thesis. The FVM has also been recurrently employed with the SIMPLE algorithm [19, 131], often via the popular OpenFOAM framework.

New matters of study are surfacing in this scope nowadays, even though there still is interest in improving the fluid flow simulations in porous media, especially in terms of computational efficiency, as it can be seen in works as Victor et al. (2016) [144], Silva and Ginzburg (2016) [133], Alpak et al. (2018) [6], Sun and Wong (2018) [137] and Lesueur et al. (2022) [75]. The focus of academic efforts in Petrophysics is currently being shared with topics as the statistics-based modeling of uncertainty and representability of digital rock characterization, as in the various papers published by Saxena et al. since 2017 [123, 124, 125, 126] and other works as Wollner et al. (2018) [147]; and the estimation of permeability directly from  $\mu$ CT scans of rock samples via AI methods instead of numerical simulations, mostly with convolutional neural networks, as in Tembeley et al. (2021) [141], Iklassov et al. (2022) [63], Zhang et al. (2022) [153] and dos Anjos et al. (2023) [31], with particular interest in carbonates. Naturally, for both of these applications, lots of permeability analyses are required for the construction of training datasets. This Thesis aims at presenting novel fast and lightweight massively parallel FEM-based solvers for image-based permeability estimation that can be run in personal machines, so that new studies may not rely exclusively on powerful clusters, supercomputers or even cloud computing. Recall that, although all of these are surely beneficial, they can be considerably expensive. In this regard, the goal here is to provide an alternative method, with potential to reach wide communities of researchers in Digital Petrophysics and Materials Science.

## 2.3 FEM simulations of Stokes flow in porous media

FEM-based modeling of low turbulence fluid flow in porous media has been explored in the literature for decades, as in the early works of Saeger et al. (1995) [121] and Alcocer et al. (1999) [4, 5]. Let it be clear that the relative low usage of the method in image-based permeability estimation is not due to lack of knowledge or tools in regards to the numerical modeling of the Stokes problem, but to the elevated computational cost, especially in terms of memory, associated with its conventional practices of implementation. Namely, storing an unstructured mesh to describe a discretization of the pore space for a large

three-dimensional  $\mu$ CT scan is considerably expensive. Meshes built from representative volumes, usually composed of  $400^3$  voxels or more, can fill up the RAM of workstations, requiring hundreds of gigabytes to be stored. Even so, the FEM has appeared in recent papers that discuss the homogenization of permeability, as in Akanji et al. (2010) [3], Qi et al. (2013) [115], Andreassen and Andreassen (2014) [9], Yang et al. (2019) [151], Vianna et al. (2020) [142] and Lesueur et al. (2022) [75]. Notice that [9] and [142] are educational works that present concise two-dimensional analyses. In Akanji et al. [3], they deal with relatively small simulations in two-dimensional space. On the other hand, in Lesueur et al. [75], three-dimensional voxel-based analyses are considered, but they propose an unfitted boundary technique to allow for fairly accurate results to be obtained with coarsened meshes. Among these, the article by Yang et al. [151] is the closest related work to this Thesis, as it shares a similar scope. In that paper, the authors presented a massively parallel matrix-free FEM implementation for the absolute permeability of  $\mu$ CT scans of rock samples. They have performed an important validation of the FEM in this context, showing that it is capable of generating similar results to the LBM, and with lower memory allocation. However, by “massively parallel” the authors meant CPU parallelization via OpenMP and distribution via *Message Passing Interface* (MPI) resources, i.e. their solver is meant to be run in CPU clusters. Further, the simulations at the microscale domain were numerically modeled via a time-dependent stabilised scheme based on the Navier-Stokes equations. In the present Thesis, it is recalled, a matrix-free FEM solver for image-based permeability estimation is implemented in CUDA to be run in personal-use GPUs, and the physics phenomenon at the microscale is assumed to be governed by a stationary Stokes flow model, as conventional in homogenization analyses of Darcy’s law. It will be shown that the proposed scheme yields results in good agreement with analytical/empirical models and experimental data, requiring less memory to be allocated.

Finally, it is important to mention that Stokes flow is being studied with the FEM for other applications. For instance, Talischi et al. (2014) [138] presented a method for the simulation of incompressible laminar flow with polygonal finite element meshes in two-dimensional space. Their developments were later applied for flow-based topology optimization analyses in Pereira et al. (2016) [110]. It is also interesting to notice that works focused on Stokes-Brinkman-Darcy models, where low turbulence flow is coupled with porous media at the same scale, often make use of the FEM for their numerical simulations, as in Arbogast and Brunson (2007) [11], Karper et al. (2008) [65], Xie et al. (2008) [149] and Silva and Ginzburg (2016) [133]. All in all, it is stressed once more,

---

the FEM is an interesting numerical method to be explored in fluid modeling, and it has been somewhat avoided in the specific scope of image-based permeability homogenization due to the large-scale nature of the required simulations. This Thesis aims to show that nowadays it can be employed to that end in personal machines, in realistic sensible time frames and without excessive memory allocation.

# Chapter 3

## Mathematical Modeling

### 3.1 A quick note on notation

Throughout this Chapter, index notation will be used in the presentation of the adopted mathematical modeling. It will be employed for every expression that deals with fields defined in a continuous domain. A brief refresher on the notation approach will be presented in this Section, based on introductory texts that can be found in books and course notes on continuum mechanics as [15, 88]. Readers that are already familiarized with index notation may skip directly to Section 3.2.

Index notation is an alternative to usual vector notation that aims at encompassing tensor operations into concise and clear expressions. One of its major advantages is the objective statement of a term's rank in expressions that deal simultaneously with scalar, vector and tensor fields. The main idea is that each subscript index denotes an  $n$ -dimensional set associated with directions in an orthonormal basis. Here, all of the modeling assumes domains in  $\mathbb{R}^3$ . In light of this, scalar, vector, and tensor fields can be defined as follows:

$$\begin{aligned} \text{(scalar – rank 0)} \quad s &\equiv s, \\ \text{(vector – rank 1)} \quad v_i &\equiv \begin{bmatrix} v_1 \\ v_2 \\ v_3 \end{bmatrix}, \\ \text{(tensor – rank 2)} \quad t_{ij} &\equiv \begin{bmatrix} t_{11} & t_{12} & t_{13} \\ t_{21} & t_{22} & t_{23} \\ t_{31} & t_{32} & t_{33} \end{bmatrix}. \end{aligned} \tag{3.1}$$

Notice that the definitions in Equation 3.1 carry for tensors with a rank higher than 2.

In index notation, a single expression can be used to represent multiple equations. The key features that make this possible in a concise manner are free indexes and Einstein's summation convention. A free index is one that does not appear repeatedly in any additive term in both sides of an equation. Indexes that are repeated depict a sum over all possible values of those indices, by Einstein's convention. For example, if there is one free index in an expression, there are three equations ( $\mathbb{R}^3$ ), as in

$$a_i = \kappa b_i \equiv \begin{cases} a_1 = \kappa b_1 \\ a_2 = \kappa b_2 \\ a_3 = \kappa b_3 \end{cases} . \quad (3.2)$$

Analogously, if there are two free indexes, and one index that repeats itself, there are nine equations like so:

$$a_{jk} = \kappa_i b_{ijk} \equiv \begin{cases} a_{11} = \kappa_1 b_{111} + \kappa_2 b_{211} + \kappa_3 b_{311} \\ a_{12} = \kappa_1 b_{112} + \kappa_2 b_{212} + \kappa_3 b_{312} \\ a_{13} = \kappa_1 b_{113} + \kappa_2 b_{213} + \kappa_3 b_{313} \\ a_{21} = \kappa_1 b_{121} + \kappa_2 b_{221} + \kappa_3 b_{321} \\ a_{22} = \kappa_1 b_{122} + \kappa_2 b_{222} + \kappa_3 b_{322} , \\ a_{23} = \kappa_1 b_{123} + \kappa_2 b_{223} + \kappa_3 b_{323} \\ a_{31} = \kappa_1 b_{131} + \kappa_2 b_{231} + \kappa_3 b_{331} \\ a_{32} = \kappa_1 b_{132} + \kappa_2 b_{232} + \kappa_3 b_{332} \\ a_{33} = \kappa_1 b_{133} + \kappa_2 b_{233} + \kappa_3 b_{333} \end{cases} , \quad (3.3)$$

and so on for higher counts of either free or repeated indexes. With these rules in mind, the following expressions are presented. These are the algebraic operations with vectors and tensors that will appear in the following mathematical modeling:

$$\begin{aligned} \text{(vector scaling)} \quad & u_i = s v_i, \\ \text{(vector-vector inner product)} \quad & s = u_i v_i, \\ \text{(tensor-vector product)} \quad & u_i = t_{ij} v_j, \\ \text{(tensor-tensor inner product)} \quad & s = t_{ij} \tau_{ij}. \end{aligned} \quad (3.4)$$

Notice that the variables in Equation 3.4 have no particular physical meaning at this point. These are just examples.

Another important resource provided by index notation is an alternative representa-

tion of differential operators. Here, instead of the usual  $\nabla$  operator, subscript commas are employed to denote spatial derivatives like so

$$a_{,i} \equiv \nabla a \equiv \begin{bmatrix} \partial a / \partial x_1 \\ \partial a / \partial x_2 \\ \partial a / \partial x_3 \end{bmatrix}. \quad (3.5)$$

As for the algebraic operations, Einstein's convention is valid for indexes after the comma, so repeated differential indexes depict sums. Bearing these notions, the differential operators that will be used throughout the mathematical modeling of this Thesis are:

$$\begin{aligned} \text{(gradient of a scalar field)} \quad s_{,i} &\equiv \nabla s, \\ \text{(divergent of a vector field)} \quad v_{i,i} &\equiv \nabla \cdot \mathbf{v}, \\ \text{(divergent of a tensor field)} \quad \tau_{ij,j} &\equiv \nabla \cdot \boldsymbol{\tau}, \\ \text{(laplacian of a scalar field)} \quad s_{,ii} &\equiv \nabla \cdot \nabla s = \nabla^2 s, \\ \text{(laplacian of a vector field)} \quad v_{i,jj} &\equiv \nabla \cdot \nabla \mathbf{v} = \nabla^2 \mathbf{v}. \end{aligned} \quad (3.6)$$

Finally, a mathematical tool that is recurrently employed with index notation will be adopted here as well: the Kronecker delta. It consists of a rank 2 tensor that only has non-null values for its “diagonal” terms, similarly to an identity matrix:

$$\text{(Kronecker delta)} \quad \delta_{ij} = \begin{cases} 1, & \text{if } i = j \\ 0, & \text{otherwise} \end{cases} \equiv \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}. \quad (3.7)$$

This rather straightforward definition allows for scalar fields to be clearly associated with tensor fields in a mathematical expression.

## 3.2 Homogenization

Homogenization analyses aim to describe constitutive behaviors of a given medium by studying its microstructural arrangement and response to controlled stimuli. As implied, a multiscale problem is posed, as it is admitted that at least two well-defined scales exist: a microscale, in which there are distinguishable heterogeneities; and a macroscale, where these heterogeneities cannot be perceived individually and are assumed to be embedded in *effective*<sup>1</sup> (or *homogenized*) constitutive tensors. In essence, the goal of these analyses is to accurately approximate the physical properties at any point of the macroscale

---

<sup>1</sup>reduced measure of any field at the microscale that can be taken as an equivalent macroscale value.

domain via some form of reduction of the fields contained at the microscale. This sort of procedure appears frequently in Computational Materials Science applications, mostly to characterize material samples through digital testing and computational simulations [8, 9, 12, 109, 129, 130]. It can also be seen in material design and topological shape optimization studies [110, 145]. Note that, even though there are two distinct scales, for the purposes of homogenization as it is considered here, simulations are run only at the microscale. Even so, this method can be applied to multiscale simulations, as in [70], for example.

To model a homogenization problem, three key assumptions are required:

1. A constitutive law at the macroscale.
2. A reduction rule to associate microscale fields to macroscale fields.
3. Governing equations and boundary conditions at the microscale.

The first assumption refers to the constitutive behavior under study, e.g. Fourier's law for thermal conductivity, Ohm's law for electric conductivity, Hooke's law for linear stiffness, or Darcy's law for permeability. The second assumption states how fields defined at the microscale can be transformed to the macroscale. Finally, the third assumption describes the physics phenomenon of interest, i.e. the set of PDEs to be solved – numerically – at the microscale. In this work, these assumptions are made in accordance with classical computational homogenization literature, such as [93]. The adopted mathematical modeling can be formally described as follows.

Let  $Y \subseteq \mathbb{R}^3$  be an unbounded homogeneous macroscale domain such that the same effective material phase  $C$  is defined for any point contained in  $Y$ . Let  $\Omega \subset \mathbb{R}^3$  be a cuboid bounded heterogeneous microscale domain such that  $\Omega$  can be admitted as a representative unit cell of a point  $y_i \in Y$ , i.e.  $\Omega$  is a valid microscale representation of  $C$  at  $y_i$ . Notice that, as  $Y$  is assumed to be homogeneous, it follows that  $\Omega$  must be a representative unit cell of any point contained in  $Y$ . Consequently,  $\Omega$  comprehends a *periodic* cell within an unbounded microscale domain, i.e. it repeats itself in all three directions of the canonical basis in  $\mathbb{R}^3$ , composing a Cartesian grid of cells. In Figure 3.1, the domains  $Y$  and  $\Omega$ , and the relation between them, are illustrated for a two-dimensional problem, for conciseness.

Let

$$a_i = E_{ij}b_j, \quad \forall y_i \in Y, \quad (3.8)$$

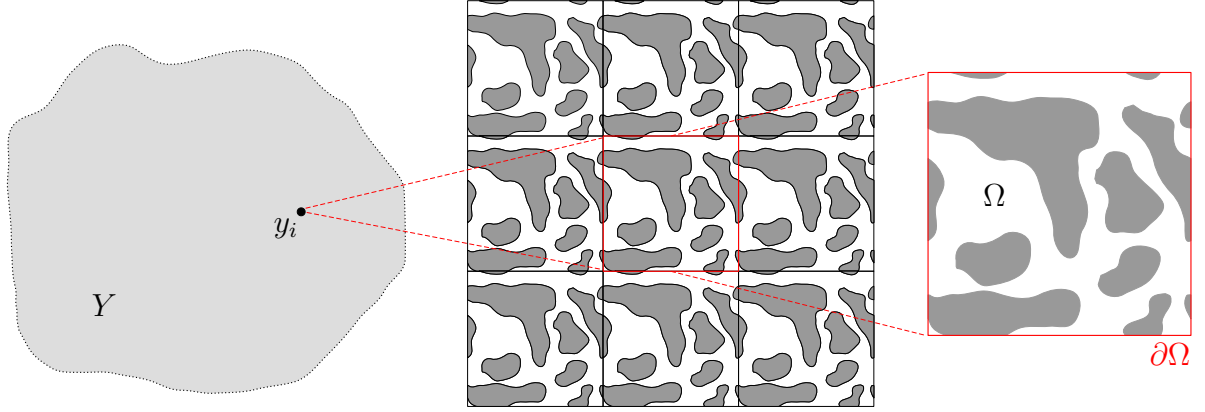


Figure 3.1: An unbounded homogeneous macroscale domain  $Y$  with its material phase being depicted by a heterogeneous microscale periodic unit cell  $\Omega$ .

be the constitutive law at the homogeneous macroscale. This is a fairly common representation of linear material behavior, where  $a_i$  and  $b_j$  would be kinetic-like and kinematic-like vector fields related by a second-order (rank 2) constitutive tensor  $E_{ij}$ . There are phenomena that require higher rank tensors, as in linear elasticity, but the principle in Equation 3.8 generally holds in those cases, thus it can be admitted without loss of generality for the purposes of this work. At first, all terms in this Equation are unknown. However, recall that the goal of homogenization analyses is to obtain  $E_{ij}$ , so it must be computed in terms of  $a_i$  and  $b_j$ . In that sense, the idea is to impose either of those and calculate the remaining other via governing equations at the microscale, which are described further ahead. As implied, a “*micro to macro*” rule has to be established. Here, following standard practice [57, 93], volume averages are adopted:

$$\langle \phi \rangle = \frac{1}{|\Omega|} \int_{\Omega} \phi \, d\Omega, \quad \forall y_i \in Y, \quad (3.9)$$

where  $\phi$  is any given field defined in  $\Omega$ . The angle brackets notation  $\langle \phi \rangle$  is employed to denote the volume average of  $\phi$ , hereby taken as the effective value of  $\phi$  at the macroscale domain  $Y$ .

Notice that, as  $E_{ij}$  is of a higher rank than  $a_i$  and  $b_j$ , multiple boundary value problems must be solved at the microscale. These multiple problems are chosen so that specific coefficients in  $E_{ij}$  can be isolated and calculated independently. This can be done straightforwardly by imposing elementary (or unitary) values for  $b_j$ , as in

$$\langle b_j \rangle^{(k)} = \delta_{jk}. \quad (3.10)$$

where  $\delta_{jk}$  is the Kronecker delta (Equation 3.7), and the notation  $\langle b_j \rangle^{(k)}$  is used to re-

present the imposed effective unitary value of  $b_j$  in direction  $x_k$ . It follows, then, that Equation 3.8 can be rewritten so that an expression for  $E_{ij}$  is outlined:

$$\langle a_i \rangle^{(k)} = E_{ij} \delta_{jk} \therefore E_{ik} = \langle a_i \rangle^{(k)}, \quad (3.11)$$

where, analogously,  $\langle a_i \rangle^{(k)}$  is the effective value of  $a_i$  obtained from solving the governing equations in  $\Omega$  when  $\langle b_j \rangle^{(k)}$  was imposed – a process commonly referred as “an analysis in direction  $x_k$ ”. Notice that for each  $k \in \{1, 2, 3\}$ , a column (referring to matrix notation) of the effective tensor is computed. Alternatively, it is common to see other authors express  $E_{ij}$  in terms of an energy measure that encompasses the constitutive law, by considering Hill’s lemma [57, 93]. In doing so, Equation 3.11 can be substituted by

$$E_{kl} = \langle b_i \rangle^{(k)} \langle a_i \rangle^{(l)} \equiv \langle b_i^{(k)} a_i^{(l)} \rangle, \quad (3.12)$$

where, the rightmost term  $\langle b_i^{(k)} a_i^{(l)} \rangle$  denotes volume averages of the inner product between  $a_i$  and  $b_i$  in  $\Omega$ , considering analyses in directions  $x_k$  and  $x_l$ . Notice that this expression allows for individual coefficients of  $E_{ij}$  to be singled out. It is important to stress that Equations 3.11 and 3.12 are conceptually equivalent. Both are presented here to reflect what readers may find in the literature on this topic. In the scope of this Thesis, homogenized tensors will be calculated accordingly to Equation 3.11, as usual in works about permeability [9, 128, 142]. The only component left, then, are the governing equations and boundary conditions.

Let  $P$  be the set of governing PDEs at the microscale. For reference, consider a scalar potential problem, modeled by a Laplace-like equation:

$$\left. \begin{array}{ll} \text{(flux conservation)} & a_{i,i} = 0 \\ \text{(constitutive law)} & a_i = \kappa_{ij} b_j \end{array} \right\} (\kappa_{ij} b_j)_{,i} = 0, \quad \forall x_i \in \Omega, \quad (3.13)$$

where  $\kappa_{ij}$  denotes a generic anisotropic constitutive tensor,  $a_i$  would represent a vector field that describes flux, and  $b_j$  would commonly be associated with the gradient of an unknown scalar field  $u$ , often in the opposite direction, as in

$$b_j = -u_{,j}, \quad (3.14)$$

all of which are defined for any point  $x_i \in \Omega$ . In heat transfer problems,  $u$  can be thought of as a temperature field and  $a_i$  as a heat flux field, or, in electrostatics,  $u$  can represent an electric potential field and  $a_i$  a current density field. In both cases,  $\kappa_{ij}$  can be referred as a conductivity tensor. Notice that a constitutive law is embedded in Equation 3.13, which

is valid at the microscale. As detailed in [93], in homogenization analyses, the gradient field  $b_j$  can be described as a known (applied) constant average term compensated by a fluctuation term that accounts for the local heterogeneities in  $\Omega$ , as in

$$b_j(x_i) = \overline{b_j} + b_j^*(x_i), \quad \forall x_i \in \Omega, \quad (3.15)$$

where  $\overline{b_j}$  is the average gradient and  $b_j^*$  is the gradient fluctuation, such that  $\langle b_j \rangle = \overline{b_j}$  and  $\langle b_j^* \rangle = 0$ . Recalling that  $\Omega$  is a periodic cell, in order to meet these definitions, it must be ensured via boundary conditions that  $b_j^*$  is periodic and that outwards flux  $a_i \hat{n}_i$  at the border  $\partial\Omega$  is anti-periodic, where  $\hat{n}_i$  is an outwards unitary vector.

It is possible, now, to define an algorithm for homogenization. Given a microscale periodic cell  $\Omega$ , and the material phase distribution at the microscale  $m : \Omega \rightarrow M$ , where  $M$  is the set of unique materials in  $\Omega$ , the analyses can be performed. One can solve the set of governing equations  $P$  (Equation 3.13) for each of the imposed unitary  $\langle b_j \rangle$  to obtain  $b_j^*$  and  $a_i$  in  $\Omega$ , then take volume averages (Equation 3.9) to compute  $\langle a_i \rangle$ , so that the coefficients of the target homogenized tensor can be calculated (Equation 3.11). These notions are summarized in Algorithm 1.

---

**Algorithm 1** Homogenization - Scalar potential problem

---

|                                                                                                                                                                                                                                                                                                                                                                                                    |                                                                                                                                                          |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Input:</b> $\Omega, m$<br><b>Output:</b> $E_{ij}$<br>1: <b>for each</b> $k \in \{1, 2, 3\}$ <b>do</b><br>2: $\overline{b_j} \leftarrow \langle b_j \rangle^{(k)} = \delta_{jk}$<br>3:   Solve P for $b_j^*$<br>4: $a_i \leftarrow \kappa_{ij} (\overline{b_j} + b_j^*)$<br>5: $E_{ik} \leftarrow \langle a_i \rangle^{(k)} = \frac{1}{ \Omega } \int_{\Omega} a_i d\Omega$<br>6: <b>end for</b> | ▷ periodic cell, material distribution<br>▷ homogenized constitutive tensor<br>▷ $\mathbb{R}^3$<br>▷ analysis in direction $x_k$<br>▷ Eqs. 3.13 and 3.15 |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|

---

In Algorithm 1, generally the most troublesome step is the solution of the governing equations, in line 3. Authors have explored analytical solutions for relatively simple domains that may approximate real materials with “well-behaved” microstructures. A good example of this is the study of periodically arranged arrays of cylinders to model fiber-reinforced composites, as it can be seen in works as [24, 111]. The computational approach aims at solving these equations numerically, often exploring voxel-based techniques to deal with fixed-grid spatial discretization and three-dimensional microstructural images. Here, a matrix-free FEM approach is employed. In the following Section, it will be shown how this modeling applies for the homogenization of permeability.

### 3.3 Effective permeability of porous microstructures

The characterization of a porous material's absolute permeability is a homogenization problem, i.e. the notions depicted in the last Section are present in this context as well. The core idea of associating fields defined in a periodic microscale domain  $\Omega$  with effective fields in a macroscale domain  $Y$  via volume averaging rules so that a homogenized constitutive tensor can be calculated remains the same. What mostly differs it from other usual homogenization analyses is the relative higher complexity of the governing equations and boundary conditions that model fluid flow at the microstructure. In heat transfer and elastostatics, for example, elliptic equations, as in Equation 3.13, are admitted. For permeability, it is considered a linearized stationary form of the classical Navier-Stokes equations, known as Stokes flow, which leads to a saddle-point problem. Further, it is assumed that flow can only happen in the pore space and velocity must be null at the solid-pore interface, so extra considerations have to be made on top of the periodicity of the microscale domain. Generally, permeability analyses are more difficult to be handled than other linear homogenization problems, both in terms of the mathematical modeling and the computational cost.

Following the description presented in Section 3.2, three components must be described for the homogenization of permeability. The first, a constitutive law at the macroscale, is taken as Darcy's law. It can be concisely defined as

$$q_i = -\frac{1}{\mu} K_{ij} \langle p_{,j} \rangle, \quad \forall y_i \in Y, \quad (3.16)$$

where  $q_i$  is the flux per unit area,  $\mu$  is the fluid's dynamic viscosity,  $K_{ij}$  is the permeability tensor, and  $\langle p_{,j} \rangle$  is an effective pressure gradient, commonly referred as a pressure drop as well. In regards to units,  $q_i$  is usually depicted in  $[m/s]$ ,  $\mu$  is described in  $[Pa \cdot s]$ ,  $K_{ij}$  is measured in  $[m^2]$  – often written in  $[D]$  (Darcy) or  $[mD]$  (miliDarcy), with  $[D] \approx [\mu m^2]$  – and  $\langle p_{,j} \rangle$  is represented in  $[Pa/m]$ . Notice that  $q_i$  resembles a measure of velocity, as it models the fluid discharge through a normalized porous area. Indeed, it is associated with the flow at the microscale, being equivalent to an effective (volume averaged) velocity field, hereby represented as a vector field  $u_i$ , as in

$$q_i \equiv \langle u_i \rangle, \quad (3.17)$$

which means that Equation 3.16 can be rewritten as

$$\langle u_i \rangle = -\frac{1}{\mu} K_{ij} \langle p_{,j} \rangle. \quad \forall y_i \in Y. \quad (3.18)$$

By extension, analogously to the development shown in Equations 3.10 and 3.11, the permeability tensor  $K_{ij}$  can be written in terms of applied elementary effective pressure gradients, i.e.  $\langle p_{,j} \rangle^{(k)} = \delta_{jk}$ , as in

$$\langle u_i \rangle^{(k)} = -\frac{1}{\mu} K_{ij} \delta_{jk} \therefore K_{ik} = -\mu \langle u_i \rangle^{(k)}, \quad (3.19)$$

where  $\langle u_i \rangle^{(k)}$  is the effective velocity field obtained from analyses with the elementary effective gradient applied in direction  $x_k$ .

The second ingredient would be a reduction rule to represent “*micro to macro*” relations. As it has already been stated in this Section, volume averages are adopted, as it was portrayed in Equation 3.9. In that sense, the effective velocity and pressure gradient fields can be written as

$$\begin{cases} \langle u_i \rangle = \frac{1}{|\Omega|} \int_{\Omega} u_i d\Omega, \\ \langle p_{,j} \rangle = \frac{1}{|\Omega|} \int_{\Omega} p_{,j} d\Omega, \end{cases} \quad \forall y_i \in Y, \quad (3.20)$$

where, it is stressed,  $\Omega$  depicts the domain of the whole microscale periodic cell – not just the pore or solid space. These will be properly differentiated ahead.

The third item are the governing equations and boundary conditions at the microscale. As mentioned previously, the mathematical modeling of fluid flow in porous media is admitted to be described by Stokes equations. It can be derived via a development similar to that of the classical Navier-Stokes equations, by simplifying balance of momentum, which is often defined as

$$\sigma_{ij,j} + b_i = \rho (\dot{u}_i + u_{i,j} u_j), \quad \forall x_i \in \Omega, \quad (3.21)$$

where  $\sigma_{ij}$  is the Cauchy stress tensor,  $b_i$  would represent body forces, and  $\rho (\dot{u}_i + u_{i,j} u_j)$  corresponds to an inertial volume force with respect to the material derivative of the velocity field  $u_i$ .  $\rho$  is the fluid density. Notice that the convective term  $u_{i,j} u_j$  is non-linear. Considering creeping flow through a porous medium, i.e. a non-turbulent flow (low Reynolds number), which is a fair and consistent assumption for the modeling of the permeability homogenization problem, as Darcy’s law is derived from this consideration (see Whitaker, 1986 [146]), the inertial term can be neglected, i.e.

$$\dot{u}_i + u_{i,j} u_j \approx 0. \quad (3.22)$$

Physically, one can think of the statement in Equation 3.22 as an assumption that no

significant turbulence or time-dependent behavior occurs and so we are modeling a stationary laminar flow. Further, no body forces will be considered here, so  $b_i$  is taken as a null field. In that sense, Equation 3.21 can be rewritten as

$$\sigma_{ij,j} = 0, \quad \forall x_i \in \Omega, \quad (3.23)$$

which is analogous to the equilibrium equation in elastostatics or the flux conservation equation in heat transfer.

A constitutive law is considered in  $\Omega$  as well. In this case, it is a different relation from the one adopted in  $Y$ , Darcy's law (Equation 3.18), although it is also linear. Admitting incompressible flow, the stress tensor  $\sigma_{ij}$  is assumed to be expressed in terms of a deviatoric stress tensor  $\tau_{ij}$  and a hydrostatic pressure state, as in

$$\sigma_{ij} = \tau_{ij} - p\delta_{ij}, \quad (3.24)$$

where  $p$  is the pressure scalar field and, it is recalled,  $\delta_{ij}$  is the Kronecker delta. In this scope,  $\tau_{ij}$  can be defined as

$$\tau_{ij} = 2\mu\varepsilon_{ij}, \quad (3.25)$$

where  $\varepsilon$  is the rate of strain due to the spatial variations of flow, i.e. the gradient of the velocity field. It is written as

$$\varepsilon_{ij} = \frac{1}{2}(u_{i,j} + u_{j,i}), \quad (3.26)$$

which is analogous to the strain-displacement relation for small strain that is commonly employed in linear elasticity. An important difference, however, is that Equation 3.26 states a rate of strain-velocity relation instead. Coupling all Equations from 3.23 to 3.26, a concise partial differential equation for the balance of momentum is obtained:

$$\left. \begin{array}{ll} \text{(momentum)} & \sigma_{ij,j} = 0 \\ \text{(constitutive law)} & \sigma_{ij} = 2\mu\varepsilon_{ij} - p\delta_{ij} \\ \text{(rate of strain)} & \varepsilon_{ij} = \frac{1}{2}(u_{i,j} + u_{j,i}) \end{array} \right\} \mu u_{i,jj} - p_{,i} = 0, \quad \forall x_i \in \Omega. \quad (3.27)$$

Additionally, mass conservation has to be accounted in fluid flow simulations. To put it simply, it must be ensured that the difference between how much fluid “enters” and “leaves” the system is correspondent to either sinks or sources in the domain. For incompressible fluid without any source-like effects in the domain, this can be expressed

as in

$$\begin{aligned} \textbf{(mass conservation)} \quad \dot{\rho} + (\rho u_i)_{,i} &= 0, \quad \text{where } \rho \text{ is a non-null constant,} \\ \therefore u_{i,i} &= 0, \quad \forall x_i \in \Omega. \end{aligned} \quad (3.28)$$

It is recalled that  $\rho$  is the fluid density, which is assumed to neither change with time nor space. In light of this, as stated in Equation 3.28, mass conservation can be formally imposed by enforcing the divergent of the velocity field to be null throughout the microscale domain.

By coupling the balance of momentum (Equation 3.27) and mass conservation (Equation 3.28), the well-known system of partial differential equations that models Stokes flow is obtained:

$$\begin{aligned} \mu u_{i,jj} - p_{,i} &= 0 \\ u_{i,i} &= 0 \end{aligned}, \quad \forall x_i \in \Omega. \quad (3.29)$$

Notice that, for single-fluid analyses to compute absolute permeability, the dynamic viscosity of the fluid, identified as  $\mu$ , can be straightforwardly admitted as unitary. This consideration simplifies both the constitutive law in  $Y$  (Equations 3.18 and 3.19) and the governing equations in  $\Omega$  (Equation 3.29).

Additionally, proper periodic boundary conditions are required. Similarly to the development shown in Equation 3.15 in Section 3.2, the pressure gradient is taken as a known effective average value that is corrected locally by a fluctuation field, which depicts the effects of the non-homogeneous arrangement in  $\Omega$ . In that sense,  $p_{,i}$  is defined as

$$p_{,i}(x_i) = \overline{p_{,i}} + p_{,i}^*(x_i), \quad \forall x_i \in \Omega, \quad (3.30)$$

such that  $\langle p_{,i} \rangle = \overline{p_{,i}}$  and  $\langle p_{,i}^* \rangle = 0$ . This is ensured with the imposition that  $p_{,i}^*$  is periodic and that the outwards stress at the border  $\partial\Omega$  is anti-periodic. Notice that Equation 3.30 implies

$$p(x_i) = \overline{p_{,i}} x_i + p^*(x_i), \quad \forall x_i \in \Omega, \quad (3.31)$$

where  $p^*$  is a periodic pressure fluctuation field, and thus, having the microscale constitutive law in mind (Equation 3.24), the outwards velocity field must be anti-periodic at  $\partial\Omega$  as well. As  $\Omega$  is taken as a cuboid-shaped cell, the outwards unit vector at  $\partial\Omega$  is naturally anti-periodic, so these conditions can be met by enforcing the same pressure fluctuation and velocity values on opposite faces, edges and corners at  $\partial\Omega$ .

Finally, we must deal with the different phases in  $\Omega$ , as fluid flow is admitted to only occur in the pores, and boundary conditions must be imposed at the solid-pore interfaces.

Let  $\Omega_s$  be the union of all bounded continuous domains that represent the solid phase within  $\Omega$ . Likewise, let  $\Omega_p$  be the union of all bounded continuous domains that represent the pores. As it is considered that any point within  $\Omega$  is either in the pore or solid phase, it follows that  $(\Omega_s, \Omega_p) \subseteq \Omega$  and  $\Omega_s = \Omega \setminus \Omega_p$ . Let  $\Gamma$  be the border between  $\Omega_p$  and  $\Omega_s$ . Notice that  $\Gamma = \partial\Omega_p = \partial\Omega_s \neq \partial\Omega$ . Recall that the notation  $\partial\Omega$  refers to the borders of the periodic cell. It is assumed that the friction between fluid and solid walls causes constraints to the flow at  $\Gamma$ , analogously to the usual modeling of flow through a pipe, for example. In that sense, a *no-slip* boundary condition, i.e. null velocity, must be imposed at  $\Gamma$ .

With governing equations and boundary conditions defined, a boundary value problem can be stated:

$$\begin{cases} \mu u_{i,jj} - p_{,i}^* = \overline{p}_{,i} \\ u_{i,i} = 0 \end{cases}, \quad \forall x_i \in \Omega_p, \\ u_i = 0, \quad \forall x_i \in \Gamma \cup \Omega_s, \\ p_{,i}^* = 0, \quad \forall x_i \in \Omega_s, \\ (u_i, p^*) \text{ periodic}, \end{cases} \quad (3.32)$$

where  $u_i$  and  $p^*$  are unknown and a different  $\overline{p}_{,i}$  is imposed for the permeability analyses in each direction. In light of this, similarly to what was done for a scalar problem in Algorithm 1, Algorithm 2 can be determined for a concise description of the homogenization of absolute permeability.

---

**Algorithm 2** Homogenization - Absolute permeability of porous media

---

**Input:**  $\Omega, \Omega_p$  ▷ periodic cell, pore space domain  
**Output:**  $K_{ij}$  ▷ homogenized permeability tensor  
1: **for each**  $k \in \{1, 2, 3\}$  **do** ▷  $\mathbb{R}^3$   
2:    $\overline{p}_{,i} \leftarrow \langle p_{,i} \rangle^{(k)} = \delta_{ik}$  ▷ elementary  $\langle p_{,j} \rangle$  in direction  $x_k$   
3:   Solve Stokes flow PDEs for  $(u_i, p^*)$  ▷ (Eq. 3.32)  
4:    $K_{ik} \leftarrow -\mu \langle u_i \rangle^{(k)} = -\mu \frac{1}{|\Omega|} \int_{\Omega} u_i d\Omega$   
5: **end for**

---

In Algorithm 2, notice that all the necessary domain information can be extracted from  $\Omega$ , the periodic cell as a whole, and  $\Omega_p$ , the pore space within  $\Omega$ . Analogously to the scalar potential problem, here, the most costly step in the solution of the boundary value problem at the microscale, stated in line 3. Some analytical solutions have been demonstrated for simple domains, but, to handle the complex microstructures provided by  $\mu$ CT imaging, numerical methods are essential. In the following Chapter, it will be shown how this problem can be solved numerically considering an image-based domain.

## Chapter 4

# Numerical Simulations for Permeability Estimation

The boundary value problem governed by Stokes flow shown in Equation 3.32 will be tackled in the context of this work via the FEM. Namely, a specific version of the method known as voxel- or image-based FEM is considered, to take advantage of the fixed-grid nature of the studied discrete domains,  $\mu$ CT images. To that end, at first, a weak-form of the partial differential equations at hand must be defined. The weighted residual method is applied like so

$$\begin{aligned} \int_{\Omega_p} \omega_i (\mu \tilde{u}_{i,jj} - \tilde{p}_{,i}^* - \overline{p}_{,i}) d\Omega &= 0 \\ \int_{\Omega_p} m (\tilde{u}_{i,i}) d\Omega &= 0 \end{aligned}, \quad \forall x_i \in \Omega_p, \quad (4.1)$$

where  $\omega_i$  and  $m$  are vector and scalar non-null weight fields, respectively. The idea here is to search for approximations  $\tilde{u}_i$  and  $\tilde{p}^*$  for  $u_i$  and  $p^*$ , respectively, that minimize the overall error in  $\Omega_p$ . For simplicity, the notation  $u_i$  and  $p^*$  will remain in the following developments, as they must hold for approximations and analytical results alike. Notice that when these integrals are null, the statement in Equation 3.32 holds. The top line expression in Equation 4.1 can be broken in separate integral terms, as in

$$\begin{aligned} \int_{\Omega_p} \omega_i (\mu u_{i,jj} - p_{,i}^*) d\Omega &= \int_{\Omega_p} \omega_i \overline{p}_{,i} d\Omega \\ \int_{\Omega_p} m u_{i,i} d\Omega &= 0 \end{aligned}, \quad \forall x_i \in \Omega_p, \quad (4.2)$$

where now the unknown terms remain in the left-hand side of Equation 4.2 and the known average pressure gradient is written in the right-hand side.

As customary, the expression in Equation 4.2 that contains the laplacian term (the

top one) is externalized, avoiding second derivatives and explicitly separating a contour integral at  $\Gamma$  that carries boundary condition information. By Green's identity, the integral of the laplacian of the velocity field can be rewritten as

$$\int_{\Omega_p} \omega_i \mu u_{i,jj} d\Omega = \oint_{\Gamma} \omega_i \mu u_{i,j} \hat{n}_j d\Gamma - \int_{\Omega_p} \omega_{j,i} \mu u_{i,j} d\Omega , \quad (4.3)$$

where  $\hat{n}_j$  is outwards unit vector with respect to  $\Gamma$ . Similarly, the pressure gradient fluctuation term can be externalized like so

$$\int_{\Omega_p} \omega_i p_{,i}^* d\Omega = \oint_{\Gamma} \omega_i p^* \hat{n}_i d\Gamma - \int_{\Omega_p} \omega_{i,i} p^* d\Omega . \quad (4.4)$$

Then, Equation 4.4 is subtracted from 4.3, which yields

$$\int_{\Omega_p} \omega_i (\mu u_{i,jj} - p_{,i}^*) d\Omega = \oint_{\Gamma} \omega_i (\mu u_{i,j} - \delta_{ij} p^*) \hat{n}_j d\Gamma - \int_{\Omega_p} \omega_{j,i} (\mu u_{i,j} - \delta_{ij} p^*) d\Omega , \quad (4.5)$$

where the term  $(\mu u_{i,j} - \delta_{ij} p^*) \hat{n}_j$  within the contour integral corresponds to a surface traction vector field at  $\Gamma$ , which is a Neumann boundary condition. As no traction is considered at the solid-pore interface in the proposed homogenization analyses, this contour integral vanishes, i.e.

$$\int_{\Omega_p} \omega_i (\mu u_{i,jj} - p_{,i}^*) d\Omega = -\mu \int_{\Omega_p} \omega_{j,i} u_{i,j} d\Omega + \int_{\Omega_p} \omega_{i,i} p^* d\Omega . \quad (4.6)$$

With Equation 4.6, we can then rewrite Equation 4.2 as

$$\begin{aligned} \mu \int_{\Omega_p} \omega_{j,i} u_{i,j} d\Omega - \int_{\Omega_p} \omega_{i,i} p^* d\Omega &= - \int_{\Omega_p} \omega_i \overline{p_{,i}} d\Omega \\ \int_{\Omega_p} m u_{i,i} d\Omega &= 0 \end{aligned} , \quad \forall x_i \in \Omega_p, \quad (4.7)$$

which is the usual weak-form expression of the Stokes equations that is solved with the FEM for permeability characterization, as it can be seen in Andreassen and Andreassen [9] and Vianna et al. [142]. It is common to characterize the weight fields as *virtual* fields, i.e. small non-null deviations from the analytical solution for  $(u_i, p^*)$ , denoted as

$$\begin{aligned} w_i &= \delta u_i, \\ m &= \delta p^*, \end{aligned} \quad (4.8)$$

such that Equation 4.7 is rewritten as

$$\begin{aligned} \mu \int_{\Omega_p} \delta u_{j,i} \tilde{u}_{i,j} d\Omega - \int_{\Omega_p} \delta u_{i,i} \tilde{p}^* d\Omega &= - \int_{\Omega_p} \delta u_i \overline{p_{,i}} d\Omega \\ \int_{\Omega_p} \delta p^* \tilde{u}_{i,i} d\Omega &= 0 \end{aligned}, \quad \forall x_i \in \Omega_p. \quad (4.9)$$

These are the governing equations that will be studied with the FEM in this work. Notice that the notation  $\tilde{u}_i$  and  $\tilde{p}^*$  was brought back in Equation 4.9 as a reminder that numerical approximations will be sought for  $u_i$  and  $p^*$ , and, nevertheless, these integrals must hold.

## 4.1 Finite Element Method

The FEM aims at approximating unknown variable fields defined in a bounded continuous domain, as is  $\Omega_p$ , via a linear combination of compactly supported interpolation functions, often referred as *shape functions*, that describe the influence of discrete values of those fields at sampled points, called *nodes*, on their vicinity. The studied domain is discretized into a finite set of subdomains, called *elements*, where the behavior of the continuous fields is modeled as a sum of the effects of the discrete values at each surrounding node. The collective of all elements is denominated the *mesh*. In conventional FEM implementations, obtaining good representative meshes can be a challenging task by itself, and those are often unstructured, meaning that topological data structures and connectivity maps are usually required. In the scope of this Thesis, the adopted meshes are based on voxels, as the domains of analyses are high-resolution raster images. In that sense, all elements have the geometry of a normalized square (in  $\mathbb{R}^2$ ) or cube (in  $\mathbb{R}^3$ ), which allows for simpler numerical modeling and considerably less expensive computational implementations. Figure 4.1 illustrates the difference between unstructured and voxel-based meshes. The trade-off in such a strategy is the large number of variables to be calculated per simulation and the possibility of localized “stair-step” errors at boundaries or interfaces, which tends to be minimized with fine discretization. The FEM formulation for the solution of Equation 4.7 is presented ahead. The following numerical modeling is based on classical books on the subject, as [27, 42, 117, 155].

Let  $E$  be the set of all elements that compose a mesh that discretizes  $\Omega_p$ . As no element is superimposed to one another, the union of all elements  $e \in E$  yields  $\Omega_p$ . In that sense, it follows that an integral in  $\Omega_p$  can be defined as

$$\int_{\Omega_p} \phi d\Omega = \sum_{e \in E} \int_{\Omega_e} \phi d\Omega, \quad (4.10)$$

where  $\phi$  is a continuous field defined in  $\Omega_p$ , and  $\Omega_e$  is the subdomain associated with element  $e$ , which is contained in  $\Omega_p$ .

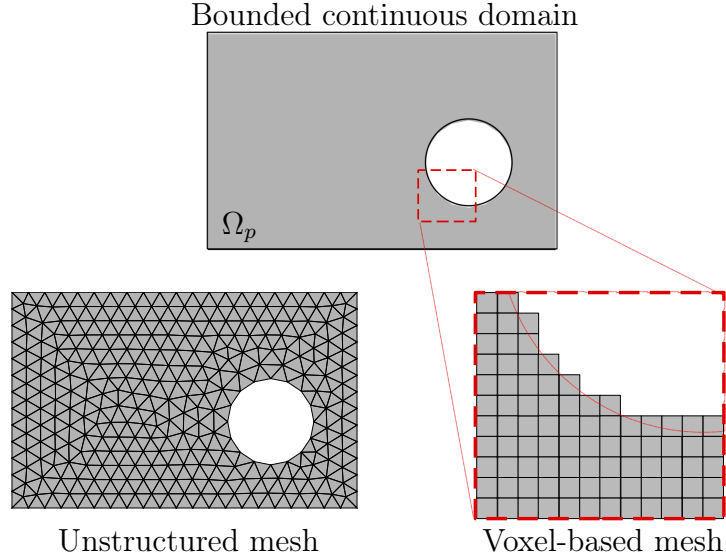


Figure 4.1: Difference between an unstructured mesh (to the left) and a voxel-based mesh (to the right).

Let  $\mathbf{N}^{(u)}$  and  $\mathbf{N}^{(p)}$  be shape function matrices that interpolate nodal values of  $u_i$  and  $p^*$ , respectively, within the domain an element. In this Thesis, bi- and tri-linear (2D and 3D) shape functions are adopted for both velocity and pressure fields, for all elements. In that sense, all elements in the mesh are said to be Q1Q1 elements. These same-order interpolations require some special care in regards to stability, as it will be discussed shortly ahead. However, they allow for direct and seamless employment of the voxelized microstructures, with nodes only at elements' corners (voxels' vertexes), which is an important feature for the efficient meshing of the pore space in an image-based context. This sort of element has been employed in other works within a similar scope, as [9, 142, 151].

Let  $\mathbf{u}$  and  $\mathbf{p}^*$  be the vectors containing the discrete values of  $\tilde{u}_i$  and  $\tilde{p}^*$  for each node of the mesh. Let  $\mathbf{u}_e$  and  $\mathbf{p}_e^*$  be the vectors of nodal values of  $\tilde{u}_i$  and  $\tilde{p}^*$  for the nodes around an element  $e \in E$ , such that

$$\begin{aligned} \mathbf{u}_e &= \mathbf{T}_e^{(u)} \mathbf{u}, \\ \mathbf{p}_e^* &= \mathbf{T}_e^{(p)} \mathbf{p}^*, \end{aligned} \quad \forall e \in E, \quad (4.11)$$

where  $\mathbf{T}_e^{(u)}$  and  $\mathbf{T}_e^{(p)}$  are *projection matrices* that gather the nodal values of  $\mathbf{u}$  and  $\mathbf{p}^*$  around an element  $e$ .

With shape functions and projection matrices defined, it is possible to write approximations for  $u_i$  and  $p^*$  within an element  $e$ , like so

$$\begin{aligned} u_i &\approx \tilde{u}_i = \mathbf{N}^{(u)} \mathbf{T}_e^{(u)} \mathbf{u} = \mathbf{N}^{(u)} \mathbf{u}_e, \\ p^* &\approx \tilde{p}^* = \mathbf{N}^{(p)} \mathbf{T}_e^{(p)} \mathbf{p}^* = \mathbf{N}^{(p)} \mathbf{p}_e^*, \end{aligned} \quad \forall x_i \in \Omega_e, \forall e \in E. \quad (4.12)$$

Notice that the vectors of nodal values are not functions of  $x_i$ , only the bi- or tri-linear polynomials in  $\mathbf{N}^{(u)}$  and  $\mathbf{N}^{(p)}$  are. In light of this, it is straightforward to define gradient operators in terms of spatial derivatives, as in

$$\begin{aligned} u_{i,j} &\approx \tilde{u}_{i,j} = \overbrace{\nabla \mathbf{N}^{(u)}}^{\mathbf{B}^{(u)}} \mathbf{u}_e, \\ p_{,i}^* &\approx \tilde{p}_i^* = \underbrace{\nabla \mathbf{N}^{(p)}}_{\mathbf{B}^{(p)}} \mathbf{p}_e^*, \end{aligned} \quad \forall x_i \in \Omega_e, \forall e \in E, \quad (4.13)$$

where  $\mathbf{B}^{(u)}$  and  $\mathbf{B}^{(p)}$  are matrices that hold the spatial derivatives of velocity and pressure shape functions.

The virtual fields  $\delta u_i$  and  $\delta p^*$  also have to be approximated. By Galerkin's method, it is admitted that the same shape functions adopted for  $\tilde{u}_i$  and  $\tilde{p}^*$  are employed to approximate their respective virtual counterparts as well. In doing so, the projection matrices are also the same. Then, it follows that

$$\begin{aligned} \delta u_i &= \mathbf{N}^{(u)} \mathbf{T}_e^{(u)} \delta \mathbf{u} = \mathbf{N}^{(u)} \delta \mathbf{u}_e, \\ \delta p^* &= \mathbf{N}^{(p)} \mathbf{T}_e^{(p)} \delta \mathbf{p}^* = \mathbf{N}^{(p)} \delta \mathbf{p}_e^*, \end{aligned} \quad \forall x_i \in \Omega_e, \forall e \in E, \quad (4.14)$$

where, analogously,  $\delta \mathbf{u}$  and  $\delta \mathbf{p}^*$  are vectors of nodal values of the virtual fields. The spatial derivatives of shape functions remain valid for the virtual fields as well:

$$\begin{aligned} \delta u_{i,j} &= \mathbf{B}^{(u)} \delta \mathbf{u}_e, \\ \delta p_{,i}^* &= \mathbf{B}^{(p)} \delta \mathbf{p}_e^*, \end{aligned} \quad \forall x_i \in \Omega_e, \forall e \in E. \quad (4.15)$$

With these approximations for the continuous fields defined in terms of the interpolation of nodal values, it is possible to turn the system of integral equations in Equation 4.7 into a sparse system of algebraic equations. Each of the integral terms can be substituted like so

$$\begin{aligned}
\mu \int_{\Omega_p} \delta u_{j,i} \tilde{u}_{i,j} d\Omega &= \delta \mathbf{u}^T \left( \sum_{e \in E} \mathbf{T}_e^{(u)T} \int_{\Omega_e} \mathbf{B}^{(u)T} \mu \mathbf{B}^{(u)} d\Omega \mathbf{T}_e^{(u)} \right) \mathbf{u} = \delta \mathbf{u}^T \mathbf{K} \mathbf{u}, \\
\int_{\Omega_p} \delta u_{i,i} \tilde{p}^* d\Omega &= \delta \mathbf{u}^T \left( \sum_{e \in E} \mathbf{T}_e^{(u)T} \int_{\Omega_e} \mathbf{B}^{(u)T} \mathbf{N}^{(p)} d\Omega \mathbf{T}_e^{(p)} \right) \mathbf{p}^* = \delta \mathbf{u}^T \mathbf{G} \mathbf{p}^*, \\
\int_{\Omega_p} \delta u_{i,i} \overline{p}_{,i} d\Omega &= \delta \mathbf{u}^T \left( \sum_{e \in E} \mathbf{T}_e^{(u)T} \int_{\Omega_e} \mathbf{N}^{(u)T} \overline{p}_{,i} d\Omega \right) = -\delta \mathbf{u}^T \mathbf{f}, \\
\int_{\Omega_p} \delta p^* \tilde{u}_{i,i} d\Omega &= \delta \mathbf{p}^{*T} \left( \sum_{e \in E} \mathbf{T}_e^{(p)T} \int_{\Omega_e} \mathbf{N}^{(p)T} \mathbf{B}^{(u)} d\Omega \mathbf{T}_e^{(u)} \right) \mathbf{u} = \delta \mathbf{p}^{*T} \mathbf{G}^T \mathbf{u},
\end{aligned} \tag{4.16}$$

where  $\mathbf{K}$  is the global viscosity matrix,  $\mathbf{G}$  is the global gradient matrix, and  $\mathbf{f}$  is a right-hand side vector that imposes the elementary pressure gradient, which is akin to a body force vector. In Equation 4.16, notice that the remaining integrals are only associated with shape functions – bi- and tri-linear polynomials – in the domain of each element, which is geometrically the same for every voxel. Further, as we are focused on single-fluid flow,  $\mu$  is also the same for every voxel. Consequently, these integrals can be calculated analytically just once, instead of per each element, so it is possible to work with single elemental matrices, defined as

$$\begin{aligned}
\mathbf{K}_e &= \int_{\Omega_e} \mathbf{B}^{(u)T} \mu \mathbf{B}^{(u)} d\Omega, \\
\mathbf{G}_e &= \int_{\Omega_e} \mathbf{B}^{(u)T} \mathbf{N}^{(p)} d\Omega, \quad \forall e \in E, \\
\mathbf{f}_e &= - \int_{\Omega_e} \mathbf{N}^{(u)T} \overline{p}_{,i} d\Omega,
\end{aligned} \tag{4.17}$$

where  $\mathbf{K}_e$  is an elemental viscosity matrix,  $\mathbf{G}_e$  is an elemental gradient matrix, and  $\mathbf{f}_e$  is an elemental contribution to the right-hand side vector.

It is possible, then, to rewrite the integral weak-form of the governing equations as a linear system of algebraic equations that looks like

$$\begin{aligned}
\delta \mathbf{u}^T (\mathbf{K} \mathbf{u} - \mathbf{G} \mathbf{p}^* - \mathbf{f}) &= 0, \\
\delta \mathbf{p}^{*T} (\mathbf{G}^T \mathbf{u}) &= 0.
\end{aligned} \tag{4.18}$$

Recalling that one of the conditions of the weighted residual method is that the weight (or virtual) fields be non-null, it is evident in Equation 4.18 that the expressions within parenthesis must result in null vectors for the Equation to be satisfied. In light of this,

the following matrix system is obtained

$$\begin{bmatrix} \mathbf{K} & \mathbf{G} \\ \mathbf{G}^T & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{u} \\ -\mathbf{p}^* \end{bmatrix} = \begin{bmatrix} \mathbf{f} \\ \mathbf{0} \end{bmatrix}. \quad (4.19)$$

This is the global system of equations that arises from the FEM for the Stokes equations. It is symmetric and sparse. Usually, from this point on, the numerical “heavy work” is delegated to an iterative solver [112, 135], so that  $\mathbf{u}$  and  $\mathbf{p}^*$  may be computed. For instance, Conjugate Gradient methods [56, 74, 132] tend to perform well for elliptic problems discretized via the FEM, and can be parallelized [10, 55, 67, 68, 81, 116]. The MINRES or GMRES methods [44, 112, 136], on the other hand, are more indicated for indefinite matrices (positive and negative eigenvalues) that arise from saddle-point problems, as is the case for the Stokes equations. However, there is still an important matter to be resolved in Equation 4.19: stability. The employment of same-order shape functions for velocity and pressure, although very useful in terms of working with a voxel-based mesh, as discussed in Yang et al. (2019) [151], leads to failure in regards to the Babuška-Brezzi stability conditions [136], i.e. it yields highly unstable ill-conditioned systems of algebraic equations. In this work, a stabilization term will be added to the mass conservation (Equation 3.28), so that the bi- and tri-linear Q1Q1 elements may be employed. Notice that this sort of practice is common in the literature, from the derivation of stabilization techniques, as in the series of papers by Hughes et al. (1986) [60, 59], to its application, as in Qi et al. (2013) [115], Andreassen and Andreassen (2014) [9], Yang et al. (2019) [151], Vianna et al. (2020) [142], and Lesueur et al. (2022) [75], to cite a few.

To stabilize the system of equations to be solved, a numerical penalty term is introduced to the mass conservation, written in weak-form as follows

$$\int_{\Omega_p} \delta p^* (\tilde{u}_{i,i} + \tau \tilde{p}^*) \, d\Omega = 0, \quad (4.20)$$

where the optimal value [9, 142, 151] of

$$\tau = \frac{h^2}{12\mu}$$

is adopted for the parameter  $\tau$ , with  $h$  as the length of the longest possible straight line within an element, i.e. the main diagonal of the voxel, and, it is recalled,  $\mu$  as the dynamic viscosity of the fluid. Notice that this implies a penalization of incompressibility, making the fluid quasi-incompressible, or “pseudo-compressible” [151]. By applying shape functions to Equation 4.20 and following a similar development to those in Equation 4.16,

a new global matrix is obtained:

$$\int_{\Omega_p} \delta p^* \tau \tilde{p}^* d\Omega = \delta \mathbf{p}^{*T} \left( \sum_{e \in E} \mathbf{T}_e^{(p)T} \int_{\Omega_e} \mathbf{N}^{(p)T} \tau \mathbf{N}^{(p)} d\Omega \mathbf{T}_e^{(p)} \right) \mathbf{p}^* = \delta \mathbf{p}^{*T} \mathbf{S} \mathbf{p}^*, \quad (4.21)$$

where  $\mathbf{S}$  is a stabilization matrix. Likewise, an elemental stabilization matrix  $\mathbf{S}_e$  is defined as

$$\mathbf{S}_e = \int_{\Omega_e} \mathbf{N}^{(p)T} \tau \mathbf{N}^{(p)} d\Omega, \quad \forall e \in E. \quad (4.22)$$

Analogously, a stabilized mass conservation algebraic equation is determined:

$$\delta \mathbf{p}^{*T} (\mathbf{G}^T \mathbf{u} + \mathbf{S} \mathbf{p}^*) = 0, \quad (4.23)$$

and, finally, as the virtual pressure fluctuation field  $\delta p^*$  is non-null, the expression within parenthesis in the equation above must be null. This allows for the system of equations in 4.19 to be updated to

$$\begin{bmatrix} \mathbf{K} & \mathbf{G} \\ \mathbf{G}^T & -\mathbf{S} \end{bmatrix} \begin{bmatrix} \mathbf{u} \\ -\mathbf{p}^* \end{bmatrix} = \begin{bmatrix} \mathbf{f} \\ \mathbf{0} \end{bmatrix}, \quad (4.24)$$

which is still a saddle-point problem, but better conditioned than the system in Equation 4.19. It can be solved with iterative Krylov subspace solvers [28, 51, 135], such as the CG method [132] and the MINRES method [44, 104]. These solvers are particularly interesting in terms of computational cost, as they essentially rely on conventional vector operations and matrix-vector products. Matrix-free strategies, as in the case of this Thesis, focus on performing the large sparse matrix operations without ever allocating the global matrices in memory, even in sparse format.

## 4.2 Iterative solvers

Iterative methods are broadly employed for the solution of large sparse systems of algebraic equations in Scientific Computing [54]. When dealing with numerical simulations for stationary physics phenomena, it is common to adopt iterative solvers that aim at minimize an error or residual metric, with the physical correspondence of minimizing the potential energy of a system. Among the vast roster of possible solvers, which includes classical methods such as Gauss-Seidel and Gauss-Jacobi, a particular family of iterative procedures stands out for the solution of large-scale systems, i.e. 1 million *degrees-of-freedom* (DOFs) – variables – or more: Krylov subspace solvers [28, 51, 108, 135]. Some key notions behind these will be presented shortly ahead.

Firstly, it is important to properly write the system of equations to be solved in a  $\mathbf{Ax} = \mathbf{b}$  format. This is done by defining the following matrix and vectors:

$$\begin{aligned}\mathbf{A} &= \begin{bmatrix} \mathbf{K} & \mathbf{G} \\ \mathbf{G}^T & -\mathbf{S} \end{bmatrix}, \\ \mathbf{x} &= \begin{bmatrix} \mathbf{u} \\ -\mathbf{p}^* \end{bmatrix}, \\ \mathbf{b} &= \begin{bmatrix} \mathbf{f} \\ \mathbf{0} \end{bmatrix}.\end{aligned}\tag{4.25}$$

Further, for the sake of clarity, let's differentiate *error* from *residual*. The error  $\boldsymbol{\epsilon}^{(k)}$  is a measure of how distant the solution vector at the  $k$ -th iteration  $\mathbf{x}^{(k)}$  is to the actual solution  $\mathbf{x}_{\text{sol.}}$ . The residual  $\mathbf{r}^{(k)}$  is a measure of how distant the product  $\mathbf{Ax}^{(k)}$  is from the right-hand side vector  $\mathbf{b}$ , also at the  $k$ -th iteration. These can be formally written as

$$\begin{aligned}(\text{error}) \quad \boldsymbol{\epsilon}^{(k)} &= \mathbf{x}_{\text{sol.}} - \mathbf{x}^{(k)}, \\ (\text{residual}) \quad \mathbf{r}^{(k)} &= \mathbf{b} - \mathbf{Ax}^{(k)}.\end{aligned}\tag{4.26}$$

Naturally, good solutions are those that yield both error and residual as close as possible (or as required) to null. Notice, however, that in real scenarios we usually can only keep track of the residual, as if  $\mathbf{x}_{\text{sol.}}$  was already available, the system would have already been solved.

Accordingly to Strang (2006) [136], a Krylov subspace can be straightforwardly defined by analyzing the progressive updates of  $\mathbf{x}^{(k)}$  in a simple iterative procedure. Assume, for example, that  $\mathbf{x}^{(0)} = \mathbf{b}$  and that, at each iteration, the solution will be corrected by the residual. Then, it follows that  $\mathbf{x}^{(1)} = \mathbf{b} - \mathbf{Ax}^{(0)} + \mathbf{x}^{(0)} = 2\mathbf{b} - \mathbf{Ab}$ . Analogously,  $\mathbf{x}^{(2)} = \mathbf{b} - \mathbf{Ax}^{(1)} + \mathbf{x}^{(1)} = 3\mathbf{b} - 3\mathbf{Ab} + \mathbf{A}^2\mathbf{b}$ . For  $\mathbf{x}^{(3)}$ , an extra  $\mathbf{A}^3\mathbf{b}$  term will appear, for  $\mathbf{x}^{(4)}$  it will be  $\mathbf{A}^4\mathbf{b}$ , and so forth. The main idea here is that, at any  $k$ -th iteration, the solution  $\mathbf{x}^{(k)}$  is a linear combination of the elements of the set  $\{\mathbf{b}, \mathbf{Ab}, \mathbf{A}^2\mathbf{b}, \mathbf{A}^3\mathbf{b}, \dots, \mathbf{A}^k\mathbf{b}\}$ . All of the possible linear combinations form the  $k$ -th Krylov subspace, hereby denoted as  $\mathcal{K}^{(k)}$ .

Krylov subspace solvers are iterative methods that aim at finding the best combination possible for each  $\mathcal{K}^{(k)}$ . Notice that “best” may vary with context. For example, one could argue that the goal is to minimize the amount of iterations to achieve an acceptable approximation of  $\mathbf{x}_{\text{sol.}}$ , i.e. for a residual norm to be within a specified numerical tolerance. However, if those iterations are too expensive in terms of either computational performance or memory, such a scheme might be slower than others, or even unfeasible for

large systems. Likewise, it is possible to make a point that the simpler the combination, and thus the fewer required computations per iteration, the better. But if it takes too many iterations to achieve sufficiently low residuals, it may require too long processing times to achieve an acceptable solution, which would also be counterproductive. Yet, it is stressed that, in practice, computational performance issues can often be mitigated with parallelism, while memory usage is a common bottleneck for large-scale simulations. All in all, there is no definitive best solver for any use case, and different methods can be beneficial or detrimental to different applications.

In this Thesis, two Krylov subspace solvers are explored: the CG method and the MINRES method. The first is widespread in FEM-based homogenization analyses [41, 45, 81, 93] and voxel-based FEM in general [10, 67, 68]. The second is known for being a more stable method for the Stokes equations, as seen in [44, 104, 108, 112, 136, 152]. Both of these are meant for symmetric matrices. Here, *short recurrence* iterations are considered, i.e. the computation of a next iteration only requires the history of the previous one. In that sense, to update the solution vector, all that is demanded is a search direction  $\mathbf{d}$ , a step magnitude  $\alpha$ , and its own previous values, as in

$$\mathbf{x}^{(k)} = \mathbf{x}^{(k-1)} + \alpha \mathbf{d}^{(k)}. \quad (4.27)$$

Then, it follows that the the residual vector can be updated as such:

$$\mathbf{r}^{(k)} = \mathbf{r}^{(k-1)} - \alpha \mathbf{A} \mathbf{d}^{(k)}. \quad (4.28)$$

This form of the MINRES method for symmetric (hermitian) matrices is algebraically equivalent to the short-recurrence Conjugate Residual (CR) method [134]. It is common to see papers in the literature referencing the two methods interchangeably under these conditions. Herein, this solver will henceforth only be denominated as MINRES, but readers may bear in mind that it is in fact equivalent to a CR solver.

Once again following Strang (2006) [136], the definitions of “best” combination for CG and MINRES is:

- **CG**: The  $k$ -th residual must be orthogonal to  $\mathcal{K}^{(k-1)}$ , for  $k \in \mathbb{N}_{\geq 1}$ . By extension, the  $k$ -th search direction must be  $A$ -orthogonal to all previous search directions, i.e.  $\mathbf{d}^{(k)T} \mathbf{A} \mathbf{d}^{(j)} = 0$ , for  $k \in \mathbb{N}_{\geq 2}$  and  $j \in \{1, \dots, k-1\}$ . These considerations have the effect of minimizing the amount of iterations for convergence. However, the method can become unstable if  $\mathbf{A}$  is indefinite or ill-conditioned.

- **MINRES**: The norm of the  $k$ -th residual must be minimal, i.e. a step  $\alpha \mathbf{d}^{(k)}$  must be obtained from a combination of  $\mathcal{K}^{(k-1)}$  such that  $\|\mathbf{b} - \mathbf{A}\mathbf{x}^{(k)}\|$  is locally minimized. One can think about this analogously to a greedy minimization problem, where every iteration attempts to lower the norm of the residual as much as possible. In light of this, the method usually yields non-oscillating convergence patterns, although it can get “stuck” in plateau-like scenarios, where lots of iterations are required for the residual to be significantly lowered.

The implemented solvers follow the procedures shown in Algorithms 3 through 6, with a matrix-free approach. Although there are matrix operations in these, recall that no global matrix  $\mathbf{A}$  is ever stored in memory. Notice that Algorithms 5 and 6 present the Preconditioned CG (PCG) and Preconditioned MINRES (PMINRES) methods, respectively. These are enhancements of CG and MINRES that aim at solving systems of the form  $\mathbf{M}^{-1}\mathbf{A}\mathbf{x} = \mathbf{M}^{-1}\mathbf{b}$ , where  $\mathbf{M}$  is a preconditioner matrix, whose purpose is to improve the conditioning of  $\mathbf{A}$ . There exist a variety of very promising preconditioners [16, 108, 135], but many of them require large matrices or blocks of matrices to be stored in memory. Others can be implemented as smaller systems of equations embedded in the original problem, to be solved on demand whenever the preconditioner is required. On the other hand, in a matrix-free scope, especially when dealing with GPUs, it is usual for researchers to employ concise Jacobi (diagonal) preconditioning [78, 81, 85, 116], which not only simplifies preconditioner-vector algebraic operations but also allows for its coefficients to be recomputed on-the-fly, i.e. for not storing it in memory. Even so, it is known that other preconditioners, such as those obtained from incomplete Cholesky factorization or multi-grid methods, tend to outperform Jacobi preconditioning, when it comes to lowering the number of iterations for convergence. Here, bearing in mind the lightweight strategy and focus on memory efficiency, only Jacobi preconditioning is adopted for the the PCG and PMINRES methods. It is stressed, however, that the developments presented ahead can be extend for better, although either more memory or more computationally demanding, preconditioners as well.

In Algorithms 3 through 6, aside from the solution, residual and search direction vectors ( $\mathbf{x}$ ,  $\mathbf{r}$  and  $\mathbf{d}$ ), two other vectors were introduced:  $\mathbf{s}$  and  $\mathbf{q}$ . The first is a transformed residual while the second is the direction of change for the residual, associated with a step in direction  $\mathbf{d}$  to correct  $\mathbf{x}$ . In the PCG method, notice that a single vector can be allocated to handle both  $\mathbf{s}$  and  $\mathbf{q}$ , so it requires 4 vectors to be stored [81, 82], similarly to the CG method. On the other hand, MINRES and PMINRES require 5 vectors to be allocated. However, it is stressed that alternative versions of these solvers can be implemented with

fewer vectors stored in memory, at the cost of more matrix-vector products per iteration. This will be explored in Chapter 5. The scalar parameter  $\beta$  is the ratio of change of the residual, which is used to update search directions, and  $\delta$  is a measure of the norm of residuals, employed to compute the step magnitude  $\alpha$ . The following stopping criteria are considered:

- Normalized  $L_2$ -like norm of the preconditioned residual:

$$\left| \frac{\mathbf{r}^{(k)\top} \mathbf{M}^{-1} \mathbf{r}^{(k)}}{\mathbf{b}^\top \mathbf{M}^{-1} \mathbf{b}} \right| \leq \epsilon^2, \quad \text{where } \epsilon \text{ is the tolerance.} \quad (4.29)$$

Notice that, if no preconditioning is considered,  $\mathbf{M}$  can be taken as the identity matrix. In that case,

$$\left| \frac{\mathbf{r}^{(k)\top} \mathbf{r}^{(k)}}{\mathbf{b}^\top \mathbf{b}} \right| \leq \epsilon^2. \quad (4.30)$$

- Maximum number of iterations (for safety):  $k \leq k_{\max}$ .

---

**Algorithm 3** CG method

---

**Input:**  $\mathbf{A}, \mathbf{x}_0, \mathbf{b}$

**Output:**  $\mathbf{x}$

```

1:  $\mathbf{x} \leftarrow \mathbf{x}_0, \mathbf{d} \leftarrow \mathbf{0}$ 
2:  $\mathbf{r} \leftarrow \mathbf{b} - \mathbf{A}\mathbf{x}$ 

3:  $\delta^{(0)} \leftarrow \mathbf{r}^\top \mathbf{r}$ 
4:  $\beta \leftarrow 0$ 
5: while stopping criteria is not met do
6:    $\mathbf{r} \leftarrow \mathbf{s} + \beta \mathbf{d}$ 
7:    $\mathbf{q} \leftarrow \mathbf{A}\mathbf{d}$ 
8:    $\alpha \leftarrow \delta^{(k-1)} / \mathbf{q}^\top \mathbf{d}$ 
9:    $\mathbf{x} \leftarrow \mathbf{x} + \alpha \mathbf{d}$ 
10:   $\mathbf{r} \leftarrow \mathbf{r} - \alpha \mathbf{q}$ 

11:   $\delta^{(k)} \leftarrow \mathbf{r}^\top \mathbf{r}$ 
12:   $\beta \leftarrow \delta^{(k)} / \delta^{(k-1)}$ 
13: end while
```

---



---

**Algorithm 4** MINRES (CR) method

---

**Input:**  $\mathbf{A}, \mathbf{x}_0, \mathbf{b}$

**Output:**  $\mathbf{x}$

```

1:  $\mathbf{x} \leftarrow \mathbf{x}_0, \mathbf{d} \leftarrow \mathbf{0}, \mathbf{q} \leftarrow \mathbf{0}$ 
2:  $\mathbf{r} \leftarrow \mathbf{b} - \mathbf{A}\mathbf{x}$ 
3:  $\mathbf{s} \leftarrow \mathbf{A}\mathbf{r}$ 
4:  $\delta^{(0)} \leftarrow \mathbf{r}^\top \mathbf{s}$ 
5:  $\beta \leftarrow 0$ 
6: while stopping criteria is not met do
7:    $\mathbf{d} \leftarrow \mathbf{r} + \beta \mathbf{d}$ 
8:    $\mathbf{q} \leftarrow \mathbf{s} + \beta \mathbf{q}$ 
9:    $\alpha \leftarrow \delta^{(k-1)} / \mathbf{q}^\top \mathbf{q}$ 
10:   $\mathbf{x} \leftarrow \mathbf{x} + \alpha \mathbf{d}$ 
11:   $\mathbf{r} \leftarrow \mathbf{r} - \alpha \mathbf{q}$ 
12:   $\mathbf{s} \leftarrow \mathbf{A}\mathbf{r}$ 
13:   $\delta^{(k)} \leftarrow \mathbf{r}^\top \mathbf{s}$ 
14:   $\beta \leftarrow \delta^{(k)} / \delta^{(k-1)}$ 
15: end while
```

---

The most computationally intensive operations in these Algorithms are those that involve matrix-vector products, as in lines 2 and 7 of Algorithm 3, and lines 2, 3 and 12 of Algorithm 4. This is commonly the dominant operation in regards to the time per iteration, and even more so in a matrix-free scheme. In the following Section, it will be discussed how these matrix operations can be carried out without having  $\mathbf{A}$  allocated in memory.

**Algorithm 5** PCG method**Input:**  $\mathbf{A}, \mathbf{x}_0, \mathbf{b}, \mathbf{M}^{-1}$ **Output:**  $\mathbf{x}$ 

```

1:  $\mathbf{x} \leftarrow \mathbf{x}_0, \mathbf{d} \leftarrow \mathbf{0}$ 
2:  $\mathbf{r} \leftarrow \mathbf{b} - \mathbf{A}\mathbf{x}$ 
3:  $\mathbf{s} \leftarrow \mathbf{M}^{-1}\mathbf{r}$ 
4:  $\delta^{(0)} \leftarrow \mathbf{r}^T \mathbf{s}$ 
5:  $\beta \leftarrow 0$ 
6: while stopping criteria is not met do
7:    $\mathbf{d} \leftarrow \mathbf{s} + \beta \mathbf{d}$ 
8:    $\mathbf{q} \leftarrow \mathbf{A}\mathbf{d}$ 
9:    $\alpha \leftarrow \delta^{(k-1)} / \mathbf{q}^T \mathbf{d}$ 
10:   $\mathbf{x} \leftarrow \mathbf{x} + \alpha \mathbf{d}$ 
11:   $\mathbf{r} \leftarrow \mathbf{r} - \alpha \mathbf{q}$ 
12:   $\mathbf{s} \leftarrow \mathbf{M}^{-1}\mathbf{r}$ 
13:   $\delta^{(k)} \leftarrow \mathbf{r}^T \mathbf{s}$ 
14:   $\beta \leftarrow \delta^{(k)} / \delta^{(k-1)}$ 
15: end while

```

**Algorithm 6** PMINRES (PCR) method**Input:**  $\mathbf{A}, \mathbf{x}_0, \mathbf{b}, \mathbf{M}^{-1}$ **Output:**  $\mathbf{x}$ 

```

1:  $\mathbf{x} \leftarrow \mathbf{x}_0, \mathbf{d} \leftarrow \mathbf{0}, \mathbf{q} \leftarrow \mathbf{0}$ 
2:  $\mathbf{r} \leftarrow \mathbf{b} - \mathbf{A}\mathbf{x}$ 
3:  $\mathbf{s} \leftarrow \mathbf{M}^{-1}\mathbf{r}$ 
4:  $\delta^{(0)} \leftarrow \mathbf{s}^T \mathbf{A}\mathbf{s}$ 
5:  $\beta \leftarrow 0$ 
6: while stopping criteria is not met do
7:    $\mathbf{d} \leftarrow \mathbf{s} + \beta \mathbf{d}$ 
8:    $\mathbf{q} \leftarrow \mathbf{A}\mathbf{d}$ 
9:    $\alpha \leftarrow \delta^{(k-1)} / \mathbf{q}^T \mathbf{M}^{-1} \mathbf{q}$ 
10:   $\mathbf{x} \leftarrow \mathbf{x} + \alpha \mathbf{d}$ 
11:   $\mathbf{r} \leftarrow \mathbf{r} - \alpha \mathbf{q}$ 
12:   $\mathbf{s} \leftarrow \mathbf{M}^{-1}\mathbf{r}$ 
13:   $\delta^{(k)} \leftarrow \mathbf{s}^T \mathbf{A}\mathbf{s}$ 
14:   $\beta \leftarrow \delta^{(k)} / \delta^{(k-1)}$ 
15: end while

```

### 4.3 Matrix-free operations

Matrix-free techniques for the FEM aim at making simulations feasible without storing the global matrix  $\mathbf{A}$  in memory. The idea is that trade-offs in computational cost are acceptable to avoid large data allocation. In that sense, the matrix operations seen in Algorithms 3 through 4, namely matrix-vector and preconditioner-vector products, have to be adapted so that the coefficients of  $\mathbf{A}$  may be recomputed on-the-fly from combinations of local elemental matrices  $\mathbf{A}_e$ , i.e.

$$\mathbf{A} = \sum_{e \in E} \mathbf{T}_e^T \mathbf{A}_e \mathbf{T}_e. \quad (4.31)$$

Recall that, as we are interested in solving Stokes equations,  $\mathbf{A}_e$  is an assembly of elemental  $\mathbf{K}_e$ ,  $\mathbf{G}_e$  and  $\mathbf{S}_e$ , like so

$$\mathbf{A}_e = \begin{bmatrix} \mathbf{K}_e & \mathbf{G}_e \\ \mathbf{G}_e^T & -\mathbf{S}_e \end{bmatrix}, \quad \forall e \in E, \quad (4.32)$$

and, once again,  $\mathbf{T}_e$  are projection matrices. Notice that the matrices  $\mathbf{T}_e$  serve a purpose for the numerical modeling, but they are not required in code, as all of the projection information can be inferred from DOF connectivity mapping.

When dealing with an unstructured mesh or a highly heterogeneous domain, both of which would generate an elevated variability of  $\mathbf{A}_e$ , element-by-element (EBE) strategies

are often of interest [10, 68, 85, 86, 116], as storing multiple different elemental matrices might be troublesome, and each  $\mathbf{A}_e$  can be recomputed on demand. Further, the vicinity of nodes in the mesh may be unbalanced. However, when dealing with structured periodic meshes, the vicinity of every node is balanced and can be quickly recovered from fixed-grid numbering rules, allowing for simple functions to describe “movements” in the mesh, or the connectivity. Also, if there are few unique material phases in the domain, few  $\mathbf{A}_e$  have to be stored. In that sense, node-by-node (NBN) strategies have been explored for structured meshes. It was shown by the author of this Thesis, in Lopes et al. (2021) [79], that NBN is well suited for image-based simulations, being able to provide up to  $2\times$  speed-up in comparison to a conventional EBE scheme. Therefore, an NBN approach is considered here. Additionally, notice that, as single-fluid flow is of interest, only one  $\mathbf{A}_e$  has to be allocated for the simulations. The proposed scheme is formalized as follows.

#### 4.3.1 Node-by-node

Let  $N$  and  $D$  be the set of all nodes and DOFs in the mesh, respectively. Recall that  $E$  is the set of all elements. Let  $\mathcal{E} : N \rightarrow E$  be a function that, provided a node  $n \in N$ , returns its neighboring elements. Similarly, let  $\mathcal{N} : E \rightarrow N$  be a function that returns the nodes at the corners of a provided element  $e \in E$ . Let  $\mathcal{D} : N \rightarrow D$  be a function that associates an input node to its respective DOFs. It is stressed that  $\mathcal{E}$ ,  $\mathcal{N}$  and  $\mathcal{D}$  can be implemented as fairly straightforward arithmetic operations when dealing with a fixed-grid numbering scheme in a voxel-based mesh. For completeness, let  $L$  be the set of local DOFs, within an element’s  $e \in E$  reference. Let  $\mathcal{L} : (E, D) \rightarrow L$  be a function that, given a element-DOF pair, returns the local index of that DOF. Notice that in code, it is possible to keep track of local indexes with counter variables. With these definitions, it is possible to lay out algorithms for an NBN matrix-vector product  $\mathbf{q} \leftarrow \mathbf{A}\mathbf{d}$  and a preconditioner-vector product  $\mathbf{q} \leftarrow \mathbf{M}^{-1}\mathbf{r}$ , as shown below in Algorithms 7 and 8. The process of traversing the vicinity of a node in an NBN scheme is illustrated in Figure 4.2.

**Algorithm 7** NBN matrix-vector product**Input:**  $N$ ,  $\mathbf{d}$ **Output:**  $\mathbf{q}$ 

```

1: for each  $n \in N$  do
2:    $\mathbf{q}_{\mathcal{D}(n)} \leftarrow 0$ 
3:   for each  $e \in \mathcal{E}(n)$  do
4:     for each  $d \in \mathcal{D}(n)$  do
5:        $k \leftarrow \mathcal{L}(e, d)$ 
6:        $\mathbf{q}_d \leftarrow \mathbf{q}_d + \mathbf{A}_{e[k,:]} \mathbf{d}_e$ 
7:     end for
8:   end for
9: end for

```

**Algorithm 8** NBN Diag. preconditioner**Input:**  $N$ ,  $\mathbf{r}$ **Output:**  $\mathbf{q}$ 

```

1: for each  $n \in N$  do
2:   for each  $d \in \mathcal{D}(n)$  do
3:      $p \leftarrow 0$ 
4:     for each  $e \in \mathcal{E}(n)$  do
5:        $k \leftarrow \mathcal{L}(e, d)$ 
6:        $p \leftarrow p + \mathbf{A}_{e[k,k]}$ 
7:     end for
8:      $\mathbf{q}_d \leftarrow \mathbf{r}_d / p$ 
9:   end for
10: end for

```

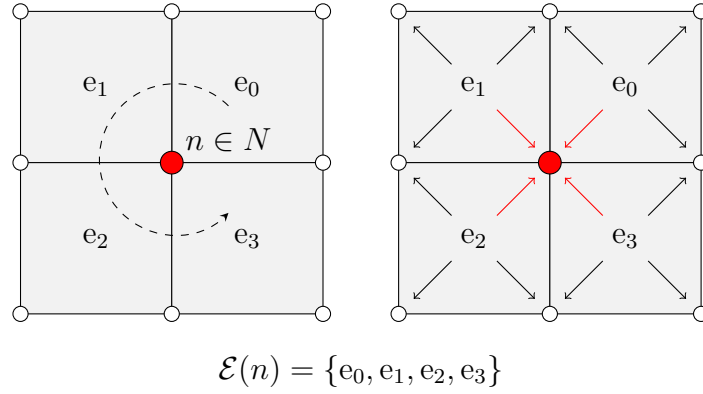


Figure 4.2: Illustration of the access patterns for a node in a NBN scheme, for a two-dimensional problem. On the left, a depiction of the order of access of neighboring elements: counterclockwise. On the right, the arrows indicate memory accesses to data associated with each neighboring node, and floating point multiplications. Notice that each node has 3 DOFs in 2D (or 4 DOFs, in 3D). The arrows highlighted in red depict such operations with the current node. The notion carries to three-dimensional problems, where there are 8 voxels, arranged in “layers” of 4 voxels.

In Algorithms 7 and 8, the notation  $\mathbf{A}_{e[i,j]}$  is used to depict the term at row  $i$  column  $j$  of  $\mathbf{A}_e$ , with “:” meaning either all rows or all columns. Notice that the inner loops over  $\mathcal{E}(n)$  and  $\mathcal{D}(n)$  are constant-sized and can be unrolled in code. Further, if the loop over  $N$  is parallelized, no concurrent access to write in  $\mathbf{q}$  happens. The NBN strategy naturally eliminates race conditions. On the other hand, it is important to perceive in Algorithm 7 that accesses to read from  $\mathbf{d}$  are repeated for neighboring nodes that belong to more than one neighboring element, as it can be seen in Figure 4.2. It is possible to do better. By combining the coefficients of  $\mathbf{A}_e$  for the DOFs around each node, i.e. precomputing the contribution of the local matrices of the surrounding elements, it is possible to operate

on, and thus to read, the entries of  $\mathbf{d}$  associated with each surrounding DOF just once. In a nutshell, the idea is to work with stencils instead of element-wise local matrices. This notion will be further detailed in the next Section.

### 4.3.2 Stencil-by-stencil

Given the fixed-grid nature of the considered image-based meshes and the Q1Q1 elements, the compact support of the shape functions associated with each node is always the same: a  $2 \times 2 \times 2$  voxels cube (or a  $2 \times 2$  pixels square, in 2D) centered at the node in question. Further, as only one fluid is considered, there is only one  $\mathbf{A}_e$  to be accounted. In light of this, the traversing of neighbor elements in the NBN scheme always yields the same matrix coefficients for inner pore nodes. These can be assembled in a stencil, like such

$$\mathbf{L}_n = \sum_{e \in \mathcal{E}(n)} \mathbf{A}_{e[k,:]} \mathbf{T}_e^{(\mathbf{L})}, \quad \forall n \in N, \quad \text{with } k = \mathcal{L}(e, \mathcal{D}(n)), \quad (4.33)$$

where  $\mathbf{L}_n$  represents an array of stencil coefficients. The same stencil can be used for all nodes within pores, that is, nodes that are completely surrounded by pore voxels. The caveat is that specific stencils have to be assembled for solid-pore interface nodes, where not every surrounding voxel corresponds to an element in the mesh, and the no-slip boundary condition has to be considered. Notice that the terms in  $\mathbf{L}_n$  are the non-zero coefficients that would be stored in the global matrix in sparse format, but instead of allocating multiple repeated lines of such matrix, the data is allocated just once for the inner pore nodes, plus some stencils for edge cases at the solid-pore interface.

For three-dimensional problems, the stencil has  $4 \times 27 \times 4$  coefficients, as each node has 4 DOFs, 27 neighbor nodes (including itself), and each of the neighbors has 4 DOFs as well. This amounts to 432 coefficients. In regards to matrix-free matrix-vector products, this means 432 multiplications per node (or thread), in comparison to the  $4 \times 8 \times 32 = 1024$  multiplications that were required for the NBN scheme in Algorithm 7. Evidently, the stencil-by-stencil (SBS) approach roughly halves the amount of floating point multiplications to be performed in a matrix-vector product, in comparison to NBN. The SBS approach, as depicted in Figure 4.3, can be formalized as follows.

Let  $S$  be the set of unique stencils in the mesh, per DOF. Let  $\mathcal{S} : D \rightarrow S$  be a function that, provided a DOF, returns its stencil. Let  $\mathcal{X} : N \rightarrow N$  be a function that maps all of the neighboring nodes from a given node. Finally, let  $V$  be the set of local DOFs with respect to the vicinity of a node  $n \in N$ , and let  $\mathcal{V} : (N, D) \rightarrow V$  be a function that,

given a node-DOF pair, returns the local index of that DOF with respect to that node. Then, SBS matrix-vector products and preconditioner-vector products can be defined as in Algorithms 9 and 10. Notice that  $\mathcal{X}$  can be used to find all  $\mathbf{d}_e$ 's in a NBN scheme, without any extra connectivity information.

---

**Algorithm 9** SBS matrix-vector product
 

---

**Input:**  $N, \mathbf{d}$ 
**Output:**  $\mathbf{q}$ 

```

1: for each  $n \in N$  do
2:   for each  $d \in \mathcal{D}(n)$  do
3:      $\mathbf{q}_d \leftarrow 0$ 
4:      $\mathbf{L}_n \leftarrow \mathcal{S}(d)$ 
5:     for each  $j \in \mathcal{D}(\mathcal{X}(n))$  do
6:        $k \leftarrow \mathcal{V}(n, j)$ 
7:        $\mathbf{q}_d \leftarrow \mathbf{q}_d + \mathbf{L}_{n[k]} \mathbf{d}_j$ 
8:     end for
9:   end for
10: end for

```

---



---

**Algorithm 10** SBS Diag. preconditioner
 

---

**Input:**  $N, \mathbf{r}$ 
**Output:**  $\mathbf{q}$ 

```

1: for each  $n \in N$  do
2:   for each  $d \in \mathcal{D}(n)$  do
3:      $k \leftarrow \mathcal{V}(n, d)$ 
4:      $\mathbf{L}_n \leftarrow \mathcal{S}(d)$ 
5:      $\mathbf{q}_d \leftarrow \mathbf{r}_d / \mathbf{L}_{n[k]}$ 
6:   end for
7: end for

```

---

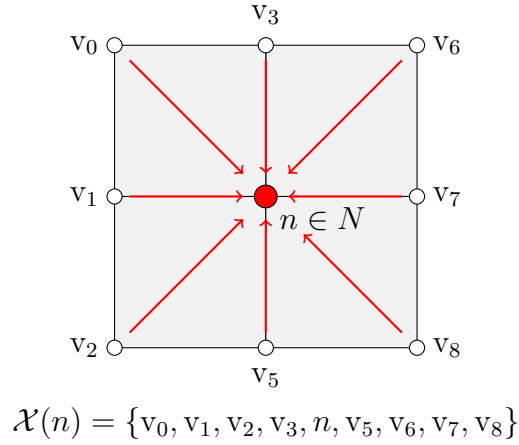


Figure 4.3: Illustration of the access patterns for a node in a SBS scheme, for a two-dimensional problem. The arrows indicate memory accesses and floating point multiplications. As for NBN, the notion carries to three-dimensional problems.

Both the NBN and SBS approaches were considered for the implementation of two- and three-dimensional Stokes flow simulations in GPU. To avoid having to compute multiple stencils for edge cases at the solid-pore interface, separate kernels are launched for DOFs within pores and at the border, as described in the next Chapter. In other words, in this Thesis, it is proposed to implement SBS operations just for the inner pore nodes, and to employ NBN for the border nodes. In doing so, just one stencil is stored, and the

bulk of operations involving nodes with non-null velocity is dealt efficiently, while the nodes at the border test their vicinity on demand and recompute local coefficients from the  $\mathbf{A}_e$ 's of surrounding elements.

## 4.4 Image-based pore space meshing

The images considered as input in the scope of this work can be seen as monolithic fixed-grid meshes. By adopting simple element (voxel) and node numbering rules that follow this structured nature, e.g. a column-major numbering scheme<sup>1</sup>, all of the connectivity information can be straightforwardly resolved with arithmetic operations on element and node indexes. One can think about this as laying out a set of rules to “walk” on the mesh, or to obtain the vicinity of any node or element [81]. Notice that this is a significant simplification from conventional FEM implementations for unstructured meshes, where neighboring indexes often cannot be computed on-the-fly. But there is an important issue to be dealt with: in fluid flow analyses, only the pore space  $\Omega_p$  is meshed, not the whole periodic cell  $\Omega$ .

A conventional approach to model just the pore space, or to characterize a pore “skeleton” within the image, would be to store connectivity maps and perhaps even topological data structures to represent the unstructured mesh. Here, it is aimed to expand on the node-based fixed-grid approach, employing a numbering scheme that allows for quick correspondence between node and DOF indexes, and for the aforementioned set of rules to “walk” in the mesh to remain valid. The mesh will be numbered like so:

1. Column-major numbering of the nodes within  $\Omega_p$ . In doing so, count the number of internal pore nodes.
2. Column-major numbering of the nodes at  $\Gamma$ , starting from the number of internal pore nodes. Count the number of border nodes.

These rules are illustrated in Figure 4.4 for a two-dimensional mesh, for ease of visualization. The principle carries to three-dimensional problems.

---

<sup>1</sup>top to down, then left to right, then near to far.

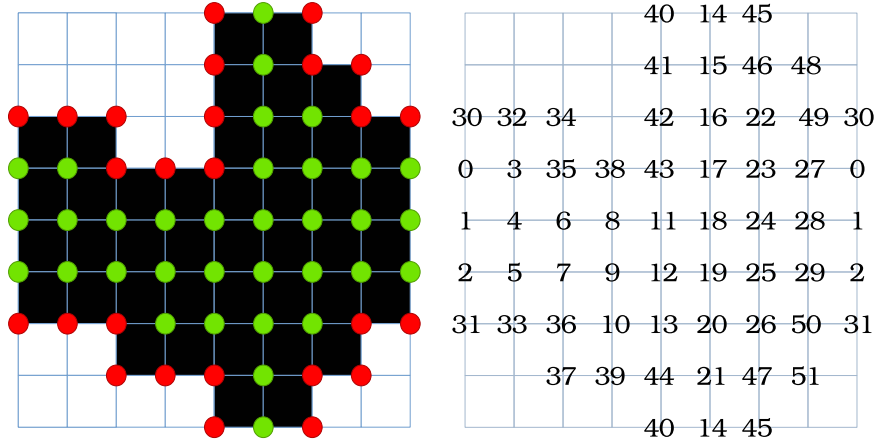


Figure 4.4: Image-based pore meshing strategy. To the left: Definition of internal pore nodes (green), within  $\Omega_p$ , and border nodes (red), at  $\Gamma$ . Solid phase identified by white voxels and pore space represented by black voxels. To the right: The application of the proposed node numbering rules to this mesh. Notice that the periodic boundary conditions at  $\partial\Omega$  lead to matching indexes in opposite faces.

Notice that these rules arrange the nodes such that the indexes of their respective DOFs can be computed from each node index, i.e. no DOF map is required. The internal pore nodes have velocity and pressure variables, so their DOFs can be mapped as

$$\begin{cases} \text{DOF}_n^{(u_1)} = 3n, \\ \text{DOF}_n^{(u_2)} = 3n + 1, \\ \text{DOF}_n^{(u_3)} = 3n + 2, \\ \text{DOF}_n^{(p)} = 3(\# \text{ pore nodes}) + n, \end{cases} \quad \forall n \in \{0, 1, 2, \dots, \# \text{ pore nodes} - 1\}, \quad (4.34)$$

where  $n$  is a node index,  $\text{DOF}_n^{(u_i)}$  are indices of the velocity DOFs of the node  $n$ , and, likewise,  $\text{DOF}_n^{(p)}$  are pressure indices. On the other hand, due to the no-slip boundary condition at  $\Gamma$ , the velocity field is null for every border node, so it is not a variable. It follows then that border nodes have only pressure variables, with their indices analogously mapped as

$$\text{DOF}_n^{(p)} = 3(\# \text{ pore nodes}) + n, \quad \forall n \in \{\# \text{ pore nodes}, \dots, \# \text{ nodes} - 1\}. \quad (4.35)$$

Notice that these rules are valid for three-dimensional problems. For two-dimensional simulations, they are similar:

$$\begin{cases} \text{DOF}_n^{(u_1)} = 2n, \\ \text{DOF}_n^{(u_2)} = 2n + 1, \\ \text{DOF}_n^{(p)} = 2(\# \text{ pore nodes}) + n, \end{cases} \quad \forall n \in \{0, 1, 2, \dots, \# \text{ pore nodes} - 1\}, \quad (4.36)$$

$$\text{DOF}_n^{(p)} = 2(\# \text{ pore nodes}) + n, \quad \forall n \in \{\# \text{ pore nodes}, \dots, \# \text{ nodes} - 1\}. \quad (4.37)$$

This separation of internal pore and border nodes via numbering allows for a seamless decomposition of the algebraic operations for each kind of node in matrix-vector products. This is a vital asset in the implementation of separate CUDA kernels to handle pore and border nodes.

Returning to the matrix-free operations shown in the last Section, synthesized in Algorithms 7 through 10, notice that these node numbering and DOF mapping rules allow for  $\mathcal{E}$ ,  $\mathcal{N}$  and  $\mathcal{D}$  to be defined as static *lambda* functions instead of maps to be allocated in memory. They can even be implemented in code as macros. Further, it is interesting to observe that element indexes do not matter much in this context, as all pore voxels depict the same fluid. Nevertheless, an important component is still lacking: the function  $\mathcal{X}$ , responsible for mapping the vicinity of any node, must be defined. It is useful not only for the SBS approach, but for the NBN as well, in finding the DOFs of the surrounding elements. In practice,  $\mathcal{X}(n)$  can be used to compute  $\mathcal{N}(\mathcal{E}(n))$ .

The rules defined so far in this Section do not allow for “walk in the mesh” operations to be determined. However, we know that the monolithic grid associated to the whole image has operations of this sort embedded in its structured numbering, which is presented in Figure 4.5. In that sense, all we need for  $\mathcal{X}$  to be defined in the pore space are maps that translate node indexes from the discretization of  $\Omega_p \cup \Gamma$  to the discretization of  $\Omega \cup \partial\Omega$  and vice-versa. These will be depicted by

$\text{M}_2\text{I} \Rightarrow \text{Mesh to Image map},$

$\text{I}_2\text{M} \Rightarrow \text{Image to Mesh map}.$

Notice that  $\text{M}_2\text{I}$  always yields an image vertex index, while  $\text{I}_2\text{M}$  may provide null indices if the node in question would lie out of the mesh (out of  $\Omega_p$ ).  $\mathcal{X}$  can then be expressed as in Algorithm 11.

**Algorithm 11**  $\mathcal{X} : N \rightarrow N$ 

“Get vicinity of a node in the mesh”

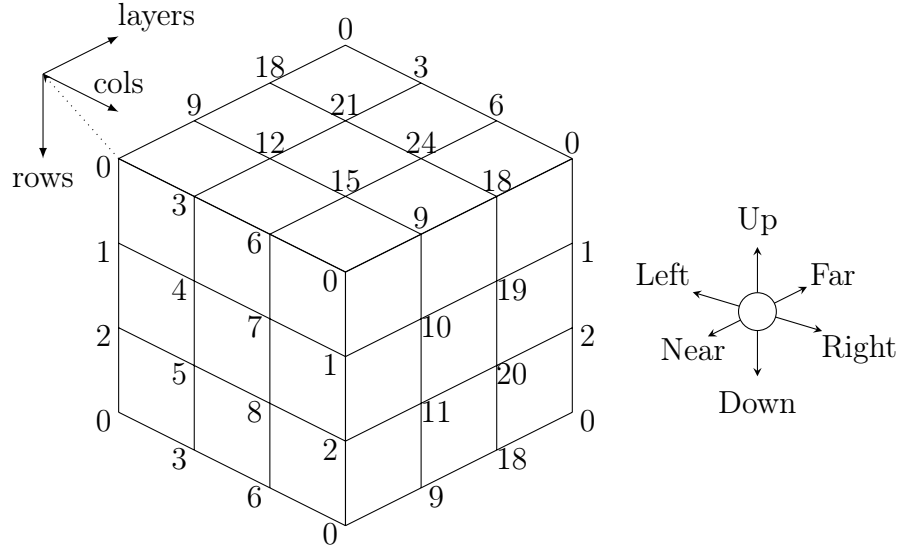
**Input:**  $n \in N$ **Output:**  $\nu \subset N$ 1: vertex  $\leftarrow M_2I(n)$  $\triangleright \nu$  is the set of surrounding nodes to the node  $n$  $\triangleright$  From mesh to image indexes2: neighbors  $\leftarrow$  vicinity of vertex $\triangleright$  in the structured image (Figure 4.5, Table 4.1)3:  $\nu \leftarrow I_2M(\text{neighbors})$  $\triangleright$  Back to mesh indexes

Figure 4.5: Periodic vertex numbering scheme in the monolithic grid that characterizes an image. Notice that the operations to compute the vicinity of any vertex ( $\{\text{Up, Down, Right, Left, Near, Far}\}$ ) can be straightforwardly implemented via arithmetic expressions with modulus operators, as indicated in Table 4.1.

| Signature                                                                                           | Definition                                                                                                                                          |
|-----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| $\text{rclnum}(v_{\text{id}})$<br>“ $v_{\text{id}}$ to (row,col,layer)”                             | $(\text{row}, \text{col}, \text{layer}) = (v_{\text{id}} \bmod R, \lfloor v_{\text{id}} / R \rfloor, \lfloor v_{\text{id}} / (R \times C) \rfloor)$ |
| $\text{periodicnum}(\text{row}, \text{col}, \text{layer})$<br>“(row,col,layer) to $v_{\text{id}}$ ” | $v_{\text{id}} = (\text{row} \bmod R) + (\text{col} \bmod C) \times R + (\text{layer} \bmod L) \times (R \times C)$                                 |
| $\text{Up}(\text{row}, \text{col}, \text{layer})$                                                   | $v_{\text{id}} \leftarrow \text{periodicnum}(\text{row}-1, \text{col}, \text{layer})$                                                               |
| $\text{Down}(\text{row}, \text{col}, \text{layer})$                                                 | $v_{\text{id}} \leftarrow \text{periodicnum}(\text{row}+1, \text{col}, \text{layer})$                                                               |
| $\text{Left}(\text{row}, \text{col}, \text{layer})$                                                 | $v_{\text{id}} \leftarrow \text{periodicnum}(\text{row}, \text{col}-1, \text{layer})$                                                               |
| $\text{Right}(\text{row}, \text{col}, \text{layer})$                                                | $v_{\text{id}} \leftarrow \text{periodicnum}(\text{row}, \text{col}+1, \text{layer})$                                                               |
| $\text{Near}(\text{row}, \text{col}, \text{layer})$                                                 | $v_{\text{id}} \leftarrow \text{periodicnum}(\text{row}, \text{col}, \text{layer}-1)$                                                               |
| $\text{Far}(\text{row}, \text{col}, \text{layer})$                                                  | $v_{\text{id}} \leftarrow \text{periodicnum}(\text{row}, \text{col}, \text{layer}+1)$                                                               |

Table 4.1: Computational functions that allow straightforward vicinity (connectivity) characterization in a periodic structured grid, following a column-major numbering scheme. Note that  $(R, C, L)$  are the numbers of rows, columns and layers in the grid, respectively, and  $v_{\text{id}}$  is used to depict a vertex’s index.

Additionally, a third concise list can be stored in memory to aid in boundary checking so that any solid voxels around a node can be identified and properly avoided in NBN schemes. Naturally, the internal pore nodes are only surrounded by fluid elements, so no check is necessary for them. The border nodes, however, are required to be aware of their vicinity. In light of this, a list of one 8-bit key per border node is assembled. Each bit carries information whether its respective neighbor element (there are always 8 in  $\mathbb{R}^3$ ) is a solid or pore voxel, as illustrated in Figure 4.6.

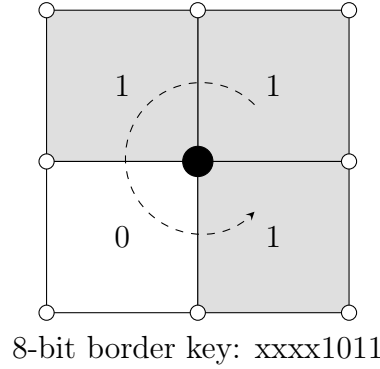


Figure 4.6: Illustration of the 8-bit keys employed to map the solid-pore interface, for a two-dimensional problem. In three-dimensional problems, the first 4 bits would have been used for the other “layer” of voxels.

To sum up, three linear data structures are stored in memory for the proposed pore meshing strategy:

1. A list of one `uint32` value per image vertex:  $I_2M$ ;
2. A list of one `uint32` value per mesh node:  $M_2I$ ;
3. A list of one `uint8` value per border node.

Notice that the first list grows linearly with respect to the amount of voxels in the image, while the second and third ones are proportional to and smaller than the first, also being functions of the porosity of the sample. Further, notice that no “color per voxel” list must be stored, nor any connectivity matrix.

# Chapter 5

## Proposed implementation

### 5.1 Massively parallel GPU programming

Throughout this chapter, specific notions and terminology related to GPUs are used in the description of the proposed implementation. In that sense, this Section is introduced as a primer on basic concepts that should aid readers who are not familiar with the topic in understanding the ideas exposed in Sections ahead. Readers who are already versed in GPU programming may skip to Section 5.2. Here, the massively parallel architecture of graphics cards will be presented, alongside some important aspects of the CUDA programming model. This Section is based on the CUDA documentation [99], and the books by Sanders and Kandrot [122], and Kirk and Wu [69]. A similar Section was present in the Author's Master's Dissertation [79] as well.

#### 5.1.1 A brief introduction

Graphics Processing Units (GPUs), often interchangeably referred as graphics cards, were designed as massively parallel machines to accelerate the rendering of images. To put it simply, their original function was to read coalesced data regarding geometry and color, for example, process those through a list of instructions being run on multiple Arithmetic Logic Units (ALUs) and Floating Point Units (FPUs), and obtain final colors to paint each pixel on screen. This workflow is commonly denominated as the *graphics pipeline*, especially in the context of OpenGL. These ALUs and FPU were not originally required to be particularly powerful individually, in comparison to those within a CPU, but it was important that as many as possible computations could take place in parallel, so that pixels could be updated in real time. In other words, GPU manufacturers would stride

primarily for improvements in throughput, not necessarily in latency. The capability of the hardware to handle large algebraic operations efficiently came from investment in parallelism. Naturally, its potential for non-graphical applications became evident with time. Nowadays, besides rendering, graphics cards are being widespread employed for a variety of computationally intensive applications, as in AI training, Scientific Computing, and Cryptography. A comparison between the architectures of GPUs and CPUs is presented in Figure 5.1.

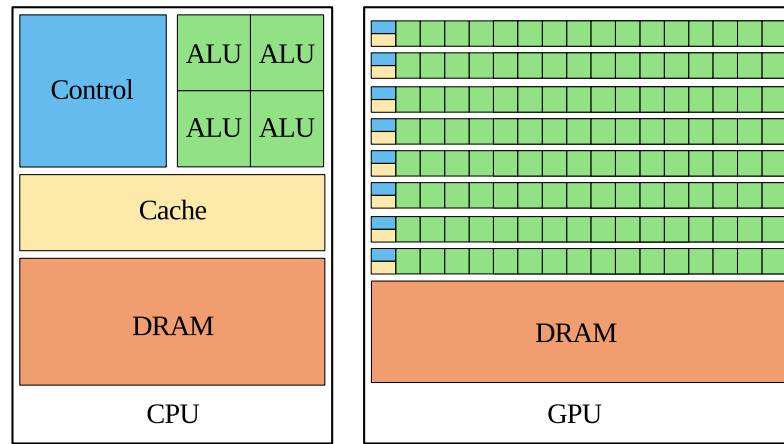


Figure 5.1: CPU vs. GPU architecture. Based on a figure seen in Kirk and Hwu [69].

In Figure 5.1, notice that multiple ALUs and FPUs in a GPU share the same control unit and memory cache. The famous *Single Instruction Multiple Threads* (SIMT) paradigm arises from this, as the same instructions are cached to be run on groups of processors. The variability in behavior is usually limited to data access patterns, which, in CUDA, can be controlled within each thread via built-in variables that hold thread, block and grid indexes. The concepts of “blocks” and “grids” are properly defined shortly ahead, in Subsection 5.1.3.

### 5.1.2 Memory

GPUs have their own memory hierarchy that affects how threads access data. Considerable performance gains can be obtained with proper use of local, shared and global memory. Figure 5.2 illustrates such memory structure within a GPU. Each component in that Figure can be described as follows:

- *Streaming multiprocessors* (SMs) are clusters of cores (the ALUs and FPUs in Figure 5.1), where blocks of threads are run.

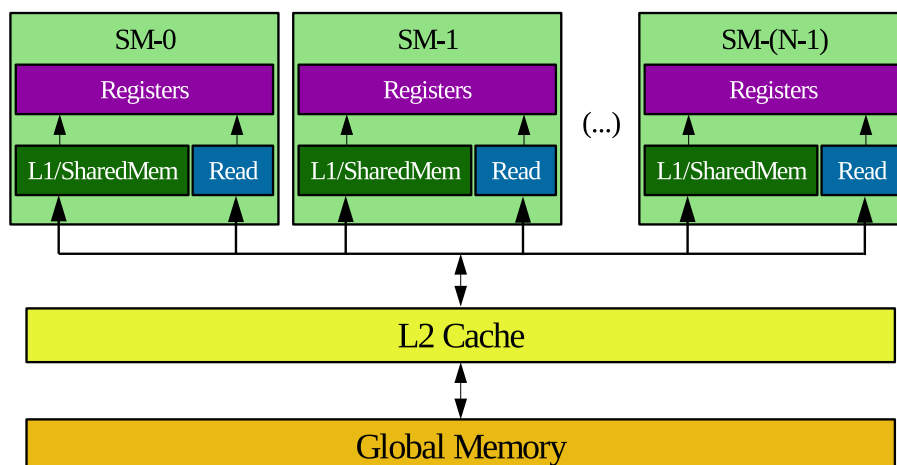


Figure 5.2: Memory hierarchy of a CUDA-enabled device. Based on a Figure seen in Gupta [50].

- The global memory is accessible to every active thread. It corresponds to the RAM of the GPU, having the most space, but requiring more machine cycles for data to be accessed.
- The L2 cache is shared among all SMs. It allows faster memory access, but has limited space. Good usage of this cache usually demands well planned data access patterns to the global memory. Perfectly coalesced (aligned) data will generally yield more cache hits, and thus faster read operations.
- The L1/Shared memory is a fast-access memory cache within each SM that can be reached locally by every thread running in that SM. It is particularly useful for operations such as reductions, where threads in a block need to share data among themselves. It is limited in size, but a vital tool for parallel algorithms that need to be aware of neighboring data.
- The Read-only memory is an on-chip cache commonly employed for constant memory. It can speed up access to constant-value small-sized data that needs to be visible to all threads and has a common access pattern.
- The Registers are used to store local variables for each thread, and have their usage controlled at compiler level.

From Figure 5.2 and the above bullet point descriptions, notice that memory spaces with more localized scope, such as the L1 shared memory cache and the registers, can be used to hold SM- or thread-wise temporary values. This can be useful for long SIMD-like algebraic operations. For example, consider the piece-wise addition of a vector and

a matrix-vector product:  $\mathbf{w} \leftarrow \mathbf{u} + \mathbf{M}\mathbf{v}$ . In this simple example,  $\mathbf{u}$ ,  $\mathbf{v}$  and  $\mathbf{w}$  would be large vectors and  $\mathbf{M}$  is a large sparse matrix. In a vector-based language, it might be tempting to write a single line of code to handle such operation. However, if not planned carefully, such a line might require the allocation of a temporary large vector to accumulate the result of the matrix-vector product ( $\mathbf{M}\mathbf{v}$ ), prior to computing the vector addition ( $\mathbf{u} + \mathbf{M}\mathbf{v}$ ) and then writing to the results. In the context of GPU programming, we can implement specific massively parallel functions that store piece-wise intermediate results in ephemeral local-scope memory. In doing so, long algebraic expressions may be evaluated with powerful “one liner” calls from the CPU without having to resort to possibly strenuous temporary allocation in global memory. This sort of reasoning will be explored in the implementations presented in Section 5.5 of this Chapter.

### 5.1.3 The CUDA framework

The *Compute Unified Device Architecture* (CUDA) framework is a collection of proprietary software from Nvidia dedicated to enabling usage of their graphics cards for general purpose GPU programming. These include extensive APIs for C/C++ and FORTRAN. It is worth mentioning that CUDA has also been extending its reach to modern higher level languages such as Python and Julia. In the scope of this Thesis, all purely CPU code has been implemented in C, and the heterogeneous computing (CPU/GPU) code has been implemented in CUDA C++. Even though CUDA code is similar to conventional C++, some specific concepts regarding GPU programming are essential for this sort of application, and have been hinted at throughout this Section, such as:

- The CPU is referred as *host*, while the GPU is called *device*.
- The global memory of the GPU is not the same as the RAM of the CPU, so data transfer between the two is often required. These operations are referred as `memcpy`'s. Note, however, that recent hardware development points towards efficient shared memory between host and device, so it is possible that a paradigm shift in this regard may occur in the years to come.
- In comparison to CPUs, GPUs can manage large numbers of threads at a time. For example, in a vector operation, it is customary to launch one thread per entry of the input vector. In CUDA, threads run in packs of 32, called *warps*, and are grouped in *blocks*.

- Blocks are capped at 1024 threads, and are usually created with lower dimensions (always a power of 2, starting from 32). They can be abstractly characterized as 1D, 2D or 3D, to better represent the data to be worked on. Each block is assigned to a single SM. Groups of blocks are part of a *grid*. In a single-GPU system, usually, there is just one grid.
- The functions that run in GPU are called *kernels*. They can have a global scope, being callable from host and device, or be restricted to calls from the device. A kernel call from the host initiates multiple blocks of threads to perform the programmed instructions in parallel.
- Kernel calls from the host are non-blocking. When required, synchronization between host and device has to be enforced. Further, kernels do not issue automatic `memcpy` calls, so data transfer between host and device has to be managed in code as well.

## 5.2 An outlook of the developed code

The solvers developed in the scope of this Thesis were designed with memory efficiency as a primary target. More specifically, the goal was to make large-scale image-based permeability analyses, with hundreds of millions of DOFs, feasible in desktop computers equipped with consumer-grade graphics cards. In this regard, the main idea that guided the implementation, besides the matrix-free approach, was to explore possibilities to avoid the allocation in global memory of commonly stored arrays of variables, in the context of the CG and MINRES methods. Recall that limited memory availability is often a decisive factor in arguments for simulations of this nature to be ported to supercomputers or clusters. The employment of massively parallel GPU programming aims at tackling “memory vs. computational cost” trade-offs with the hardware’s high throughput for SIMT operations. These trade-offs arise from having to recompute the coefficients of large matrices and vectors instead of storing them, which is a tenet of matrix-free schemes.

The code was implemented in C and CUDA C++, to be run on machines equipped with Nvidia GPUs. Namely, all of the higher level (closer to the mathematical modeling) homogenization functions were written in C, making use of CPU parallelism with OpenMP. This includes all tasks related to input/output, voxel-based meshing (building maps), assembling local matrices and right-hand side vectors, and computing effective fields at the macroscale. It is stressed that none of these comprehend significant bottlenecks for the application, neither in memory nor in overall processing time. Solving the

large sparse systems of equations from the FEM is, by far, the most troublesome step in this scheme, in regards to computational resources. For large (representative) enough models, the time required to solve the systems is essentially the same as that of the whole analysis, as all of the other tasks demand processing time of lower orders of magnitude. In that sense, the solvers were implemented in CUDA C++. The iterative process is controlled with single-threaded host code that calls CUDA kernels to employ the GPU for large algebraic operations, such as vector scaling, dot products, and matrix-vector products.

An important matter in heterogeneous computing scenarios like this is the management of data transfer between the involved hardware. Memory has to be properly allocated in both host and device, and arrays often have to be copied (`memcpy`'ed) from one to the other. Excessive amounts of, or poor timing in, these transactions can completely undermine performance gains obtained from massive parallelization [99, 69, 122]. In this implementation, all of the vectors related to the Krylov subspace solvers ( $\mathbf{x}, \mathbf{r}, \mathbf{d}, \mathbf{q}, \mathbf{s}$ ) reside only in arrays stored in the global memory of the GPU, and no large arrays are transferred in either direction (host to device, or device to host) during the main `while` loop that controls the iterations. This means that the solvers do not “hang” waiting for large data transactions between CPU and GPU to finish at every iteration. `memcpy`'s are kept to a minimum, only transferring from device to host two double precision scalars (results of dot products) at each iteration. Schematics of the memory management throughout a run of the developed program are illustrated on top of the flowchart presented in Figure 5.3.

In Figure 5.3, a high level depiction of the flow of activities in the proposed implementation is shown. Notice that all of the process blocks that correspond to CUDA C++ code, including host functions and device kernels, were highlighted in yellow. It is stressed that the loop until convergence of the solver is the most computationally demanding task of the program. As indicated, each iteration involves multiple kernel calls, from relatively straightforward vector operations to more intricate matrix-free operations, as `mtxfreepod`, which will be detailed ahead. In a nutshell, all of the algebraic “heavy work” is delegated to the GPU.

In the following Section, the kernels that enable the proposed matrix-free operations are exposed in pseudo-code that details how they were implemented. Then, in the following Sections, it is discussed how the solvers have been written in code, from their most “conventional” form, seen in Algorithms 3 to 6, to alternative solutions that require less memory (fewer vectors) to be allocated.

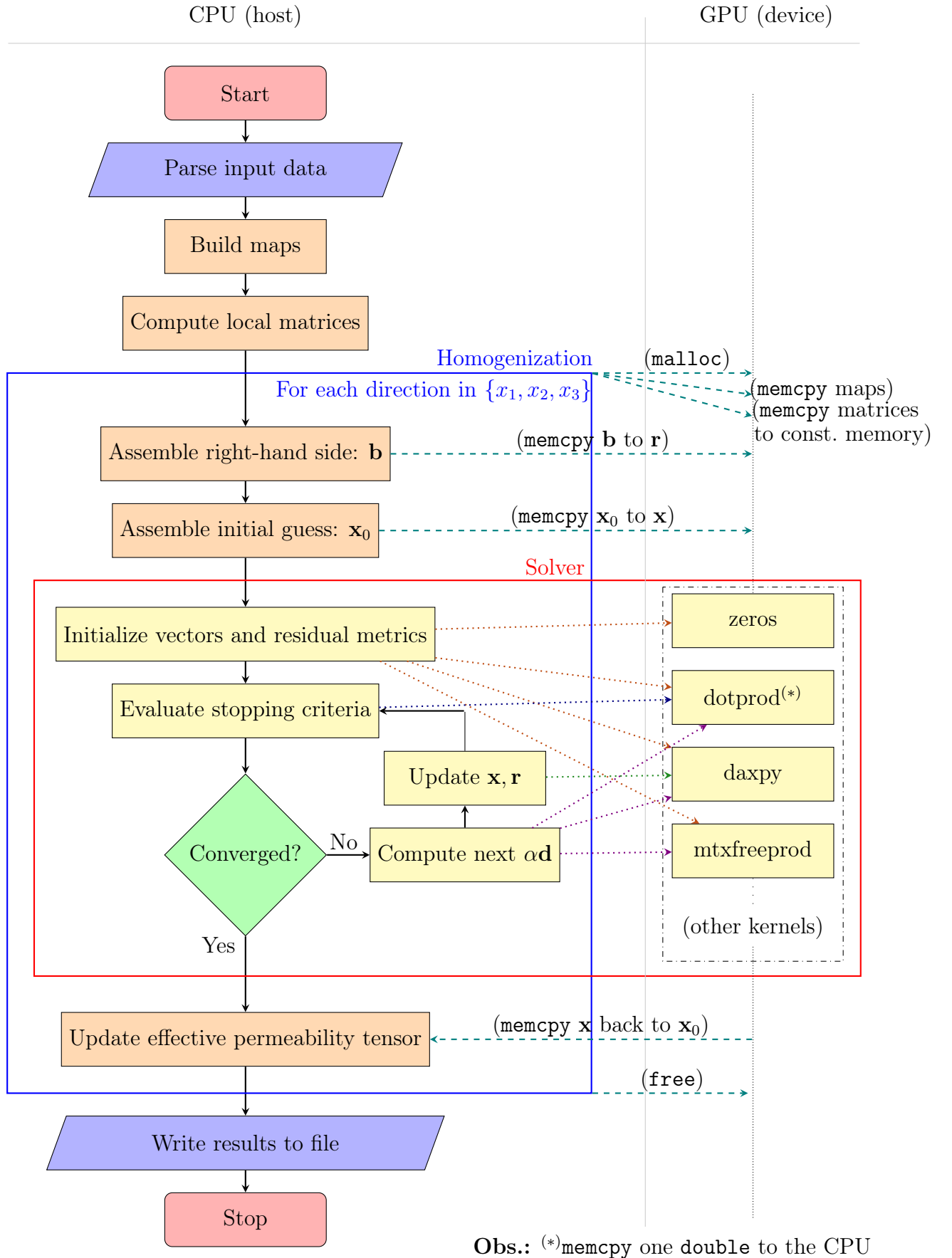


Figure 5.3: Flowchart schematics to illustrate the functioning of the developed program.

## 5.3 Kernels for matrix-free operations

The matrix operations in (P)CG and (P)MINRES can be particularly expensive for large-scale systems, in terms of both memory and processing time. For that reason, it is usual to embed considerations about the application in the code. For example, FEM solvers commonly use the global matrix's sparsity and symmetry to avoid unnecessary memory allocation and computations. Here, following a matrix-free scheme, the idea is to employ the rules for the sparse assembly of the global matrix to recompute its coefficients on demand, without ever allocating a large two-dimensional array in memory. Naturally, there is a significant compromise in computational cost to this sort of approach. In that sense, CUDA kernels were implemented to handle matrix-vector and preconditioner-vector products, and derivations of these that involve reductions and `axpy`-like operations. Evidently, other kernels for conventional vector operations were also implemented.

As discussed at the end of Chapter 4, two strategies that are well suited for image-based meshes were considered: NBN and SBS. Notice that these two are akin to one another, with SBS being a natural extension of NBN to perform less computations. The adopted node numbering rules (see Figure 4.4) allow for the operations with inner pore nodes and border nodes to be separated. This is advantageous because pore nodes (usually responsible for the majority of the DOFs) have velocity variables to be accounted and do not have to check their surrounding elements for possible solid voxels, while border nodes have only pressure variables and must be aware of neighboring voxels. In separating these behaviors, unnecessary boundary checks are avoided for the bulk of algebraic operations in matrix-vector products, and each type of node gets its own treatment in code. Further, to avoid allocating a list of stencils for edge cases in border nodes, it was decided to work with the NBN scheme at the solid-pore interface, while SBS is employed within pores. All things considered, these matrix-free operations were developed:

1. SBS matrix-vector product for pore nodes (Algorithm 12).
2. NBN matrix-vector product for border nodes (Algorithm 13).
3. SBS preconditioner-vector product for pore nodes (Algorithm 14).
4. NBN preconditioner-vector product for border nodes (Algorithm 15).

Specific device functions were implemented in code for each of the operations listed above, for two- and three-dimensional simulations. The goal was for these to be called from

within the GPU, being reusable for different kernels, which is useful for the alternative solvers detailed in Section 5.5. Pseudo-code for each of these functions is presented in Algorithms 12 through 15. There, notice that **bkeys** represents the list of one **uint8** key per border node. These algorithms depict in detail how the more conceptual description of the NBN and SBS schemes (Algorithms 7 to 10) were actually implemented for the adopted Q1Q1 finite elements. Notice that, now, these procedures are not generic, but were tailored specifically for the needs of the proposed simulations.

In Algorithms 12 through 15, notice that the node index  $n$  is obtained in terms of the current thread index (**threadId**). As implied, one CUDA thread is launched for every node of the mesh, that is, ( $\#$  pore nodes) threads are used for the inner pore node operations, and ( $\#$  border nodes) threads are called for the border node operations. Evidently, as CUDA threads run in warps of 32, the actual number of launched threads is a multiple of 32, hence the check if the current index is within bounds of the data to be dealt with. Herein, it was decided to work with blocks of 128 threads. Further, it is vital to perceive that all of the nested loops over a node's own DOFs or its vicinity are constant-sized and can be unrolled in the code.

---

**Algorithm 12** SBS **mtxfreeprod** kernel for pore nodes – 3D

---

**Input:**  $\mathbf{d}$ ,  $\mathbf{I}_2\mathbf{M}$ ,  $\mathbf{M}_2\mathbf{I}$ , ( $\#$  pore nodes)  $\triangleright \mathbf{I}_2\mathbf{M}$  and  $\mathbf{M}_2\mathbf{I}$  are used in  $\mathcal{X}(n)$  (see Alg. 11).

**Output:**  $\mathbf{q}$

```

1:  $n \leftarrow \text{threadId}$ 
2: if  $n < (\# \text{ pore nodes})$  then
3:   for each  $d \in \{3n, 3n + 1, 3n + 2, 3(\# \text{ pore nodes}) + n\}$  do
4:      $\mathbf{L} \leftarrow \mathcal{S}(d)$   $\triangleright$  hard-coded stencil, same for the DOFs of any  $n$ 
5:      $\mathbf{q}_d \leftarrow 0$ 
6:     for each  $(i, j) \in (\mathcal{X}(n), \{0, 1, \dots, 26\})$  do  $\triangleright$  27 neighbor nodes
7:       if  $i < (\# \text{ pore nodes})$  then  $\triangleright$  check if neighbor  $i$  has velocity DOFs
8:          $k \leftarrow 3 * j$ 
9:          $l \leftarrow 3 * i$ 
10:         $\mathbf{q}_d \leftarrow \mathbf{q}_d + (\mathbf{L}_{[k]} * \mathbf{d}_l$ 
            $+ \mathbf{L}_{[k+1]} * \mathbf{d}_{l+1}$ 
            $+ \mathbf{L}_{[k+2]} * \mathbf{d}_{l+2})$ 
11:      end if
12:       $k \leftarrow 3 * (\# \text{ pore nodes}) + j$ 
13:       $l \leftarrow 3 * (\# \text{ pore nodes}) + i$ 
14:       $\mathbf{q}_d \leftarrow \mathbf{q}_d + \mathbf{L}_{[k]} * \mathbf{d}_l$ 
15:    end for
16:  end for
17: end if
```

---

**Algorithm 13** NBN mtxfreeprod kernel for border nodes – 3D**Input:**  $\mathbf{d}$ ,  $I_2M$ ,  $M_2I$ ,  $bkeys$ , ( $\#$  pore nodes), ( $\#$  border nodes)**Output:**  $\mathbf{q}$ 

```

1:  $n \leftarrow \text{threadId}$ 
2: if  $n < (\# \text{ border nodes})$  then
3:    $\text{borderkey} \leftarrow bkeys_{[n]}$ 
4:    $n \leftarrow n + (\# \text{ pore nodes})$   $\triangleright$  offset threadId index to match numbering scheme
5:    $(d, k) \leftarrow (3(\# \text{ pore nodes}) + n, 24)$   $\triangleright$  pressure DOF
6:    $\mathbf{q}_d \leftarrow 0$ 
7:   for each  $(e, l) \in (\mathcal{E}(n), \{0, 1, \dots, 7\})$  do  $\triangleright$  8 neighbor voxels
8:     if  $(\text{borderkey} \gg l) \ \& \ 1$  then  $\triangleright$  check if neighbor voxel  $l$  is pore
9:        $r \leftarrow l + k$   $\triangleright$  row index in  $\mathbf{A}_e$ 
10:      for each  $(i, j) \in (\mathcal{N}(e), \{0, 1, \dots, 7\})$  do  $\triangleright$  8 nodes per element
11:        if  $i < (\# \text{ pore nodes})$  then  $\triangleright$  check if neighbor  $i$  has velocity DOFs
12:           $c \leftarrow 3 * j$   $\triangleright$  column index in  $\mathbf{A}_e$ 
13:           $m \leftarrow 3 * i$ 
14:           $\mathbf{q}_d \leftarrow \mathbf{q}_d + (\mathbf{A}_{e[r,c]} * \mathbf{d}_m$   $\triangleright$  Read  $\mathbf{A}_e$  from const. memory
15:             $+ \mathbf{A}_{e[r,c+1]} * \mathbf{d}_{m+1}$ 
16:             $+ \mathbf{A}_{e[r,c+2]} * \mathbf{d}_{m+2})$ 
17:        end if
18:       $c \leftarrow 3 * (\# \text{ pore nodes}) + j$   $\triangleright$  column index in  $\mathbf{A}_e$ 
19:       $m \leftarrow 3 * (\# \text{ pore nodes}) + i$ 
20:       $\mathbf{q}_d \leftarrow \mathbf{q}_d + \mathbf{A}_{e[r,c]} * \mathbf{d}_m$ 
21:    end for
22:  end if

```

**Algorithm 14** SBS jacobi kernel for pore nodes – 3D**Input:**  $\mathbf{r}$ , ( $\#$  pore nodes)**Output:**  $\mathbf{q}$ 

```

1:  $n \leftarrow \text{threadId}$ 
2: if  $n < (\# \text{ pore nodes})$  then
3:   for each  $(d, k) \in (\{3n, 3n + 1, 3n + 2\}, \{0, 1, 2\})$  do  $\triangleright$  velocity DOFs
4:      $\mathbf{L} \leftarrow \mathcal{S}(d)$   $\triangleright$  hard-coded stencil, same for the DOFs of any  $n$ 
5:      $p \leftarrow \mathbf{L}_{[3 \times 13 + k]}$   $\triangleright$  center of the stencil is at node 13 (in 3D)
6:      $\mathbf{q}_d \leftarrow \mathbf{r}_d / p$ 
7:   end for
8:    $d \leftarrow 3(\# \text{ pore nodes}) + n$   $\triangleright$  pressure DOF
9:    $\mathbf{L} \leftarrow \mathcal{S}(d)$ 
10:   $p \leftarrow \mathbf{L}_{[3 \times 27 + 13]}$ 
11:   $\mathbf{q}_d \leftarrow \mathbf{r}_d / p$ 
12: end if

```

---

**Algorithm 15** NBN `jacobi` kernel for border nodes – 3D

---

**Input:**  $\mathbf{r}$ ,  $\mathbf{bkeys}$ , ( $\#$  pore nodes), ( $\#$  border nodes)**Output:**  $\mathbf{q}$ 

```

1:  $n \leftarrow \text{threadId}$ 
2: if  $n < (\# \text{ border nodes})$  then
3:    $\text{borderkey} \leftarrow \mathbf{bkeys}_{[n]}$ 
4:    $n \leftarrow n + (\# \text{ pore nodes})$   $\triangleright$  offset threadId index to match numbering scheme
5:    $(d, k) \leftarrow (3(\# \text{ pore nodes}) + n, 24)$   $\triangleright$  pressure DOF
6:    $p \leftarrow 0$ 
7:   for each  $i \in \{0, 1, \dots, 7\}$  do  $\triangleright$  8 neighbor voxels
8:     if  $(\text{borderkey} \gg i) \ \& \ 1$  then  $\triangleright$  check if neighbor voxel  $i$  is pore
9:        $p \leftarrow p + \mathbf{A}_{e[k+i, k+i]}$ 
10:    end if
11:  end for
12:   $\mathbf{q}_d \leftarrow \mathbf{r}_d / p$ 
13: end if

```

---

Finally, having defined how each of the considered matrix-free operations was implemented, the host functions that act as wrappers for matrix-vector products (`mtxfreeprod`) and preconditioner-vector products (`jacobi`) can be concisely written as in Algorithms 16 and 17, respectively. These are called multiple times during the iterative process of the CG and MINRES solvers (and their preconditioned versions).

---

**Algorithm 16** `mtxfreeprod`

---

**Input:**  $\mathbf{d}$ ,  $\mathbf{I}_2\mathbf{M}$ ,  $\mathbf{M}_2\mathbf{I}$ ,  $\mathbf{bkeys}$ , ( $\#$  pore nodes), ( $\#$  border nodes)**Output:**  $\mathbf{q}$ 

```

1:  $\mathbf{q} \leftarrow \text{SBS } \text{mtxfreeprod} \text{ pore nodes } (\mathbf{d}, \mathbf{I}_2\mathbf{M}, \mathbf{M}_2\mathbf{I}, (\# \text{ pore nodes}))$   $\triangleright$  Alg. 12
2:  $\mathbf{q} \leftarrow \mathbf{q}$ 
   + NBN mtxfreeprod border nodes  $(\mathbf{d}, \mathbf{I}_2\mathbf{M}, \mathbf{M}_2\mathbf{I}, \dots$   $\triangleright$  Alg. 13
    $\mathbf{bkeys}, (\# \text{ pore nodes}), (\# \text{ border nodes}))$ 

```

---



---

**Algorithm 17** `jacobi`

---

**Input:**  $\mathbf{r}$ ,  $\mathbf{bkeys}$ , ( $\#$  pore nodes), ( $\#$  border nodes)**Output:**  $\mathbf{q}$ 

```

1:  $\mathbf{q} \leftarrow \text{SBS } \text{jacobi} \text{ pore nodes } (\mathbf{r}, (\# \text{ pore nodes}))$   $\triangleright$  Alg. 14
2:  $\mathbf{q} \leftarrow \mathbf{q}$ 
   + NBN jacobi border nodes  $(\mathbf{r}, \dots$   $\triangleright$  Alg. 15
    $\mathbf{bkeys}, (\# \text{ pore nodes}), (\# \text{ border nodes}))$ 

```

---

## 5.4 CG and MINRES for Stokes flow in CUDA

The mathematical description of the CG and MINRES methods shown in Algorithms 3 through 6 can be directly ported to massively parallel frameworks with little to no modifications. A possible and welcome change that stands out for the PCG method is to allocate only a single array for the  $\mathbf{q}$  and  $\mathbf{s}$  vectors, since they are only employed to store results of matrix operations and are required at different stages within each iteration. Evidently, given the matrix-free approach, all of the matrix operations require the application-specific kernels detailed in the last Section. Algorithms 18 and 19 depict the implementation in CUDA C++ of the PCG and PMINRES methods, that is, considering Jacobi preconditioning. Similar versions of these without preconditioning were implemented as well.

---

**Algorithm 18** PCG – CUDA C++
 

---

**Input:**  $\mathbf{x}_0, \mathbf{b}, \epsilon, \text{maps} \equiv [\text{I}_2\text{M}, \text{M}_2\text{I}, \text{bkeys}]$ 
**Output:**  $\mathbf{x}$ 

```

1:  $\mathbf{x} \leftarrow \text{memcpy}(\mathbf{x}_0), \mathbf{r} \leftarrow \text{memcpy}(\mathbf{b})$ 
2:  $\mathbf{d} \leftarrow \text{zeros}()$ 
3:  $\Delta \leftarrow \text{dotjacobi}(\mathbf{r}, \text{bkeys})$ 
4: if  $\Delta \approx 0$  then  $\triangleright$  check if  $\mathbf{x} = \mathbf{0}$  is sol.
5:    $\mathbf{x} \leftarrow \text{zeros}()$ 
6:   return
7: end if
8:  $\mathbf{q} \leftarrow \text{mtxfreeprod}(\mathbf{x}, \text{maps})$ 
9:  $\mathbf{r} \leftarrow \text{daxpy}(-1, \mathbf{q}, \mathbf{r})$ 
10:  $\mathbf{q} \leftarrow \text{jacobi}(\mathbf{r}, \text{bkeys})$ 

11:  $\delta^{(1)} \leftarrow \text{dotprod}(\mathbf{r}, \mathbf{q})$ 
12:  $\beta \leftarrow 0$ 
13: while  $|\delta^{(k-1)} / \Delta| > \epsilon^2$  do  $\triangleright k \in [2, \dots)$ 
14:    $\mathbf{d} \leftarrow \text{daxpy}(\beta, \mathbf{d}, \mathbf{q})$ 
15:    $\mathbf{q} \leftarrow \text{mtxfreeprod}(\mathbf{d}, \text{maps})$ 
16:    $\alpha \leftarrow \delta^{(k-1)} / \text{dotprod}(\mathbf{q}, \mathbf{d})$ 
17:    $\mathbf{x} \leftarrow \text{daxpy}(\alpha, \mathbf{d}, \mathbf{x})$ 
18:    $\mathbf{r} \leftarrow \text{daxpy}(-\alpha, \mathbf{q}, \mathbf{r})$ 
19:    $\mathbf{q} \leftarrow \text{jacobi}(\mathbf{r}, \text{bkeys})$ 

20:    $\delta^{(k)} \leftarrow \text{dotprod}(\mathbf{r}, \mathbf{q})$ 

21:    $\beta \leftarrow \delta^{(k)} / \delta^{(k-1)}$ 
22: end while
```

---



---

**Algorithm 19** PMINRES – CUDA C++
 

---

**Input:**  $\mathbf{x}_0, \mathbf{b}, \epsilon, \text{maps} \equiv [\text{I}_2\text{M}, \text{M}_2\text{I}, \text{bkeys}]$ 
**Output:**  $\mathbf{x}$ 

```

1:  $\mathbf{x} \leftarrow \text{memcpy}(\mathbf{x}_0), \mathbf{r} \leftarrow \text{memcpy}(\mathbf{b})$ 
2:  $\mathbf{d} \leftarrow \text{zeros}()$ 
3:  $\Delta \leftarrow \text{dotjacobi}(\mathbf{r}, \text{bkeys})$ 
4: if  $\Delta \approx 0$  then
5:    $\mathbf{x} \leftarrow \text{zeros}()$ 
6:   return
7: end if
8:  $\mathbf{q} \leftarrow \text{mtxfreeprod}(\mathbf{x}, \text{maps})$ 
9:  $\mathbf{r} \leftarrow \text{daxpy}(-1, \mathbf{q}, \mathbf{r})$ 
10:  $\mathbf{q} \leftarrow \text{jacobi}(\mathbf{r}, \text{bkeys})$ 
11:  $\mathbf{s} \leftarrow \text{mtxfreeprod}(\mathbf{q}, \text{maps})$ 
12:  $\delta^{(1)} \leftarrow \text{dotprod}(\mathbf{q}, \mathbf{s})$ 
13:  $\beta \leftarrow 0$ 
14: while  $|\text{dotjacobi}(\mathbf{r}, \text{bkeys}) / \Delta| > \epsilon^2$  do
15:    $\mathbf{d} \leftarrow \text{jacobi}(\text{axpy}(1, \mathbf{r}, \beta, \mathbf{d}), \text{bkeys})$ 
16:    $\mathbf{q} \leftarrow \text{daxpy}(\beta, \mathbf{q}, \mathbf{s})$ 
17:    $\alpha \leftarrow \delta^{(k-1)} / \text{dotjacobi}(\mathbf{q}, \text{bkeys})$ 
18:    $\mathbf{x} \leftarrow \text{daxpy}(\alpha, \mathbf{d}, \mathbf{x})$ 
19:    $\mathbf{r} \leftarrow \text{daxpy}(-\alpha, \mathbf{q}, \mathbf{r})$ 
20:    $\mathbf{r} \leftarrow \text{jacobi}(\mathbf{r}, \text{bkeys})$ 
21:    $\mathbf{s} \leftarrow \text{mtxfreeprod}(\mathbf{r}, \text{maps})$ 
22:    $\delta^{(k)} \leftarrow \text{dotprod}(\mathbf{r}, \mathbf{s})$ 
23:    $\mathbf{r} \leftarrow \text{invjacobi}(\mathbf{r}, \text{bkeys})$ 
24:    $\beta \leftarrow \delta^{(k)} / \delta^{(k-1)}$ 
25: end while
```

---

In the Algorithms above, all of the operations written as computational functions are calls to CUDA kernels, apart from `memcpy`. It is stressed that `zeros`, `daxpy` and `dotprod` are fairly standard parallelizable vector operations, as in

$$\begin{aligned}\mathbf{d} &\leftarrow \text{zeros}() \equiv \mathbf{d} \leftarrow \mathbf{0}, \\ \mathbf{d} &\leftarrow \text{daxpy}(a, \mathbf{x}, \mathbf{y}) \equiv \mathbf{d} \leftarrow a\mathbf{x} + \mathbf{y}, \\ p &\leftarrow \text{dotprod}(\mathbf{x}, \mathbf{y}) \equiv p \leftarrow \mathbf{x}^T \mathbf{y},\end{aligned}$$

where `zeros` and `daxpy` are naturally SIMT procedures, while `dotprod` requires a parallel reduction using shared memory caches [69, 122]. Further, notice that three variations of the `jacobi` operation were introduced: `dotjacobi`, in line 4 of both solvers; `jacobiexpby` and `invjacobi`, in lines 15 and 23 of PMINRES, respectively. These are kernels that make use of the device functions that compute the preconditioner-vector product within larger algebraic expressions, without having to resort to temporary memory allocation to store “intermediate” result vectors. The idea is to hold the results of `jacobi` in either local variables or in shared memory and then use them to compute the full required operation locally before writing to global memory. They are defined as follows:

$$\begin{aligned}\text{dotjacobi}(\mathbf{x}) &= \mathbf{x}^T \mathbf{M}^{-1} \mathbf{x}, \\ \text{jacobiexpby}(\alpha, \mathbf{x}, \beta, \mathbf{y}) &= \alpha \mathbf{M}^{-1} \mathbf{x} + \beta \mathbf{y}, \\ \text{invjacobi}(\mathbf{x}) &= \mathbf{M} \mathbf{x},\end{aligned}$$

where, as for `dotprod`, `dotjacobi` involves a parallel reduction of data cached in shared memory. In the expressions above, the list of border keys is omitted from the input parameters, for conciseness. It is important to remark that `dotprod` and `dotjacobi` are the only synchronizing calls inside the `while` loop, i.e. they make the host wait for their result to be copied from the device at every iteration.

Notice that the matrix-vector products were underlined. They are the most expensive algebraic operations required by the solvers, especially in a matrix-free scope. This was done to highlight the fact that both (P)CG and (P)MINRES, require only one matrix-vector product per iteration. (P)MINRES does require an extra one at the initialization of its residual metrics, but it tends to not make much of a difference when dealing with large-scale systems (lots of iterations). This notion of working strictly with one matrix-vector product per iteration is rather usual in more conventional scenarios, where the global matrix is stored in sparse format, for example. Generally, the idea is that if the huge memory cost associated with the matrix is “paid”, there is no need to worry about getting rid of a few vectors in the solver, as they are not so significant in that case.

However, the matrix-free approach raises a doubt: *is it necessary to allocate all of these vectors?* After all, the major trade-off in not storing the matrix has already been accepted and dealt with. Further, in the present context, allocating all of the vectors is the main memory bottleneck of these solvers. With that in mind, alternative implementations that extend the trade-off by not allocating certain vectors in exchange for extra matrix-vector products per iteration are proposed. These are detailed in the next Section.

## 5.5 3 and 2 vector alternatives

From Algorithms 18 and 19, it is noticeable that the PCG solver requires 4 vectors to be allocated ( $\mathbf{x}, \mathbf{r}, \mathbf{d}, \mathbf{q}$ ), while PMINRES demands 5 vectors (those 4 plus  $\mathbf{s}$ ). It is important to solidify the notion that this is a decision rather than a strict requirement. As made evident in the title of this Section, here, implementations with fewer vectors stored in memory are considered, namely with 3 and 2 vectors. The goal is to expand the limits of simulation size with limited memory availability.

### 5.5.1 PCG3 and PMINRES3

Aiming firstly at a 3 vector solution, the idea is to store in memory just what is essential for the iterative scheme of Krylov subspace solvers: the current solution, the current residual, and a search direction to update both in the next iteration. This means keeping vectors  $\mathbf{x}$ ,  $\mathbf{r}$  and  $\mathbf{d}$ , and dropping vectors  $\mathbf{q}$  and  $\mathbf{s}$ .

For the PCG method, this process is conceptually straightforward. The auxiliary vector  $\mathbf{s}$  is already not used, and  $\mathbf{q}$  is employed to store the results of a matrix-vector product and a preconditioner-vector product at each iteration, which are then used in `dotprod` and `daxpy` operations. To not allocate  $\mathbf{q}$ , the matrix-free operation that yields its values has to be embedded in these other vector operations, like so:

$$\left. \begin{array}{l} \mathbf{q} \leftarrow \text{mtxfreeprod}(\mathbf{d}) \\ p \leftarrow \text{dotprod}(\mathbf{q}, \mathbf{d}) \\ \mathbf{r} \leftarrow \text{daxpy}(-\alpha, \mathbf{q}, \mathbf{r}) \end{array} \right\} \begin{array}{l} p \leftarrow \text{dotmtxfree}(\mathbf{d}) = \mathbf{d}^T \mathbf{A} \mathbf{d} \\ \mathbf{r} \leftarrow \text{mtxfreeaxpy}(-\alpha, \mathbf{d}, \mathbf{r}) = -\alpha \mathbf{A} \mathbf{d} + \mathbf{r}, \end{array}$$

$$\left. \begin{array}{l} \mathbf{q} \leftarrow \text{jacobi}(\mathbf{r}) \\ \delta \leftarrow \text{dotprod}(\mathbf{q}, \mathbf{d}) \end{array} \right\} \delta \leftarrow \text{dotjacobi}(\mathbf{r}) = \mathbf{r}^T \mathbf{M}^{-1} \mathbf{r},$$

where all of the embedded matrix-free operations are implemented as kernels that call

the `mtxfreeprod` and `jacobi` device functions. Consequently, each PCG iteration now requires two matrix-vector products: one in `dotmtxfree`, another in `mtxfreeaxpy`.

Analogously, to construct the 3 vector PMINRES, the idea is to insert the expression that evaluates  $\mathbf{q}$  and  $\mathbf{s}$  wherever they are required. That is,  $\mathbf{q}^{(k)} = \mathbf{A}\mathbf{d}^{(k)}$  and  $\mathbf{s}^{(k)} = \mathbf{A}\mathbf{r}^{(k)}$ . Similarly to the PCG method, this version of PMINRES uses the introduced `dotmtxfree` and `mtxfreeaxpy` kernels, plus a third one, which is defined as:

$$\left. \begin{array}{l} \mathbf{q} \leftarrow \text{daxpy}(\beta, \mathbf{q}, \mathbf{s}) \equiv \text{mtxfreeprod}(\mathbf{d}) \\ p \leftarrow \text{dotjacobi}(\mathbf{q}) \end{array} \right\} p \leftarrow \text{dotjacobimtx2}(\mathbf{d}) = (\mathbf{A}\mathbf{d})^T \mathbf{M}^{-1} \mathbf{A}\mathbf{d},$$

where, the product  $\mathbf{A}\mathbf{d}$  in `dotjacobimtx2` can be computed just once. In that sense, the 3 vector PMINRES requires 3 matrix-vector multiplications.

Pseudo-code detailing the kernel calls for the 3 vector PCG and PMINRES implementations, named PCG3 and PMINRES3, is presented in Algorithms 20 and 21. Once more, the matrix-vector products are underlined, for quick reference. Now both solvers allocate the same amount of memory, but PMINRES3 requires more expensive iterations.

---

**Algorithm 20** PCG3 – CUDA C++

**Input:**  $\mathbf{x}_0, \mathbf{b}, \epsilon, \text{maps} \equiv [\text{I}_2\text{M}, \text{M}_2\text{I}, \text{bkeys}]$   
**Output:**  $\mathbf{x}$

- 1:  $\mathbf{x} \leftarrow \text{memcpy}(\mathbf{x}_0), \mathbf{r} \leftarrow \text{memcpy}(\mathbf{b})$
- 2:  $\mathbf{d} \leftarrow \text{zeros}()$
- 3:  $\Delta \leftarrow \text{dotjacobi}(\mathbf{r}, \text{bkeys})$
- 4: **if**  $\Delta \approx 0$  **then**  $\triangleright$  check if  $\mathbf{x} = \mathbf{0}$  is sol.
- 5:    $\mathbf{x} \leftarrow \text{zeros}()$
- 6:   **return**
- 7: **end if**
- 8:  $\mathbf{r} \leftarrow \underline{\text{mtxfreeaxpy}}(-1, \mathbf{x}, \mathbf{r}, \text{maps})$
- 9:  $\delta^{(1)} \leftarrow \text{dotjacobi}(\mathbf{r}, \text{bkeys})$
- 10:  $\beta \leftarrow 0$
- 11: **while**  $|\delta^{(k-1)} / \Delta| > \epsilon^2$  **do**  $\triangleright k \in [2, \dots]$
- 12:    $\mathbf{d} \leftarrow \text{jacobiaxpy}(1, \mathbf{r}, \beta, \mathbf{d}, \text{bkeys})$
- 13:    $\alpha \leftarrow \delta^{(k-1)} / \underline{\text{dotmtxfree}}(\mathbf{d}, \text{maps})$
- 14:    $\mathbf{x} \leftarrow \text{daxpy}(\alpha, \mathbf{d}, \mathbf{x})$
- 15:    $\mathbf{r} \leftarrow \underline{\text{mtxfreeaxpy}}(-\alpha, \mathbf{d}, \mathbf{r}, \text{maps})$
- 16:    $\delta^{(k)} \leftarrow \text{dotjacobi}(\mathbf{r}, \text{bkeys})$
- 17:    $\beta \leftarrow \delta^{(k)} / \delta^{(k-1)}$
- 18: **end while**

---



---

**Algorithm 21** PMINRES3 – CUDA C++

**Input:**  $\mathbf{x}_0, \mathbf{b}, \epsilon, \text{maps} \equiv [\text{I}_2\text{M}, \text{M}_2\text{I}, \text{bkeys}]$   
**Output:**  $\mathbf{x}$

- 1:  $\mathbf{x} \leftarrow \text{memcpy}(\mathbf{x}_0), \mathbf{r} \leftarrow \text{memcpy}(\mathbf{b})$
- 2:  $\mathbf{d} \leftarrow \text{zeros}()$
- 3:  $\Delta \leftarrow \text{dotjacobi}(\mathbf{r}, \text{bkeys})$
- 4: **if**  $\Delta \approx 0$  **then**
- 5:    $\mathbf{x} \leftarrow \text{zeros}()$
- 6:   **return**
- 7: **end if**
- 8:  $\mathbf{r} \leftarrow \underline{\text{mtxfreeaxpy}}(-1, \mathbf{x}, \mathbf{r}, \text{maps})$
- 9:  $\mathbf{r} \leftarrow \text{jacobi}(\mathbf{r}, \text{bkeys})$
- 10:  $\delta^{(1)} \leftarrow \underline{\text{dotmtxfree}}(\mathbf{r}, \text{maps})$
- 11:  $\mathbf{r} \leftarrow \text{invjacobi}(\mathbf{r}, \text{bkeys})$
- 12:  $\beta \leftarrow 0$
- 13: **while**  $|\text{dotjacobi}(\mathbf{r}, \text{bkeys}) / \Delta| > \epsilon^2$  **do**
- 14:    $\mathbf{d} \leftarrow \text{jacobiaxpy}(1, \mathbf{r}, \beta, \mathbf{d}, \text{bkeys})$
- 15:    $\alpha \leftarrow \delta^{(k-1)} / \underline{\text{dotjacobimtx2}}(\mathbf{d}, \text{maps})$
- 16:    $\mathbf{x} \leftarrow \text{daxpy}(\alpha, \mathbf{d}, \mathbf{x})$
- 17:    $\mathbf{r} \leftarrow \underline{\text{mtxfreeaxpy}}(-\alpha, \mathbf{d}, \mathbf{r}, \text{maps})$
- 18:    $\mathbf{r} \leftarrow \text{jacobi}(\mathbf{r}, \text{bkeys})$
- 19:    $\delta^{(k)} \leftarrow \underline{\text{dotmtxfree}}(\mathbf{r}, \text{maps})$
- 20:    $\mathbf{r} \leftarrow \text{invjacobi}(\mathbf{r}, \text{bkeys})$
- 21:    $\beta \leftarrow \delta^{(k)} / \delta^{(k-1)}$
- 22: **end while**

---

### 5.5.2 PCG2 and PMINRES2

PCG3 and PMINRES3 are already very efficient solvers in terms of memory usage, in comparison to the roster of simulation software often seen in the literature for permeability estimation. This claim will be made evident in the next Chapter, where numerical experiments are presented and discussed. But, still, it is possible to do better.

Up to this point, the relative low memory demand of the proposed solvers was built on the insertion of specific considerations about the problem at hand into the implementation. The voxel-based nature of the mesh and the Q1Q1 elements were taken into account in the matrix-free operations, and the expressions for the evaluation of the auxiliary vectors  $\mathbf{q}$  and  $\mathbf{s}$  were substituted in the operations where they are required. In doing so, we have avoided allocating the global matrix and two vectors. But a question remains: *what other knowledge can be embedded in the implementation?* An answer: *The final product of the simulations.* Recall that, here, the main interest is in the homogenized permeability tensor of the studied medium, i.e. the effective velocity fields. To that end, the distribution of the velocity and pressure fields throughout the microscale domain is not actually required, we just need volumetric averages of velocity. In other words, if there is no need for any post processing or visualization of the velocity fields, **we do not need the solution vector to compute the homogenized permeability.**

The 2 vector versions of the PCG and PMINRES methods, named PCG2 and PMINRES2, are built upon PCG3 and PMINRES3. They are specific solutions to the problem of permeability homomgenization. The idea is to not allocate  $\mathbf{x}$  in the device, and keep track of the effective velocity fields  $\langle \mathbf{u} \rangle$  instead. Notice that these are just 3 double precision variables to be stored, not an  $n$ -sized array (with  $n$  as the number of DOFs). This can be done by `memcpy`'ing the initial guess vector  $\mathbf{x}_0$  directly into  $\mathbf{d}$ , and then updating  $\langle \mathbf{u} \rangle$  at each iteration via a reduction of the step  $\alpha \mathbf{d}$ , like such:

$$\begin{aligned} \mathbf{x}^{(k)} &= \mathbf{x}^{(k-1)} + (\alpha \mathbf{d})^{(k)} = \mathbf{x}^{(k-2)} + (\alpha \mathbf{d})^{(k-1)} + (\alpha \mathbf{d})^{(k)} = \mathbf{x}_0 + \dots + (\alpha \mathbf{d})^{(k-2)} + (\alpha \mathbf{d})^{(k-1)} + (\alpha \mathbf{d})^{(k)}, \\ \therefore \langle \mathbf{x}^{(k)} \rangle &= \langle \mathbf{x}_0 \rangle + \dots + \langle (\alpha \mathbf{d})^{(k-2)} \rangle + \langle (\alpha \mathbf{d})^{(k-1)} \rangle + \langle (\alpha \mathbf{d})^{(k)} \rangle, \end{aligned}$$

with  $k$  as the last iteration, when convergence was achieved, such that  $\mathbf{x}^{(k)} \approx \mathbf{x}_{\text{sol.}}$ . Further, keep in mind that  $\langle \mathbf{u} \rangle$  is a reduction of just the velocity DOFs, i.e.  $\langle \mathbf{u} \rangle = \langle \mathbf{x}_{[\text{velocityDOFs}]}^{(k)} \rangle$ . In that sense,

$$\langle \mathbf{u} \rangle = \langle \mathbf{x}_{0[\text{velocityDOFs}]} \rangle + \dots + \langle (\alpha \mathbf{d}_{[\text{velocityDOFs}]})^{(k-1)} \rangle + \langle (\alpha \mathbf{d}_{[\text{velocityDOFs}]})^{(k)} \rangle.$$

Notice that the operation  $\langle(\alpha \mathbf{d}_{[\text{velocityDOFs}]})\rangle$  is not a single reduction, as it is necessary to separate the velocity DOFs in each direction. In that sense, an operation `reducevelocity` can be defined as:

$$\text{reducevelocity}(\mathbf{d}) \equiv \begin{cases} \langle \mathbf{d} \rangle_1 \leftarrow \sum_{i=1}^{(\# \text{ pore nodes})} \mathbf{d}_{[3 \times i - 2]}, \\ \langle \mathbf{d} \rangle_2 \leftarrow \sum_{i=1}^{(\# \text{ pore nodes})} \mathbf{d}_{[3 \times i - 1]}, \\ \langle \mathbf{d} \rangle_3 \leftarrow \sum_{i=1}^{(\# \text{ pore nodes})} \mathbf{d}_{[3 \times i]}. \end{cases}$$

It is important to state, however, that there is a caveat to this strategy. The reduction of  $\alpha \mathbf{d}$  is not always stable for large-scale systems, due to floating point rounding errors, especially when dealing with a highly oscillating solver, as is the PCG method for these saddle point matrices. Notice that, in the proposed scheme, if a single  $\langle \alpha \mathbf{d} \rangle$  gets computed with significant error, it can be enough to “detour” the effective velocities to wrong values, as the final result is the sum of all reductions. To mitigate this sort of issue, in code, `reducevelocity` sums positive and negative values separately, and a scale is applied to  $\mathbf{d}$  corresponding to  $(1/f \|\mathbf{d}\|_\infty)$ , where  $f$  is a normalization factor. Empirically, it was observed that for  $f \in [1e - 12, 1e - 15]$ , the sequence of reductions converges to well approximated values (within a numerical tolerance) for all of the conducted tests. That is,

$$\text{reducevelocity}(\mathbf{x}^{(k)}) \approx \sum f \|\mathbf{d}\|_\infty \times \text{reducevelocity}((\alpha/f \|\mathbf{d}\|_\infty) \mathbf{d}).$$

Further testing and formalization of this “stabilization” technique is left as future work.

Alternatively, to avoid having to deal with added numerical instability due the successive reductions of  $\alpha \mathbf{d}$ , notice that, if just the main diagonal terms in the effective permeability tensor are of interest, which is usual, it is possible to store just the components of velocity in the direction of analysis instead of either dealing with these many reductions or having to allocate a full vector. In comparison to the full  $\mathbf{x}$ , this strategy would require a vector of roughly 20% of the size, meaning that a solution with  $\sim 2.2$  vectors is also possible.

Finally, with the definition of `reducevelocity`, it is possible to write PCG2 and PMINRES2 as in Algorithms 22 and 23. It is remarkable that the amount of matrix-vector products has not changed from PCG3 and PMINRES3.

| Algorithm 22 PCG2 – CUDA C++                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | Algorithm 23 PMINRES2 – CUDA C++                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Input:</b> $\mathbf{x}_0, \mathbf{b}, \epsilon, \text{maps} \equiv [\text{I}_2\text{M}, \text{M}_2\text{I}, \text{bkeys}]$<br><b>Output:</b> $\langle \mathbf{u} \rangle$ $\triangleright$ homogenized velocity<br>1: $\mathbf{d} \leftarrow \text{memcpy}(\mathbf{x}_0), \mathbf{r} \leftarrow \text{memcpy}(\mathbf{b})$<br>2: $\Delta \leftarrow \text{dotjacobi}(\mathbf{r}, \text{bkeys})$<br>3: <b>if</b> $\Delta \approx 0$ <b>then</b> $\triangleright$ check if $\mathbf{x} = \mathbf{0}$ is sol.<br>4: $\langle \mathbf{u} \rangle \leftarrow 0$<br>5: <b>return</b><br>6: <b>end if</b><br>7: $\langle \mathbf{u} \rangle \leftarrow \text{reducevelocity}(\mathbf{d})$<br>8: $\mathbf{r} \leftarrow \text{mtxfreeaxpy}(-1, \mathbf{d}, \mathbf{r}, \text{maps})$<br>9: $\delta^{(1)} \leftarrow \text{dotjacobi}(\mathbf{r}, \text{bkeys})$<br>10: $\beta \leftarrow 0$<br>11: <b>while</b> $ \delta^{(k-1)} / \Delta  > \epsilon^2$ <b>do</b> $\triangleright k \in [2, \dots)$<br>12: $\mathbf{d} \leftarrow \text{jacobiaxpy}(1, \mathbf{r}, \beta, \mathbf{d}, \text{bkeys})$<br>13: $\alpha \leftarrow \delta^{(k-1)} / \text{dotmtxfree}(\mathbf{d}, \text{maps})$<br>14: $\langle \mathbf{u} \rangle \leftarrow \langle \mathbf{u} \rangle \alpha * \text{reducevelocity}(\mathbf{d})$<br>15: $\mathbf{r} \leftarrow \text{mtxfreeaxpy}(-\alpha, \mathbf{d}, \mathbf{r}, \text{maps})$<br>16: $\delta^{(k)} \leftarrow \text{dotjacobi}(\mathbf{r}, \text{bkeys})$<br>17: $\beta \leftarrow \delta^{(k)} / \delta^{(k-1)}$<br>18: <b>end while</b> | <b>Input:</b> $\mathbf{x}_0, \mathbf{b}, \epsilon, \text{maps} \equiv [\text{I}_2\text{M}, \text{M}_2\text{I}, \text{bkeys}]$<br><b>Output:</b> $\langle \mathbf{u} \rangle$<br>1: $\mathbf{d} \leftarrow \text{memcpy}(\mathbf{x}_0), \mathbf{r} \leftarrow \text{memcpy}(\mathbf{b})$<br>2: $\Delta \leftarrow \text{dotjacobi}(\mathbf{r}, \text{bkeys})$<br>3: <b>if</b> $\Delta \approx 0$ <b>then</b><br>4: $\langle \mathbf{u} \rangle \leftarrow 0$<br>5: <b>return</b><br>6: <b>end if</b><br>7: $\langle \mathbf{u} \rangle \leftarrow \text{reducevelocity}(\mathbf{d})$<br>8: $\mathbf{r} \leftarrow \text{mtxfreeaxpy}(-1, \mathbf{d}, \mathbf{r}, \text{maps})$<br>9: $\mathbf{r} \leftarrow \text{jacobi}(\mathbf{r}, \text{bkeys})$<br>10: $\delta^{(1)} \leftarrow \text{dotmtxfree}(\mathbf{r}, \text{maps})$<br>11: $\mathbf{r} \leftarrow \text{invjacobi}(\mathbf{r}, \text{bkeys})$<br>12: $\beta \leftarrow 0$<br>13: <b>while</b> $ \text{dotjacobi}(\mathbf{r}, \text{bkeys}) / \Delta  > \epsilon^2$ <b>do</b><br>14: $\mathbf{d} \leftarrow \text{jacobiaxpy}(1, \mathbf{r}, \beta, \mathbf{d}, \text{bkeys})$<br>15: $\alpha \leftarrow \delta^{(k-1)} / \text{dotjacobimtx2}(\mathbf{d}, \text{maps})$<br>16: $\langle \mathbf{u} \rangle \leftarrow \langle \mathbf{u} \rangle \alpha * \text{reducevelocity}(\mathbf{d})$<br>17: $\mathbf{r} \leftarrow \text{mtxfreeaxpy}(-\alpha, \mathbf{d}, \mathbf{r}, \text{maps})$<br>18: $\mathbf{r} \leftarrow \text{jacobi}(\mathbf{r}, \text{bkeys})$<br>19: $\delta^{(k)} \leftarrow \text{dotmtxfree}(\mathbf{r}, \text{maps})$<br>20: $\mathbf{r} \leftarrow \text{invjacobi}(\mathbf{r}, \text{bkeys})$<br>21: $\beta \leftarrow \delta^{(k)} / \delta^{(k-1)}$<br>22: <b>end while</b> |

## 5.6 Summary of the implemented solvers

In total, 12 different solvers were implemented, among CG and MINRES, preconditioning or not, and either one matrix-vector product per iteration or 3 or 2 vectors stored in memory. Evidently, all of these vary in computational cost and memory allocation. A summary of these characteristics for the implemented solvers is presented in Table 5.1.

From Table 5.1, it may be suggested that the CG and PCG methods appear to be more efficient, as they allocate either less or the same memory of their MINRES counterparts, and perform less algebraic operations per iteration. However, it is important to be mindful that MINRES is known for being a stable method of choice for systems characterized by indefinite matrices, which is the case. In fact, convergence is not even guaranteed with the CG and PCG methods. In that sense, MINRES and PMINRES would ideally be more trustworthy methods for this application. All of the implemented solvers are thoroughly tested and validated in the next Chapter.

| Solver   | # vectors | # mtxfreeprod | # daxpy | # reduce | # jacobi |
|----------|-----------|---------------|---------|----------|----------|
| CG       | 4         | 1             | 3       | 2        | 0        |
| CG3      | 3         | 2             | 3       | 2        | 0        |
| CG2      | 2         | 2             | 2       | 3        | 0        |
| PCG      | 4         | 1             | 3       | 2        | 1        |
| PCG3     | 3         | 2             | 3       | 2        | 2        |
| PCG2     | 2         | 2             | 2       | 3        | 2        |
| MINRES   | 5         | 1             | 4       | 3        | 0        |
| MINRES3  | 3         | 3             | 3       | 3        | 0        |
| MINRES2  | 2         | 3             | 2       | 4        | 0        |
| PMINRES  | 5         | 1             | 4       | 3        | 5        |
| PMINRES3 | 3         | 3             | 3       | 3        | 5        |
| PMINRES2 | 2         | 3             | 2       | 4        | 5        |

Table 5.1: Summary of the implemented solvers. From left to right, the columns after the solver identifier depict: the number of vectors of variables stored in memory, the amount of matrix-vector products, the amount of `daxpy` operations, the amount of `reduce`- or `dotprod`-like operations, and the amount of preconditioner-vector operations, all per iteration.

# Chapter 6

## Experiments

In this Chapter, some numerical experiments made with the developed solvers are reported. At first, verification analyses are presented, including results of permeability analyses for a two-dimensional periodic array of fiber-like cylinders, a three-dimensional periodic array of cylindrical pipes, and a three-dimensional square packed bed of spheres. These models have known solutions, be them analytical or semi-analytical, as seen in [32, 154], respectively. Then, a three-dimensional a sandstone sample and a fibrous material sample [89, 90] are studied, showcasing the capability of the developed program to handle  $\mu$ CT scans as input. Finally, a batch of digital samples of reservoir rocks is characterized, followed by reports of gigavoxel simulations ( $1000^3$  voxels images). The main objectives here are to verify the correctness of the implementation, compare the numerical results against reference values, and demonstrate the capacity of the implemented matrix-free approach to deal with large systems in personal machines.

|                 |                               |
|-----------------|-------------------------------|
| O.S.            | Ubuntu 20.04.2 LTS            |
| CPU             | AMD Ryzen 7 3700x             |
| Clock rate      | 3.60 GHz                      |
| RAM             | 64 GB                         |
| Cores (threads) | 8 (16)                        |
| GPU             | Nvidia GeForce RTX 2080 Super |
| Clock rate      | 1.85 GHz                      |
| RAM             | 8 GB                          |
| CUDA cores      | 3,072                         |
| Architecture    | Turing                        |

Table 6.1: Hardware specifications for the simulations reported in Sections 6.1 and 6.2.

For the simulations presented below, desktop hardware was employed. In the studies presented in Sections 6.1 and 6.2, the employed machine’s details are shown in Table 6.1.

Notice that the availability of RAM memory on this graphics card – 8 GB – can be a limiting factor for large-scale analyses. Another computer (and GPU) was used for the simulations reported in Section 6.3, as it will be indicated ahead.

## 6.1 Verification tests

### 6.1.1 Two-dimensional semi-analytical benchmark

Firstly, the model illustrated in Figure 6.1 was studied. The domain is formed by circular inclusions embedded in a predominant matrix, here taken as the pore space. Models like this are commonly used as validation benchmarks in numerical homogenization, since there are analytical or semi-analytical solutions for several effective physical properties of such microstructures in the literature, as in [111] for example. The work of Drummond and Tahir [32] presents an expression based on series expansions for the permeability of the studied medium, which can be written as a function of the radius  $r$  of the circular inclusions and the porosity  $\phi$  of the sample, as in

$$\begin{aligned} \kappa = K_{11} = K_{22} = \frac{r^2}{8\phi} \left( \ln(1/\phi) - 1.47633597 + 2\phi - 1.74428264\phi^2 \right. \\ \left. + 4.07770444\phi^3 - 4.84227402\phi^4 \right), \end{aligned} \quad (6.1)$$

where  $\kappa$  is the isotropic permeability of the domain. This expression is well-suited for high porosity models (70% or more). Here, it is considered a square periodic cell of dimension 1mm, with 0.25mm in diameter for the circular regions. With these parameters, the semi-analytical solution provides isotropic permeability of  $k = 2.0434\text{e}+04$  D for this model. Several two-dimensional images with different resolutions were considered to represent the domain, as depicted in Figure 6.1 and detailed in Table 6.2. Plots that depict the data in that Table are presented in Figure 6.3. The applied effective pressure gradients  $\overline{p}_{,i}$  are illustrated in Figure 6.2. The results from the numerical simulations are shown in Figure 6.4. A numerical tolerance of  $10^{-6}$  was adopted for the (non-preconditioned) CG and MINRES methods, and the starting point  $\mathbf{x}_0$  was taken as a null vector  $\mathbf{0}$ . Note that this tolerance is fairly small, and other values up to  $10^{-3}$  could have been employed for similar results. The idea here was to adopt a “strict” convergence criterion.

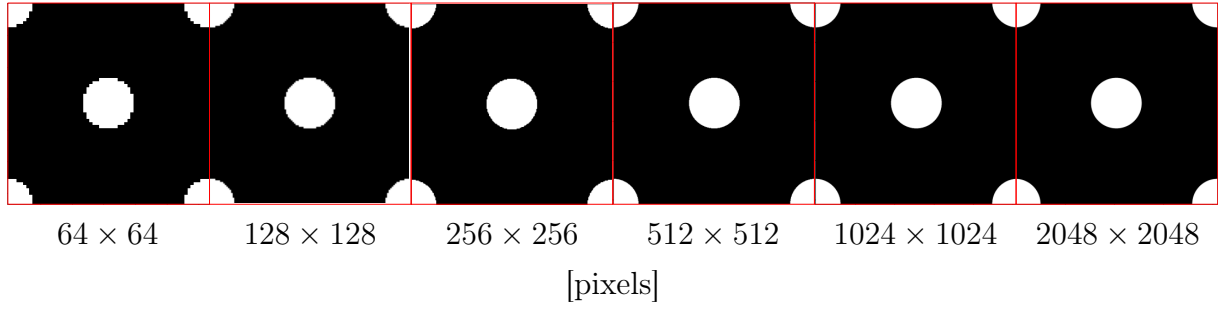


Figure 6.1: Two-dimensional medium for which there are analytical solutions, represented by raster images with increasing pixel count. Solid phase in white, pore space in black.

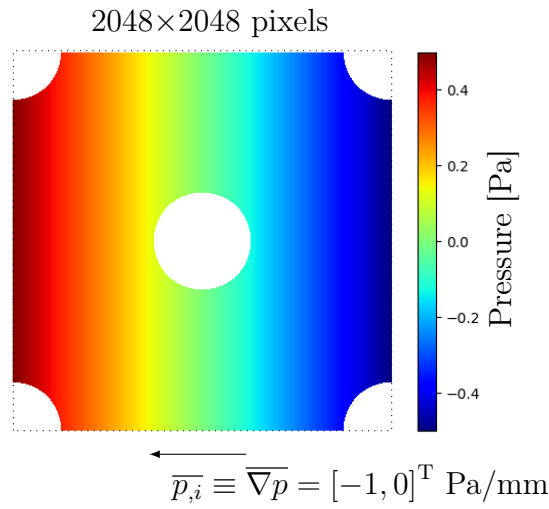


Figure 6.2: Imposed pressure field due to the effective periodic gradient  $\overline{p}_{,i}$  applied for analyses in direction  $x_1$  (horizontal) of the samples shown in Figure 6.1. Notice that fluid tends to flow against the direction of this pressure gradient, i.e. from higher to lower pressure.

| Dimensions [pixels]     | # DOFs     | Porosity [%] |
|-------------------------|------------|--------------|
| $64 \times 64$          | 10,970     | 89.84        |
| $128 \times 128$        | 44,146     | 90.09        |
| $256 \times 256$        | 176,978    | 90.15        |
| $512 \times 512$        | 708,562    | 90.16        |
| $1024 \times 1024$      | 2,835,890  | 90.18        |
| $2048 \times 2048$      | 11,345,506 | 90.18        |
| Drummond and Tahir [32] |            | 90.18        |

Table 6.2: Image-based models with different resolutions for the reproduction of the analytical benchmark (Figure 6.1).

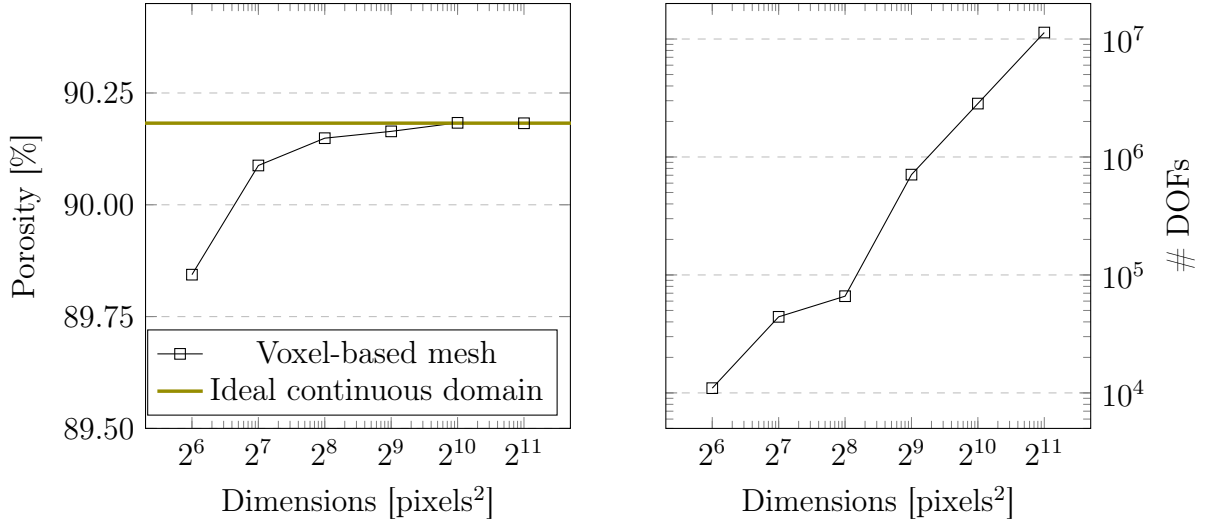


Figure 6.3: Porosity (left) and number of DOFs (right) for each of the considered samples. These are plots that illustrate the data shown in Table 6.2.

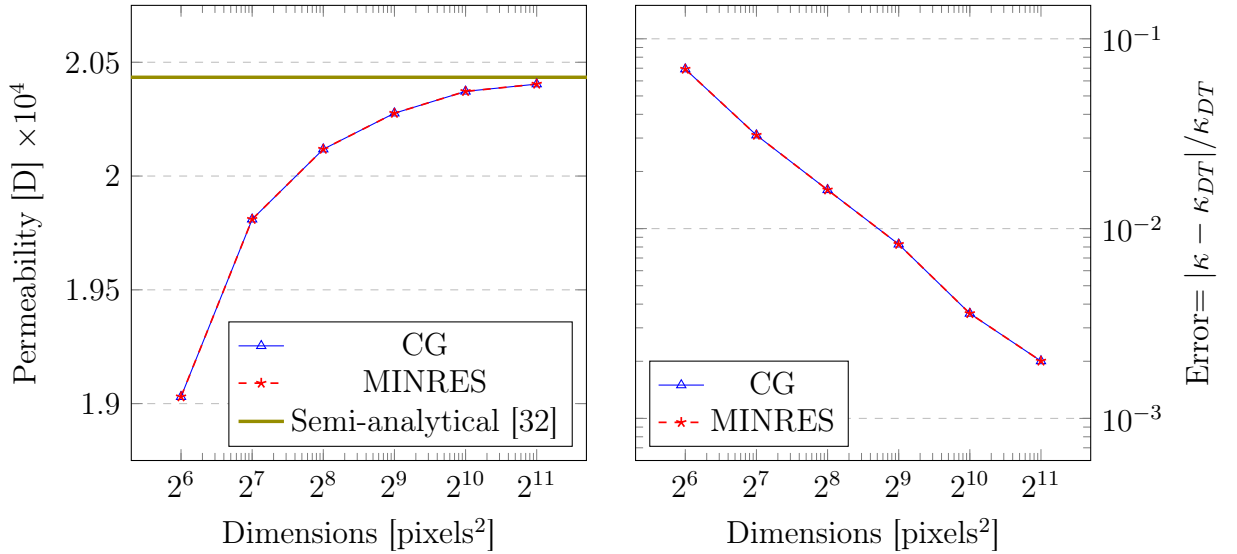


Figure 6.4: Convergence of the FEM-based permeability analysis for the model illustrated in Figure 6.1. To the left, the results for each image dimension. To the right, the numerical error on a logarithmic scale.  $k$  is the permeability obtained by the simulation,  $k_{DT} = 2.0434e+04$  D is the permeability given by the series expansion solution from Drummond and Tahir [32].

Figure 6.4, to the right, shows the convergence curve, on a logarithmic scale, of the error from the numerical solutions. The permeability values obtained in each simulation are displayed in the plot on the left. Evidently, the results were the same for both solvers. Notice that there is a clear trend of quasi-monotonic convergence for the conducted analyses. It is natural to conceive that monotonic convergence may not occur in studies like

this one, since images with finer resolution (larger dimensions) are not refinements of the mesh *per se*. The interfaces between pore and solid tend to be better resolved with each improvement in resolution, so the geometry of the mesh is not entirely preserved, as it can be seen in Figure 6.1. It was important to observe that the proposed implementation is able to reproduce results in good agreement with a known solution for a “well-behaved” microstructure.

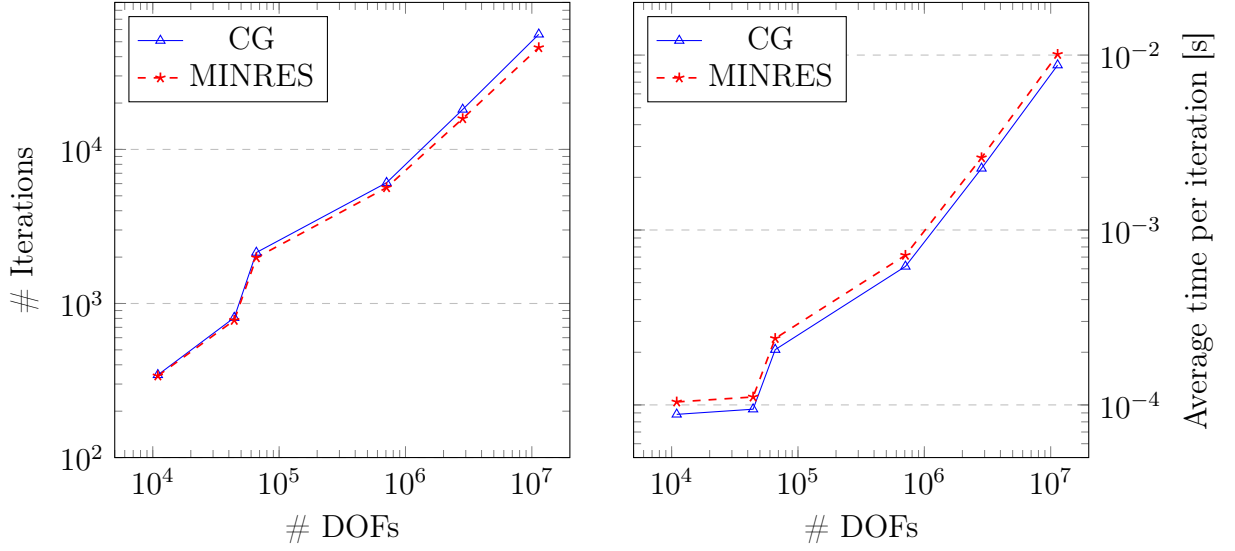


Figure 6.5: Metrics in relation to the number of DOFs of each analysis. To the left, the number of iterations for the residual to converge within the numerical tolerance. To the right, the average time per iteration. Both of these growths impact the overall elapsed time.

In regards to performance metrics, Figure 6.5 is presented. There, the amount of iterations and average time per iteration for each of the simulations are plotted against their respective number of DOFs. Notice that a quasi-linear tendency appears in both log-log axis. It is interesting to see that MINRES generally took less steps, but each of its iterations was always a bit slower in comparison to the CG method. This makes sense, as MINRES performs an extra reduction and an extra `daxpy`. As both of the number of iterations and the average time per iteration grow with increments in DOF counts, it is natural to expect a super-linear tendency for the overall elapsed time. The largest simulation in this experiment ( $2048 \times 2048$  pixels, 11 million DOFs), took about 7 minutes. It is estimated that a similar analysis would take a few hours in the CPU specified in Table 6.1 with a matrix-free approach. Recall that lots of iterations were required for the finest image (more than 10000), due to the relatively low numerical tolerance and the adoption of non-preconditioned solvers. Memory-wise, these simulations were all fairly

small, none require more than 600 MB to be allocated in the GPU. More relevant reports on memory allocation tendencies are made ahead for three-dimensional analyses.

Figure 6.6 shows the magnitude of the velocity field and resultant pressure field (accounting for the applied gradient and the computed fluctuation) for analyses in direction  $x_1$  with the  $64 \times 64$  and  $2048 \times 2048$  pixels images. Notice that the average of the velocity field yields the isotropic permeability of the model. The presented fields are in accordance with the expected response for the studied medium. The results were essentially the same between the CG and MINRES solvers. It is noticeable that, although numerical errors are evidently more prominent in the coarser image, the simulations with it are able to compute correct trends of both velocity and pressure.

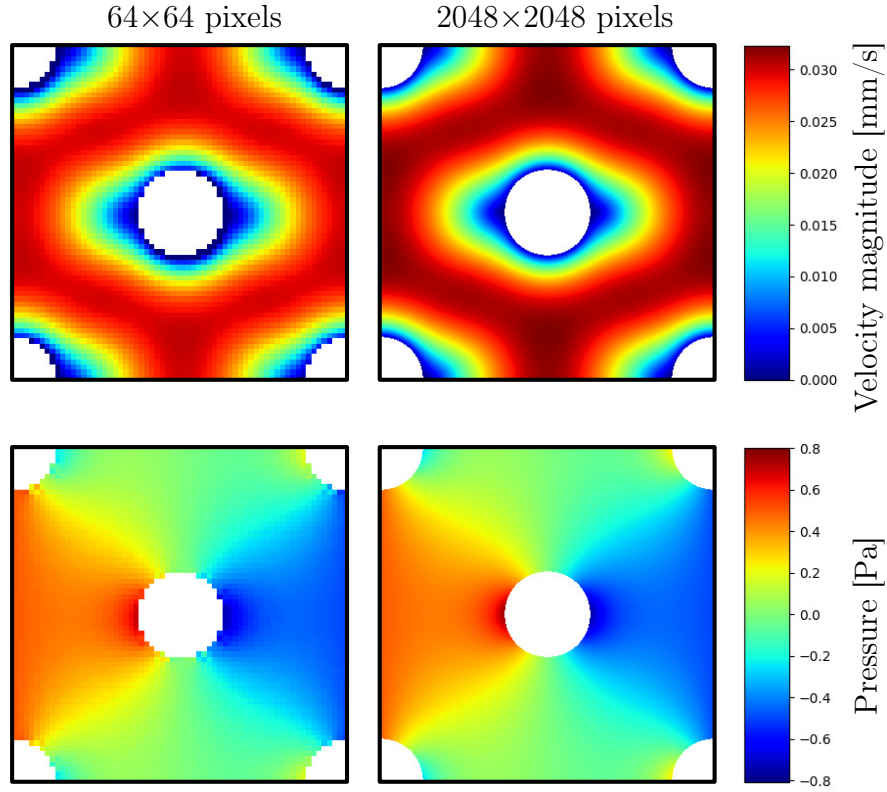


Figure 6.6: Velocity field magnitude =  $||u_i||$  [mm/s] (top) and pressure field =  $\overline{p}_i x_i + p^*$  [Pa] (bottom) for the coarser and finer samples.

Regarding stability, it is important to assess the convergence of each solver with respect to the adopted stopping criterion. For these analyses, the  $L_2$  norm of the residuals was employed. Figure 6.7 presents plots that depict the evolution of the normalized residual for the simulations with the coarsest and finest samples in Figure 6.1. Evidently, the MINRES method takes a more stable route to convergence. For all of the images,

the residual from the CG method oscillated significantly around a descending tendency that was always above the residual from MINRES. Consequently, the CG solver required more iterations, as seen in Figure 6.5. It is interesting to perceive, in this case, that both methods appear to have achieved a more inclined slope to convergence at around half of all the iterations.

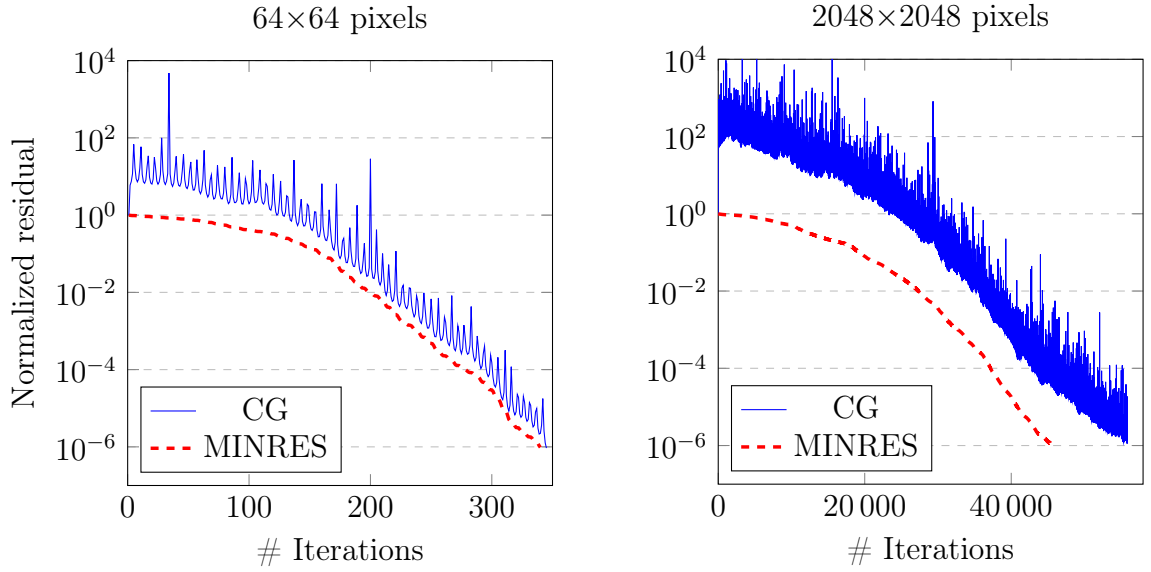


Figure 6.7: Convergence of the normalized residual by the  $L_2$  norm criterion for the coarsest and finest samples shown in Figure 6.1. Comparison between CG and MINRES solvers. Note that the curves for the  $2048^2$  pixels model have been sampled at each 10 iterations.

An interesting debate can be held upon these metrics and results. Qualitatively, both solvers obtained the same effective permeability, and velocity and pressure fields. Both took essentially the same time to do so. The CG method allowed for solutions with an array fewer stored in memory, although, as explored in Chapter 5, a memory-concise MINRES is feasible as well, at the cost of extra matrix-free matrix-vector products per iteration. It is seemingly implied that the CG is the better option among the two. However, the plots in Figure 6.7 make clear the issue with stability that comes with employing the CG method for a Stokes problem. The indefinite global matrix poses a challenge to finding the good steps  $\alpha \mathbf{d}^{(k)}$  based on conjugate search directions, and so the method tends to oscillate without a guarantee of convergence. In that sense, the stability of the MINRES method in this scope is an attractive factor. All in all, from this validation experiment, both solvers were shown to have pros and cons, and both will be considered forward as alternative approaches to solve the same problem. It was essential to verify

that the same results were achieved, and that they were in accordance with known values from the literature. In that way, they can be employed complementary to provide further confidence on the numerical results.

### 6.1.2 Three-dimensional analytical benchmark

Having verified the correctness of the implementation against a two-dimensional benchmark, the focus now shifts to three-dimensional problems, which are the main target of the proposed developments. In this experiment, the responses of the simulations are verified against a relatively simple 3D model of a periodic array of cylindrical pipes that permeate a solid body. A conceptual depiction of the domain is illustrated in Figure 6.8, and the adopted voxel-based periodic cell is shown beside it, in Figure 6.9. In this second Figure, notice that three different diameters for the pipe are considered. This sort of model is commonly studied for the validation of fluid flow simulations, as there is an analytical expression for the velocity field profile inside the pipe, known as Poiseuille flow. The idea here is to replicate the exercise seen in Yang et al. [151], by computing the absolute permeability of the medium and checking it against the expected value obtained from an expression derived from the Hagen-Poiseuille equation, like such:

$$\kappa = K_{22} = \frac{\overline{p}_{,2}\pi r^4}{8\mu A}, \quad (6.2)$$

where  $r$  is the radius of the pipe and  $A$  is the area of the cross-section of the periodic cell. It is recalled,  $\overline{p}_{,2}$  is the applied effective pressure gradient in direction  $x_2$ , and  $\mu$  is the fluid's dynamic viscosity. Here, both are equal to 1. To simplify matters even further, the cross-section of the periodic cell is admitted to be a  $1\text{mm} \times 1\text{mm}$  square. In that sense,

$$\kappa = \frac{\pi r^4}{8} \times 10^6 \text{ [D]}, \quad (6.3)$$

where  $r$  must be considered in mm.

As it was done in the first experiment, several image-based samples with increasingly improving resolution were created to study the convergence of the permeability results due to mesh quality. Notice, from Figure 6.9, that the 3D images are “thin”, that is, they have significant dimensions in directions  $x_1$  and  $x_3$ , and just a few voxels in  $x_2$ . Given the periodic boundary conditions, this should be enough to represent the ideally infinite pipes in direction  $x_2$ . Further, as mentioned previously, different models were created to consider three possible radii:  $\{0.0625, 0.125, 0.25\}$  mm. The porosity and DOF count for each of the created samples are presented in Table 6.3.

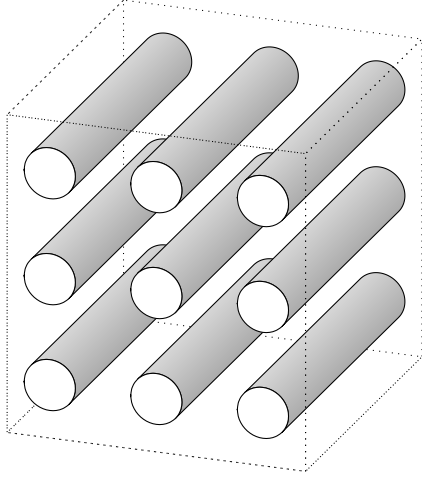


Figure 6.8: Periodic array of cylindrical pipes. Fluid flows solely within and along the pipes, the space between them can be understood as solid.

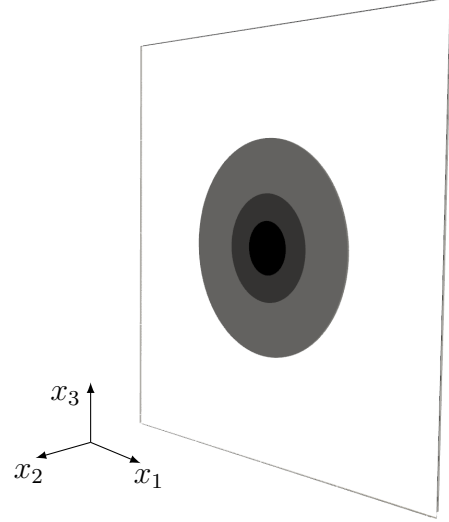


Figure 6.9: Periodic cell of an array of cylindrical pipes, with dimensions  $1\text{mm} \times 1\text{mm}$  in the  $x_1x_3$  cross-section. Three pipe diameters are considered: in black,  $0.125\text{mm}$ ; in darker grey,  $0.25\text{mm}$ ; in lighter grey,  $0.5\text{mm}$ . Notice that this is a three-dimensional model, with a “thickness” of 10 voxels. Fluid flow is simulated in direction  $x_2$ .

| Dimensions [voxels]          | $r = 0.0625\text{mm}$ |            | $r = 0.125\text{mm}$ |            | $r = 0.25\text{mm}$ |            |
|------------------------------|-----------------------|------------|----------------------|------------|---------------------|------------|
|                              | # DOFs                | $\phi$ [%] | # DOFs               | $\phi$ [%] | # DOFs              | $\phi$ [%] |
| $64 \times 64 \times 10$     | 1,800                 | 1.27       | 7,720                | 5.08       | 31,240              | 19.82      |
| $128 \times 128 \times 10$   | 7,720                 | 1.27       | 31,240               | 4.96       | 126,600             | 19.70      |
| $256 \times 256 \times 10$   | 31,240                | 1.24       | 126,600              | 4.93       | 510,600             | 19.67      |
| $512 \times 512 \times 10$   | 126,600               | 1.23       | 510,600              | 4.92       | 2,048,520           | 19.63      |
| $1024 \times 1024 \times 10$ | 510,600               | 1.23       | 2,048,520            | 4.91       | 8,215,240           | 19.64      |
| $2048 \times 2048 \times 10$ | 2,048,520             | 1.23       | 8,215,240            | 4.91       | 32,902,760          | 19.64      |
| Ideal cylinders              |                       | 1.23       |                      | 4.91       |                     | 19.64      |

Table 6.3: Image-based models with different resolutions for the reproduction of the cylindrical pipes (Figures 6.8 and 6.9). Notice that  $\phi$  was used to represent the porosity of each sample, for conciseness.

For these analyses, all of the “one matrix-vector product” solvers were employed, i.e. CG, MINRES, PCG and PMINRES. Naturally, it was goal of this experiment to validate the preconditioned solvers as well. The  $L_2$  norm of the residuals was taken as the stopping criterion, with a numerical tolerance of  $10^{-6}$ . The null vector was adopted as the starting point for all systems. The obtained effective permeability coefficients and their respective error from the analytical solution are presented in Figure 6.10.

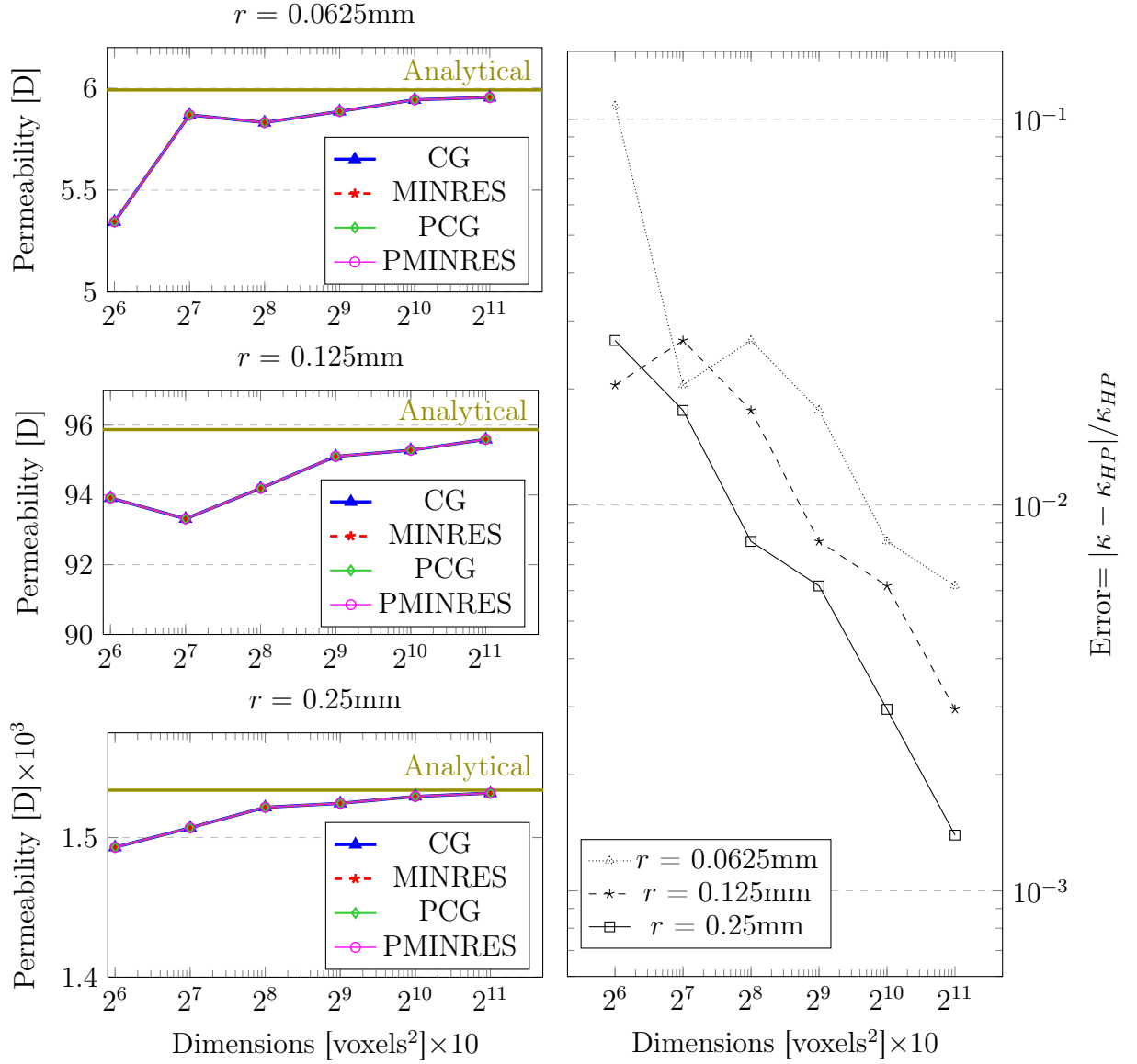


Figure 6.10: Convergence of the FEM-based permeability analysis for the model illustrated in Figure 6.9. To the left, the results for each image dimension, and for each radius of the pipes (from top to bottom). To the right, the numerical error on a logarithmic scale.  $\kappa$  is the permeability obtained by the simulation,  $\kappa_{HP}$  is the permeability given by the Hagen-Poiseuille equation.

From Figure 6.10, it is evident that the computed absolute permeability of the three different models all showed convergent behavior towards the values from the analytical expression. However, in contrast to what was observed in the first experiment, the convergence patterns were non-monotonic. Again, this can be expected, as finer images are not just refined meshes i.e. the ideally smooth walls of the pipe get better resolved geometrically at each increment in image dimensions. Still, from the error curves presented in the plot on the right, it is possible to see that, after some initial instability at the coarser

models, a somewhat stable descent behavior is verified. It is interesting to perceive that these curves have similar shapes to one another, and that the different radii seems to impact them as an offset. For example, notice that the error values for the  $r = 0.25\text{mm}$  model seem to closely match the error values for  $r = 0.125\text{mm}$  at the next (to the right) image. Overall, these simulations resulted in error metrics under 1% for all of the samples, and so this validation of the implemented solvers was deemed successful. It is important, then, to examine memory and time metrics. Both are presented in Figure 6.11.

In Figure 6.11, there are some interesting observations to be highlighted. Firstly, notice that the memory allocation of the solvers (in the plot on top) shows linear complexity, as expected. Be aware the the axis is in log-log scale for ease of visualization of the metrics for exponentially growing image dimensions, but it is stressed these straight lines remain linear tendencies in a linear-linear scale as well. As in the first experiment, MINRES (and PMINRES) allocates an extra array in comparison to CG. Further, it is made clear how porosity impacts allocation. Recall that only the pore space is meshed, so it makes absolute sense that the larger the diameter of the pipe, the greater the allocation. Overall, none of these were particularly large simulations, in comparison to our objective of dealing with  $1000^3$  voxels images, however, it is remarkable that the largest one here ( $2048 \times 2048 \times 10$  voxels, nearly 32 million DOFs) demanded roughly 1 GB to be allocated in the global memory of the GPU. Secondly, regarding processing time, notice, from the plot at the bottom, that a stable pattern of elapsed time for each of the models appears after the  $256 \times 256 \times 10$  voxels images (identified in the plot as  $2^8$ ). It is interesting to observe that these curves seem to be of similar shape, equally spaced, and shifted from one another (just like the error curves in Figure 6.10). This repeated differential pattern among curves of results and metrics is likely due to the “well-behaved” form of the domain (cylindrical pipes) and the adopted doubling increments in radius dimensions. Finally, it may be curious to some the fact that preconditioning did not seem to make much of difference in the elapsed time for the simulations. The diagonal preconditioner did lower a bit some of the number of iterations until convergence of the solvers, but not significantly. From the tests that were conducted in the scope of this Thesis, it was observed that the diagonal preconditioner does not help much with high-porosity voxel-based models that aim at reproducing ideally smooth solid-pore interfaces. However, as it will be shown in Section 6.2, it does appear to have a significant impact in accelerating convergence for simulations with actual  $\mu\text{CT}$  scans. Nonetheless, concerning the present experiment, it was satisfactory to see that the preconditioned solvers provided the same results as the non-preconditioned ones. In that sense, PCG and PMINRES have also been validated.

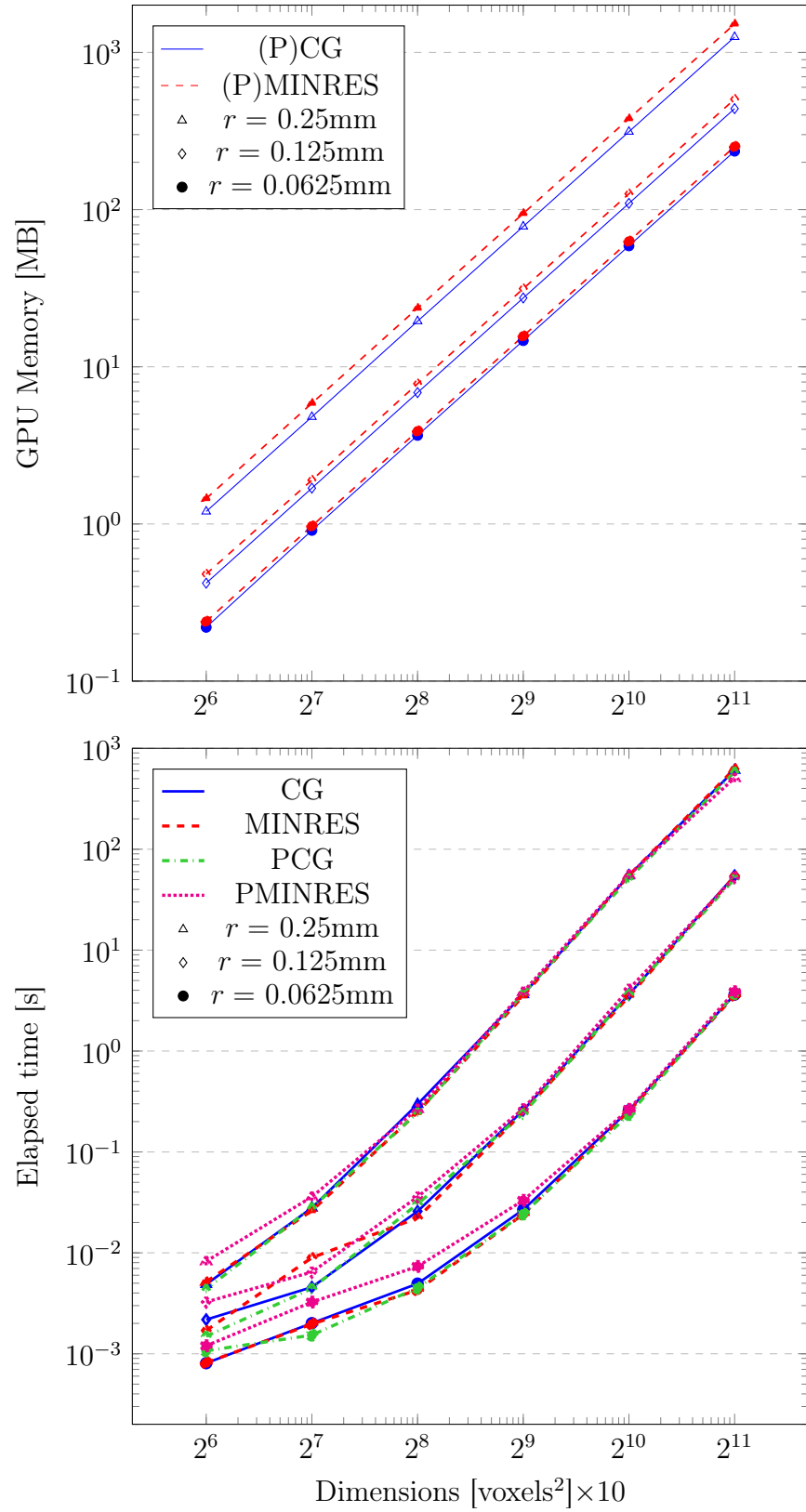


Figure 6.11: Memory (top) and time (bottom) metrics for the permeability analyses of the cylindrical pipes model.

### 6.1.3 Three-dimensional packed bed of spheres

At last, a third and final verification experiment was proposed: computing the absolute permeability of a packed bed of spheres, as depicted in Figures 6.12 and 6.13. The analysis of this sort of model is interesting in a scope of Digital Petrophysics, as it holds close similarities with the idealized domains that are employed in Kozeny-Carman expressions. Periodic domains of packed solids is a common abstraction used to approximate the porous microstructures of sandstone, for instance. Here, the goal is to reproduce a similar validation exercise to the one reported in Lesueur et al. [75], which is based in a semi-analytical prediction for a square packing of spheres based on the effective drag force acting in the domain. Accordingly to Gerke et al. [47], the absolute permeability in this case is given by

$$\kappa = K_{11} = K_{22} = K_{33} = \frac{r^2}{4.5(1 - \phi)F_d}, \quad (6.4)$$

where  $r$  is the radius of the spheres,  $\phi$  is the porosity of the medium, and  $F_d$  is a dimensionless effective drag force. Here, as indicated in Figure 6.13, the periodic cell is taken as a  $1\text{mm} \times 1\text{mm} \times 1\text{mm}$  cube and the spheres touch each other without superimposition, i.e.  $r = 0.5\text{mm}$ . For this scenario, accordingly to Zick and Homsy [154],  $F_d=42.1$ . With all of this in mind,

$$\kappa = \frac{0.5^2}{4.5 \times \left(\frac{4\pi 0.5^3}{3}\right) \times 42.1} \times 10^6 = 2.5203\text{e}+03 \text{ [D]}.$$

As it was done in the previous experiments, several samples with varying resolution were created, in an effort to investigate the convergence of permeability with respect to image resolution (mesh dimensions). The DOF count and porosity for each of the considered images are reported in Table 6.4. There, it is important to perceive that now we are dealing with large-scale meshes. Observe that the DOF count scales up rapidly, with the  $512^3$  and  $600^3$  voxels samples having the order of  $10^8$  unknowns to be computed. As an Nvidia GeForce 2080 card was employed for these simulations, all of them have to fit in under 8 GB (see Table 6.1) of memory allocation. For the largest sample, with roughly 400 million DOFs, notice that each vector of double precision variables requires 3.2 GB to be stored. Evidently, in this case, the 5 vector PMINRES or 4 vector PCG solvers are unfeasible with such memory restriction. In that sense, here, the 3 and 2 vectors alternatives are employed for the larger analyses, when required. Besides an extra validation step, this experiment aims to show that the proposed memory efficient implementations can be used to extend the limits of simulation size within a “tight” memory space.

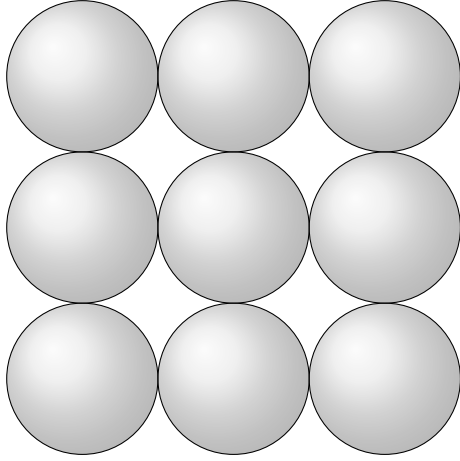


Figure 6.12: A periodic square packing of solid spheres.

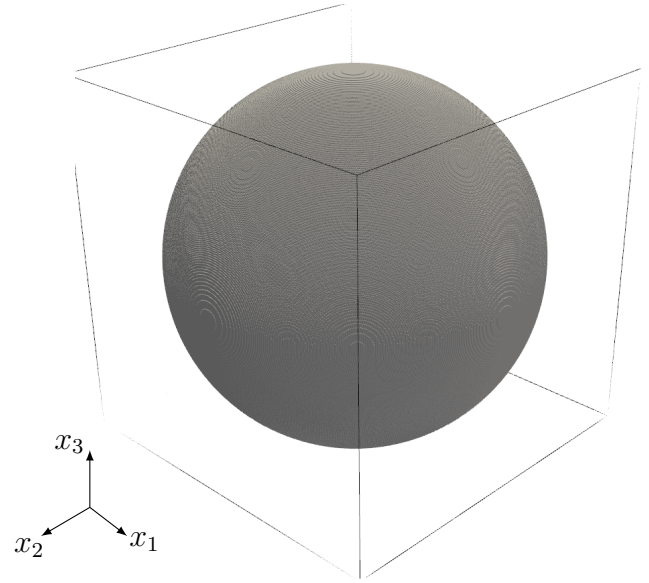


Figure 6.13: Periodic cell of a packed bed of spheres, a  $1\text{mm} \times 1\text{mm} \times 1\text{mm}$  volume. The sphere is solid and has a diameter of  $1\text{mm}$ . The empty volume around the sphere comprehends the pore space.

| Dimensions [voxels]         | # DOFs      | Porosity [%] |
|-----------------------------|-------------|--------------|
| $32 \times 32 \times 32$    | 57,300      | 47.34        |
| $64 \times 64 \times 64$    | 479,804     | 47.60        |
| $128 \times 128 \times 128$ | 3,914,812   | 47.59        |
| $256 \times 256 \times 256$ | 31,664,476  | 47.64        |
| $512 \times 512 \times 512$ | 254,527,060 | 47.64        |
| $600 \times 600 \times 600$ | 409,906,836 | 47.64        |
| Ideal sphere                |             | 47.64        |

Table 6.4: Image-based models with different resolutions for the reproduction of the packed bed of spheres (Figure 6.13).

In this experiment, all of the implemented solvers were employed. Once more, the  $L2$  norm of the residuals was admitted as the stopping criterion, accordingly to a numerical tolerance of  $10^{-6}$ . As done previously, all of the simulations here started from  $\mathbf{x}_0$  at the origin. The computed permeability values and the error from the semi-analytical prediction are shown in the plots presented in Figure 6.14.

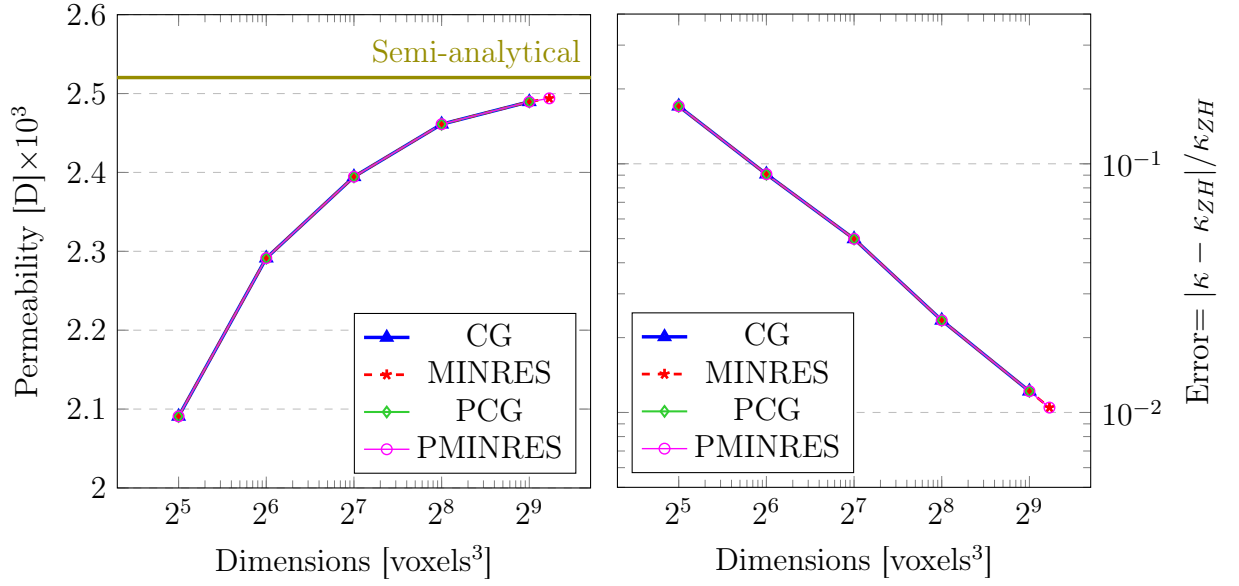


Figure 6.14: Convergence of the FEM-based permeability analysis for the model illustrated in Figure 6.13. To the left, the results for each image dimension. To the right, the numerical error on a logarithmic scale.  $k$  is the permeability obtained by the simulation,  $k_{ZH} = 2.5203\text{e}+03$  D is the permeability given by the semi-analytical solution based on the drag force coefficient from Zick and Homsy [154].

In Figure 6.14, it is satisfactory to observe that a smooth quasi-monotonic convergence was obtained for this model, much like what was obtained in the first experiment. Notice, however, that the computed permeability values seem to converge to an approximation slightly below the reference value, within less than 1% difference. This can be explained by the fact that, in Equation 6.4, permeability is expressed as inversely proportional to a drag force coefficient. The value of  $F_d = 42.1$  assumes ideally smooth spheres that only touch at a single point, while the considered voxel-based samples carry some imprecision in the representation of such model due to the rasterized nature of the mesh. In light of this, it is sensible to expect that  $F_d$  would be a bit higher for a voxelized sphere, and thus the permeability would in fact be smaller. Anyhow, it is clear that the proposed experiment provided a good approximation (at least) of the semi-analytical value. On that note, we move to computational metrics. Figure 6.15 presents memory allocation and processing time measurements for the conducted simulations.

In Figure 6.15, in the plot on the left, notice that memory allocation in GPU is reported for simulations that employ all of the implemented solvers, i.e. including the 3 and 2 vector versions. Dotted lines were used to depict how much memory would have been used if all of the simulations were conducted with each scheme. It is remarkable that,

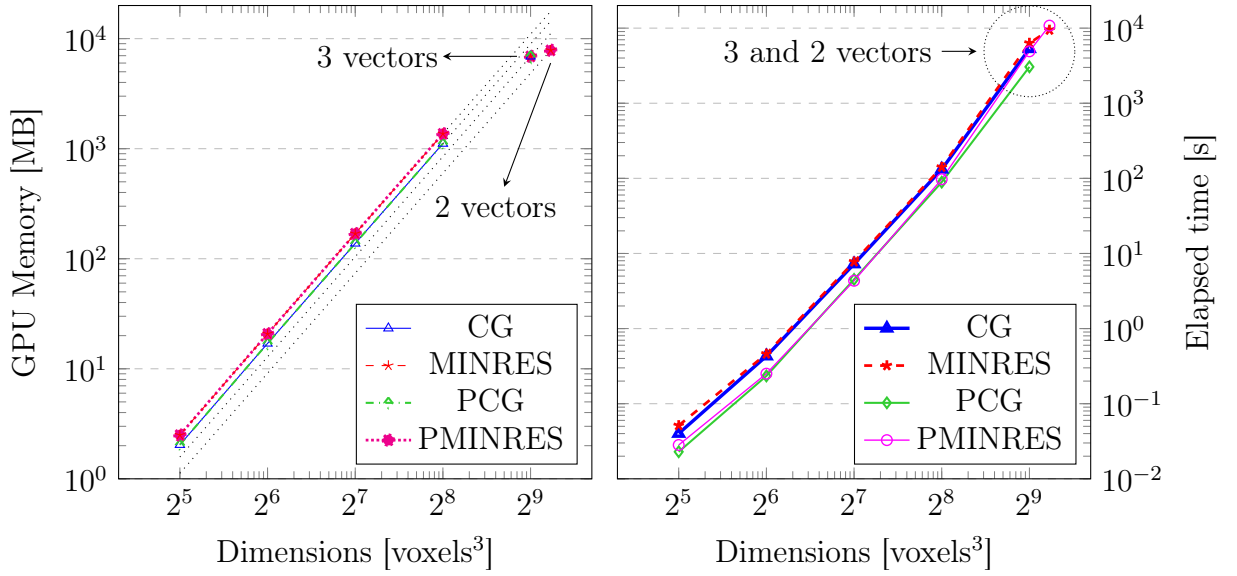


Figure 6.15: Memory (left) and time (right) metrics for the permeability analyses of the sphere pack model. Notice that the simulations with the 3 and 2 vectors solvers were highlighted in the plots.

by switching to the alternative implementations, larger images were able to be analyzed without having to change hardware to deal with the 8 GB limitation. **By employing PMINRES2 and MINRES2, 400 million DOFs simulations were carried out with a 7.9 GB footprint**, in regards to peak memory allocation. Further, from the plot on the right, observe that the larger simulations required the trade-off of memory for extra matrix-vector product iterations. Those took between 1h and 1h30min for the  $512^3$  voxels sample (254 million DOFS), and around 3h for the  $600^3$  voxels sample (400 million DOFS). All of the other samples demanded very few minutes or even seconds. It is important to be mindful that all of these analyses were conducted with a numerical tolerance of  $10^{-6}$  for relative norms of residuals, and thus required considerable amounts of iterations for convergence. The largest sample, for example, required around 10000 iterations with PMINRES. In most cases, it is likely to obtain essentially the same permeability values with higher numerical tolerance, thus fewer iterations, and thus lower processing times. In fact, as the interest here is in effective (volume averaged) velocity fields,  $10^{-6}$  can be excessive. This notion is corroborated by Figure 6.16, where the evolution of the homogenized permeability value throughout the iterations is compared to the evolution of the residual, considering the MINRES2 solver.

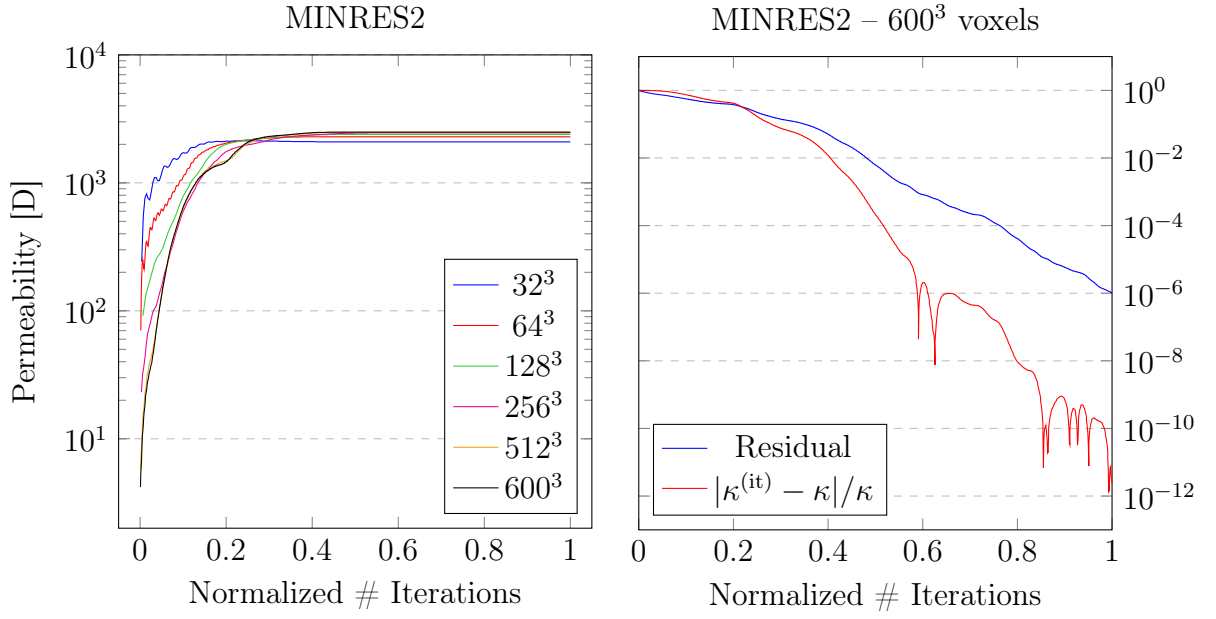


Figure 6.16: Evolution of permeability (left) and residual metrics (right) in regards to a normalized iteration count.  $|\kappa^{(it)} - \kappa|/\kappa$  represents the relative difference of the permeability at a given iteration from the final value, when the solver converged. Notice that the MINRES2 solver is considered for all of these curves, and the plot on the right depicts the behavior for the  $600^3$  voxels sample only. Similar patterns were seen for the other samples.

In Figure 6.16, it is vital to perceive that, for all of the considered samples, the effective permeability practically converged to the final values at around 40% of the iterations. In the plot on the right, notice that, from 50% of the iterations onward, the current permeability value  $\kappa^{(it)}$  had already matched the numerical solution  $\kappa$  within 6-digits precision, which was the adopted numerical tolerance. From these metrics, an important notion can be derived: a stopping criterion based on permeability values should be considered. The effective velocity fields appear to converge faster than the residuals associated with the numerical modeling of Stokes flow at the microscale. In future developments, it is an author's suggestion to test criteria such as a moving average of the rate of change of the homogenized permeability.

## 6.2 Validation with $\mu$ CT scans

After the verification of correctness against two- and three-dimensional benchmarks of known results, the developed program was employed for the characterization of the absolute permeability of three-dimensional samples of real materials imaged via  $\mu$ CT scanners.

The goal of these analyses was to validate the notion that the proposed implementation can be used for realistic studies of permeability from voxelized microstructural images. The models detailed in Table 6.5, illustrated in Figures 6.17 and 6.18, were considered. The scans of the fibrous sample were made available by Mehdikhani et al. [90] and were later segmented by Ferreira et al. [41]. Notice that both already are fairly large samples, with the fibers reaching 80 million DOFs. Due to possible anisotropy in these samples, which is especially clear for the fibers, it is necessary to perform permeability analyses in all three directions.

| Sample                                     | Dim. [voxels] | voxel [ $\mu\text{m}$ ] | # DOFs     | Porosity [%] |
|--------------------------------------------|---------------|-------------------------|------------|--------------|
| Boise IB <sup>1</sup> sandstone, Fig. 6.17 | $300^3$       | 2.52                    | 21,936,349 | 23.38        |
| Unidirectional fibers, Fig. 6.18           | $364^3$       | 1.10                    | 80,444,544 | 47.77        |

Table 6.5:  $\mu$ CT scans of real material samples.

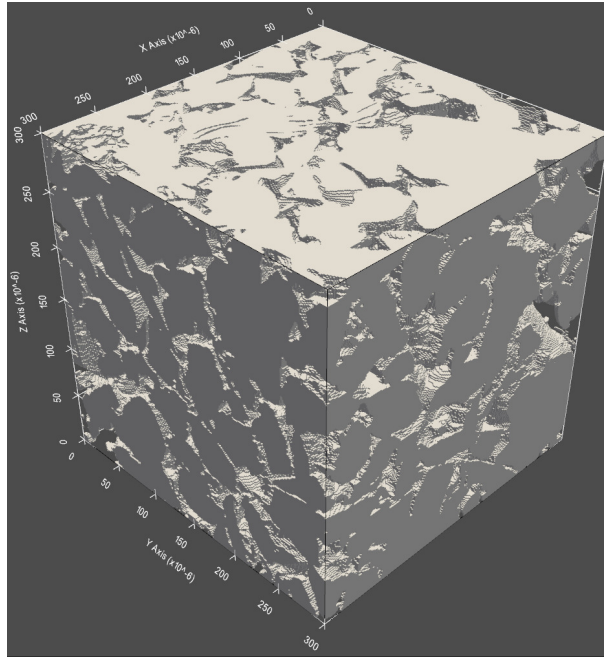
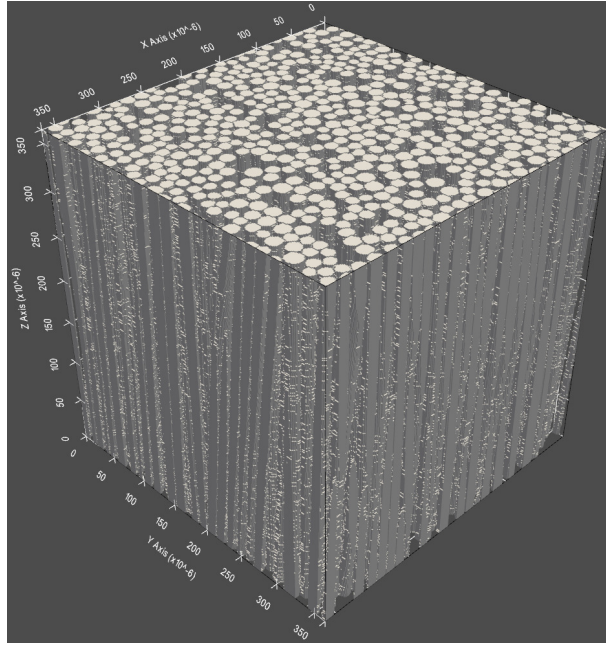


Figure 6.17:  $\mu$ CT scan of a Boise Idaho Brown sandstone.

The computational analyses of each of these two samples can provide important data for industry applications as the operation of oil wells (sandstone) and the manufacturing of fiber-reinforced composites (fibrous material). The absolute permeability of the sandstone sample aids in modeling the permeation of water, oil, or other fluids, through rock formations of this kind. This allows, for instance, for geologists and engineers to estimate regions near a well where fluids can be displaced so that the oil can be extracted. On

<sup>1</sup>Idaho Brown.

Figure 6.18:  $\mu$ CT scan of a fibrous material.

the other hand, the permeability of the fibrous material carries information about how to fabricate composites that consist of fibers embedded in a resin, for example. In fact, the sample published in [90] is made of elastic fibers embedded in a viscoelastic epoxy matrix. For the purposes of this work, the epoxy matrix was substituted by pores.

The absolute permeability coefficients obtained in each direction for the sandstone sample are shown in Table 6.6. A numerical tolerance of  $10^{-4}$  was considered for both the PCG and the PMINRES methods, admitting the  $L2$  norm of the residuals as the stopping criterion. These results are satisfactory, since they are within an expected range for this type of rock with such porosity. Accordingly to [71, 91, 113, 140, 143], the isotropic permeability for this sample should be within the interval of  $(0.7 \sim 1.5)e+03$  mD. It is stressed that the considered IB sample has a porosity of  $\sim 23\%$ , lower than the usual  $28\%$  for this kind of rock, so it is sensible to expect permeability closer to the lower bounds found in the literature, as it was obtained via the proposed FEM-based simulations.

| Direction                | Expected<br>$K_{ii}$ [mD] | PCG           |      |          | PMINRES       |      |          |
|--------------------------|---------------------------|---------------|------|----------|---------------|------|----------|
|                          |                           | $K_{ii}$ [mD] | Its. | Time [s] | $K_{ii}$ [mD] | Its. | Time [s] |
| $x_1 \Rightarrow K_{11}$ | $(0.7 \sim 1.5)e+03$      | $9.159e+02$   | 1358 | 41.2     | $9.159e+02$   | 1673 | 55.7     |
| $x_2 \Rightarrow K_{22}$ | $(0.7 \sim 1.5)e+03$      | $7.889e+02$   | 1486 | 44.8     | $7.890e+02$   | 1015 | 34.5     |
| $x_3 \Rightarrow K_{33}$ | $(0.7 \sim 1.5)e+03$      | $7.628e+02$   | 1070 | 31.8     | $7.627e+02$   | 1555 | 51.3     |

Table 6.6: Effective absolute permeability and computational metrics for the analysis of the sandstone sample.

In Table 6.6, it is interesting to compare the results and metrics obtained with the two implemented solvers. Notice that the PCG and PMINRES methods converged in amounts of iterations of the same order, although the PCG solver seems to be a little faster in general. The two solvers achieved qualitatively similar results, with small differences at the fourth significant digit. Recalling the  $10^{-4}$  numerical tolerance, these differences were deemed acceptable. The simulations demanded 0.97 GB to be allocated in RAM on the GPU for the PCG solver, while PMINRES required more: 1.15 GB. It stands out that these are lower tolls on memory than those reported in the matrix-free simulations (in CPU) performed by Yang et al. [151], where  $300^3$  voxels samples of sandstone with similar porosity required 4, 9 and 8 GB for simulations via the FEM, the LBM, and the FVM, respectively. Regarding stability, Figure 6.19 shows the convergence curves for the PCG, PMINRES, and additionally the MINRES method. Notice that MINRES is more stable than the others, but it also was about  $4\times$  slower for this sample. It is interesting to observe, though, that, among PMINRES and PCG, the first seems to be more stable. In that sense, a trade-off between computational efficiency and stability, regarding the CG and MINRES schemes, and either preconditioning or not, is made evident. It is remarkable that the performance of the preconditioned solvers for this digital sample of a real material stands in contrast to what was observed for the idealized smooth benchmarks. Here, the diagonal preconditioner clearly has aided in accelerating convergence.

Velocity magnitude fields are shown in Figure 6.20. The simulations in directions  $x_1$  and  $x_3$  are considered in this plot. Notice that the visualization of these fields is hindered by the presence of small regions where there is a concentration of higher velocities within the sample, and thus the majority of the pore space has relative low velocities. This is expected, as there are several pore channel narrowings between the grains, and, it is recalled, a low turbulence flow is assumed.

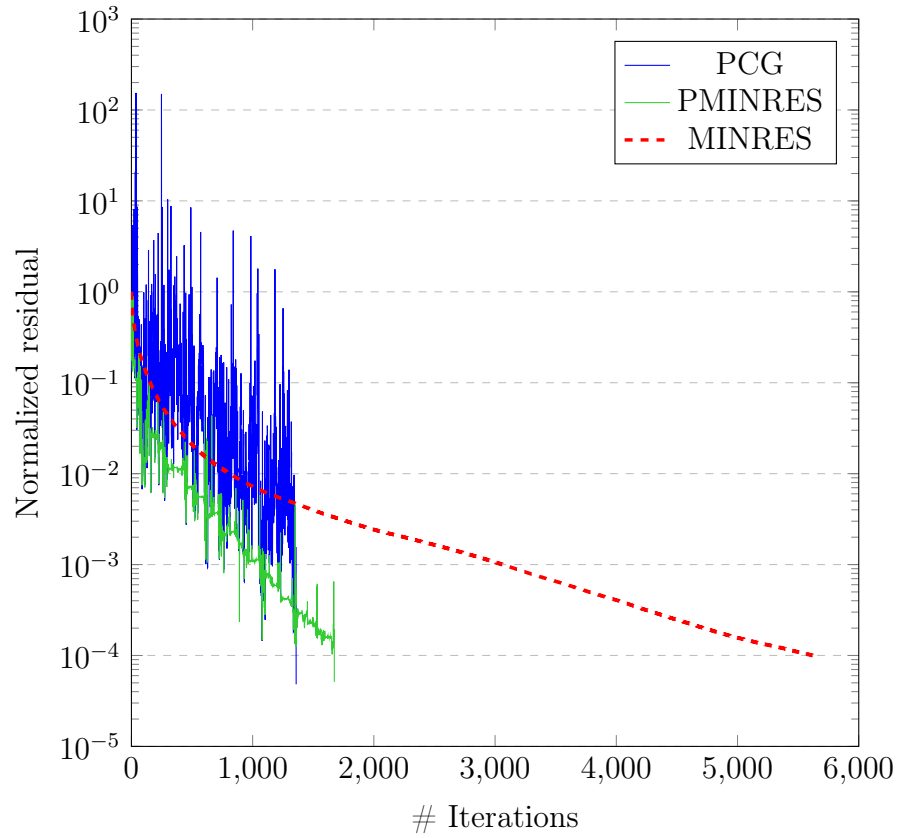


Figure 6.19: Convergence of the normalized residual by the  $L_2$  norm criterion for the analyses in direction  $x_1$  of the sandstone sample shown in Figure 6.17. Comparison between PCG, PMINRES and MINRES solvers.

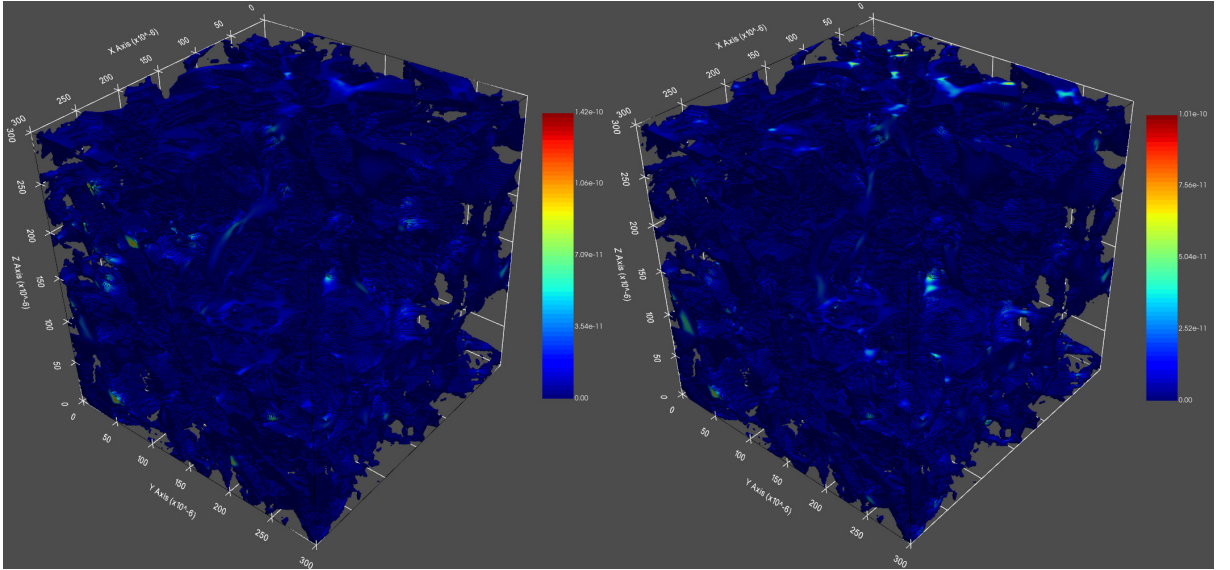


Figure 6.20: Magnitude of the velocity field  $\|u_i\|$  [m/s] obtained via simulations on the sandstone sample. Results from the simulations in directions in  $x_1$  on the left, and in  $x_3$  on the right.

The computed absolute permeability values in each direction for the fiber sample are shown in Table 6.7. Again, a numerical tolerance of  $10^{-4}$  was assumed for the PCG and PMINRES solvers. These results were also satisfactory for being of an expected order for this type of material. Seuffert et al. [131] obtained similar responses, via FVM simulations with the SIMPLE algorithm, for other samples of fibers with similar arrangement and about 48% porosity, as indicated in the table below. Further, these reference values are also in accordance with expected bounds by Gebart's expression for the permeability along fibrous domains [46].

| Direction                | Expected <sup>2</sup><br>$K_{ii}$ [mD] | PCG           |      |          | PMINRES       |      |          |
|--------------------------|----------------------------------------|---------------|------|----------|---------------|------|----------|
|                          |                                        | $K_{ii}$ [mD] | Its. | Time [s] | $K_{ii}$ [mD] | Its. | Time [s] |
| $x_1 \Rightarrow K_{11}$ | $\sim 2.0\text{e}+02$                  | 2.141e+02     | 1096 | 117      | 2.141e+02     | 1096 | 129      |
| $x_2 \Rightarrow K_{22}$ | $\sim 2.0\text{e}+02$                  | 2.397e+02     | 1453 | 157      | 2.397e+02     | 1456 | 171      |
| $x_3 \Rightarrow K_{33}$ | $\sim 2.0\text{e}+03$                  | 3.139e+03     | 890  | 95.9     | 3.139e+03     | 890  | 105      |

Table 6.7: Effective permeability and computational metrics for the fiber sample.

From Table 6.7, it is perceivable that the PCG solver was once more a bit faster than PMINRES, but not very significantly. The resulting permeability coefficients were in close agreement between the two methods. Regarding memory, the PCG solver required 3.02 GB to be allocated in RAM on the GPU, while PMINRES demanded 3.66 GB. Yet again convergence curves are shown, including a run of the MINRES method, as it can be seen in Figure 6.21. Notice that these curves show similar behavior to those of the sandstone sample (Figure 6.19), even though the microstructural arrangement of the two materials are considerably different. It was important to verify that, yet again, the diagonal preconditioner was of aid in accelerating convergence.

The magnitudes of the obtained velocity fields are shown in Figure 6.22. In contrast to the sandstone sample, the fiber sample produces clearly observable flow lines within the voxelized microstructure. In this Figure, results are shown for simulations in directions  $x_1$ , transversal to the fibers, and  $x_3$ , along the axial direction of the fibers. Notice that, as expected, higher velocity clusters appear in the pipe-like pore channels in  $x_3$ .

Considering that the permeability results for both of these samples matched expected ranges of values from the literature, these analyses were deemed successful. As the diagonal preconditioner appeared to speed up the process of obtaining these results, it was employed in the following experiments as well. It was vital to observe that the developed program can in fact be used to study digital samples obtained via  $\mu$ CT scans.

<sup>2</sup>Accordingly to Seuffert et al. [131]

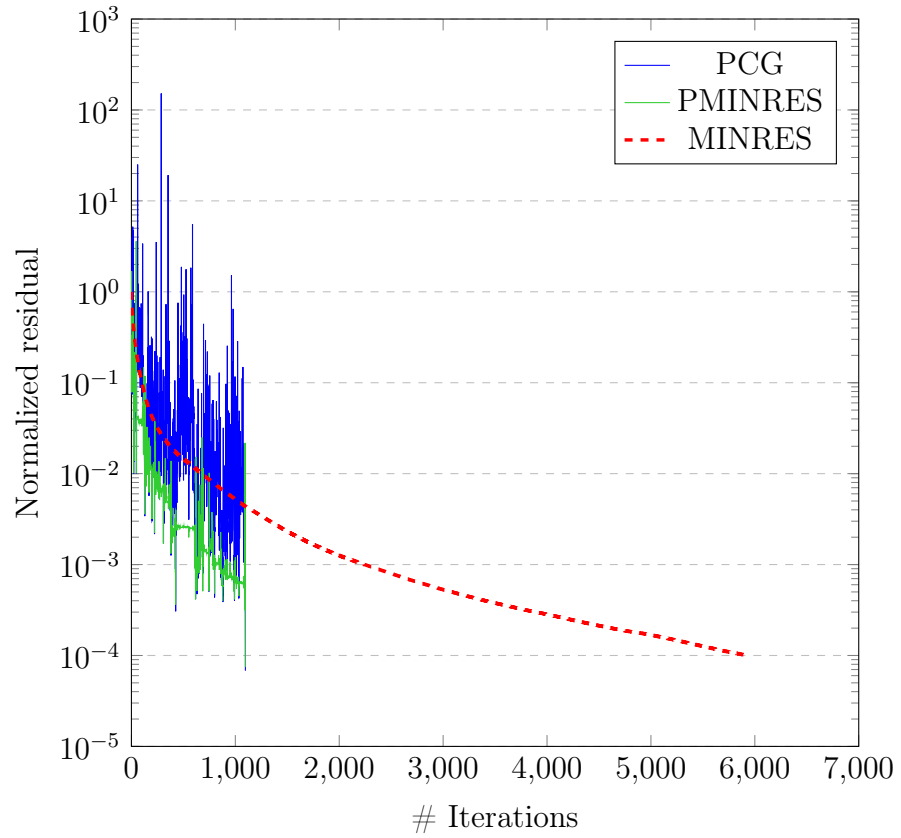


Figure 6.21: Convergence of the normalized residual by the  $L_2$  norm criterion for the analyses in direction  $x_1$  of the fiber sample shown in Figure 6.18. Comparison between PCG, PMINRES and MINRES solvers.

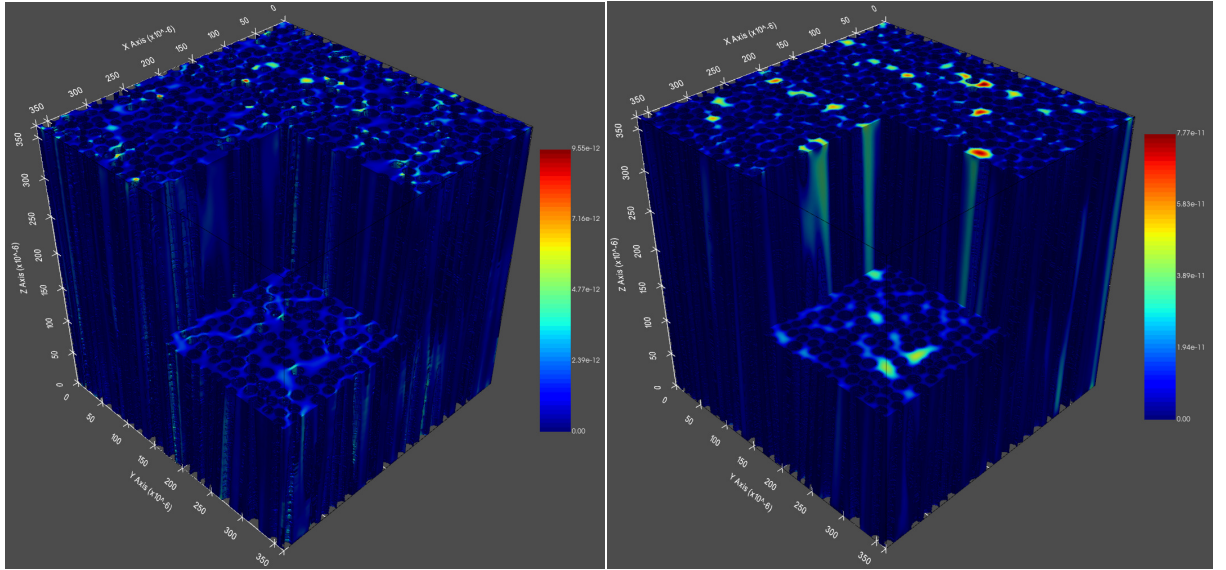


Figure 6.22: Magnitude of the velocity field  $\|u_i\|$  [m/s] obtained via simulations on the fiber sample. Results from the simulations in directions in  $x_1$  on the left, and in  $x_3$  on the right.

## 6.3 Permeability characterization of a batch of digital rock samples

At last, a series of analyses with the proposed implementations was carried out for large images of reservoir rock samples, all obtained via  $\mu$ CT scanning. These images are all of the same dimensions,  $600^3$  voxels, although with different amounts of DOFs, due to different porosity. Now, the hardware identified in Table 6.8 was employed. Observe that it has a 12 GB memory cap and more CUDA cores, in comparison to the machine used in previous experiments.

| O.S.            | Ubuntu 20.04.2 LTS         |
|-----------------|----------------------------|
| CPU             | Intel(R) Xeon(R) W-2295    |
| Clock rate      | 3.00 GHz                   |
| RAM             | 512 GB                     |
| Cores (threads) | 18 (36)                    |
| GPU             | Nvidia GeForce RTX 3080 Ti |
| Clock rate      | 1.67 GHz                   |
| RAM             | 12 GB                      |
| CUDA cores      | 10,240                     |
| Architecture    | Ampere                     |

Table 6.8: Hardware specifications for the simulations reported in this Section.

Firstly, 6 rock samples were considered for comparison between simulation results and laboratory measures ( $\kappa_{\text{lab}}$ ). They are described in Table 6.9. Notice that among these rocks there are sandstones and carbonates. Then, a batch of 17 rocks (including the first 6) was studied with the developed implementation, so that time and memory metrics could be tracked. The PCG and PMINRES2 solvers were adopted for all of the simulations in these experiments, with a numerical tolerance of  $10^{-4}$  and employing the relative  $L2$  norm of the residuals as the stopping criterion. The goal here was to showcase how the developed solvers can be realistically employed for Digital Petrophysics experiments, and to discuss some of the practical implications with adopting either a faster or a more lightweight solver.

| Rock id. | Name         | Type      | voxel [ $\mu\text{m}$ ] | Porosity [%] | $\kappa_{\text{lab}}$ [mD] |
|----------|--------------|-----------|-------------------------|--------------|----------------------------|
| BN       | Bentheimer   | sandstone | 5.0                     | 18.55        | 2700.0                     |
| BR       | Briarhill    | sandstone | 3.0                     | 16.39        | 4700.0                     |
| IB       | Idaho Brown  | sandstone | 2.0                     | 20.93        | 1510.0                     |
| BS       | Berea Stripe | sandstone | 2.0                     | 19.25        | 426.0                      |
| AC       | Austin Chalk | carbonate | 1.0                     | 13.17        | 14.4                       |
| DP       | Desert Pink  | carbonate | 0.9                     | 31.01        | 115.0                      |

Table 6.9: Sandstone and carbonate reservoir rocks studied as further validation of the implementations. All of these have experimental values for absolute permeability. Each of the considered microstructural images had  $600^3$  voxels. Slices of each of these rocks are illustrated in Figure 6.23.

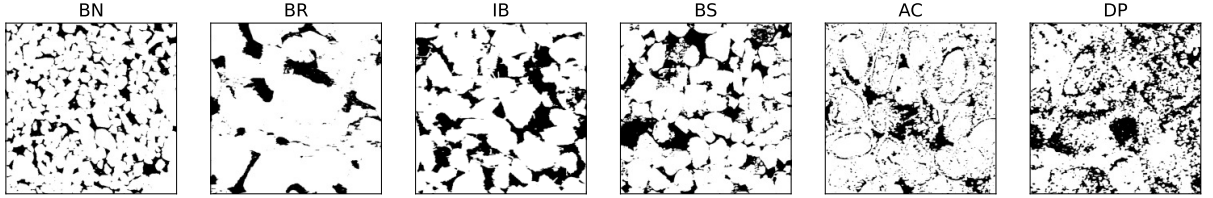


Figure 6.23: Two-dimensional slices of the  $600^3$  voxels samples detailed in Table 6.9. Pore space in black, solid phase in white.

The homogenization analyses were performed in all three directions, and the effective isotropic permeability ( $\kappa$ ) of each rock sample was taken as the harmonic mean of the absolute permeability coefficients in each direction, i.e. the main diagonal terms in the constitutive tensor:

$$\kappa = (K_{11} \cdot K_{22} \cdot K_{33})^{1/3}.$$

Generally, the simulations for each of the rocks presented in Table 6.9 provided similar results in all directions, indicating that these are all in fact nearly isotropic.

The obtained permeability values are compared to the laboratory measures in Figure 6.24. It is stressed that the results from the PCG and PMINRES2 solvers matched each other within the adopted numerical tolerance. In that Figure, notice that a fairly good agreement was achieved between simulation and experimental data. It stands out, at the top right corner, that the results of the computational analyses of three sandstone samples BN, BR and IB deviated a bit from the known values, having obtained lower permeability. That can be expected, as the respective porosities of the studied images were all lower than those of the actual samples, as indicated by the colormap. On the other hand, the result for the AC carbonate sample matched well the experimental data, with lower porosity as well. The hypothesis for this observation is that, as this is a signi-

ificantly less permeable sample than those sandstones, it is natural to imagine that there are poorly connected pores in its microstructure that do not contribute much to the fluid flow. It is possible get a sense of this by observing the slice of the AC sample shown in Figure 6.23, especially put in contrast to the sandstone samples shown in the same Figure. In light of this, if unconnected pores were removed in the segmentation process, it should not affect much the effective permeability value, even if porosity is reduced. All in all, given that these are  $600^3$  voxels subvolumes of the scanned samples, and that there is some uncertainty associated with both the laboratory experiments and the image acquisition process, the results obtained via simulations were deemed acceptable.

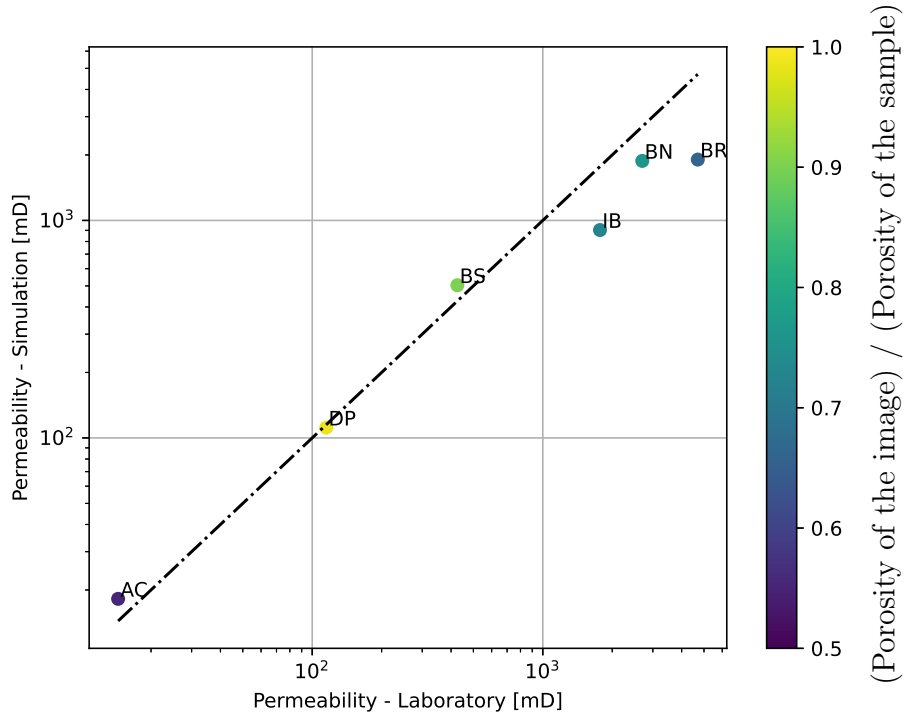


Figure 6.24: Comparison of permeability values obtained via simulation against laboratory measures

The whole batch of reservoir rocks, with 17 models obtained from  $\mu$ CT scans, is briefly described in Table 6.10. The isotropic permeability coefficients obtained for all of these are shown in Figure 6.25. It is satisfactory to observe that different trends of permeability  $\times$  porosity were obtained for carbonates and sandstones. As expected, the carbonates appear to be less permeable than sandstones, which becomes especially clear for porosity higher than 10%.

| Rock id. | Type      | voxel [ $\mu\text{m}$ ] | Porosity [%] |
|----------|-----------|-------------------------|--------------|
| AC       | carbonate | 1.0                     | 13.17        |
| DP       | carbonate | 0.9                     | 31.01        |
| EW       | carbonate | 1.0                     | 31.50        |
| IH       | carbonate | 1.5                     | 03.09        |
| IL       | carbonate | 2.0                     | 05.21        |
| IM       | carbonate | 2.0                     | 03.77        |
| LU       | carbonate | 2.0                     | 05.34        |
| SD       | carbonate | 2.0                     | 15.21        |
| BB       | sandstone | 2.0                     | 12.15        |
| BG       | sandstone | 2.0                     | 12.82        |
| BIG      | sandstone | 5.0                     | 21.56        |
| BN       | sandstone | 5.0                     | 18.55        |
| BR       | sandstone | 3.0                     | 16.39        |
| BS       | sandstone | 2.0                     | 19.25        |
| CO       | sandstone | 2.0                     | 02.74        |
| IB       | sandstone | 2.0                     | 20.93        |
| KI       | sandstone | 2.0                     | 21.67        |

Table 6.10: Batch of image-based models from 17 reservoir rock samples.

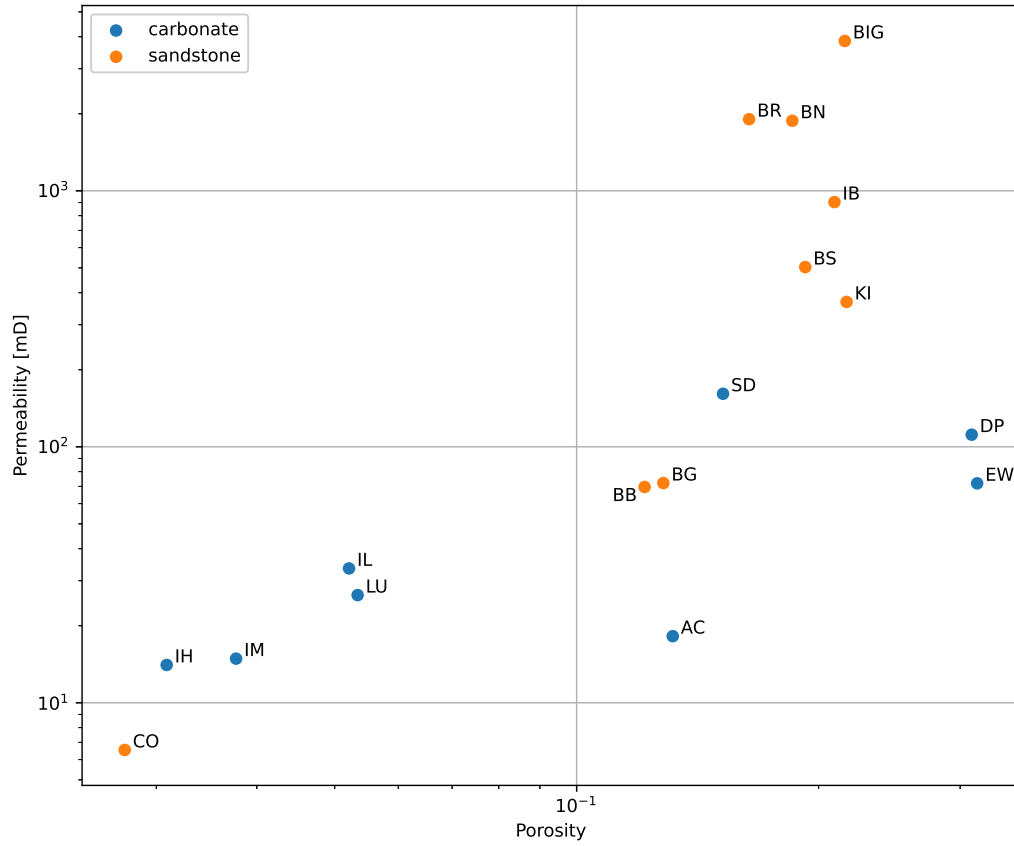


Figure 6.25: Permeability  $\times$  porosity plot for the batch of reservoir rock samples

Time metrics were tracked for these analyses. For this study, as indicated in Table 6.8, a Nvidia GeForce RTX 3080 (12 GB RAM) was employed, instead of the RTX 2080 used in the previous experiments. In Figure 6.26, the average time per simulation is plotted against the number of DOFs of each sample. There, observe that different markers are used to depict PCG and PMINRES2 runs. Comparing this with Figure 6.25, it is noticeable that porosity and number of DOFs are correlated, as intended. It is very important to observe that there are clear trends in elapsed time  $\times$  number of DOFs in this log-log plot. This allows for fair estimates of elapsed time to be made for analyses of reservoir rocks based on the number of DOFs the image-based model has.

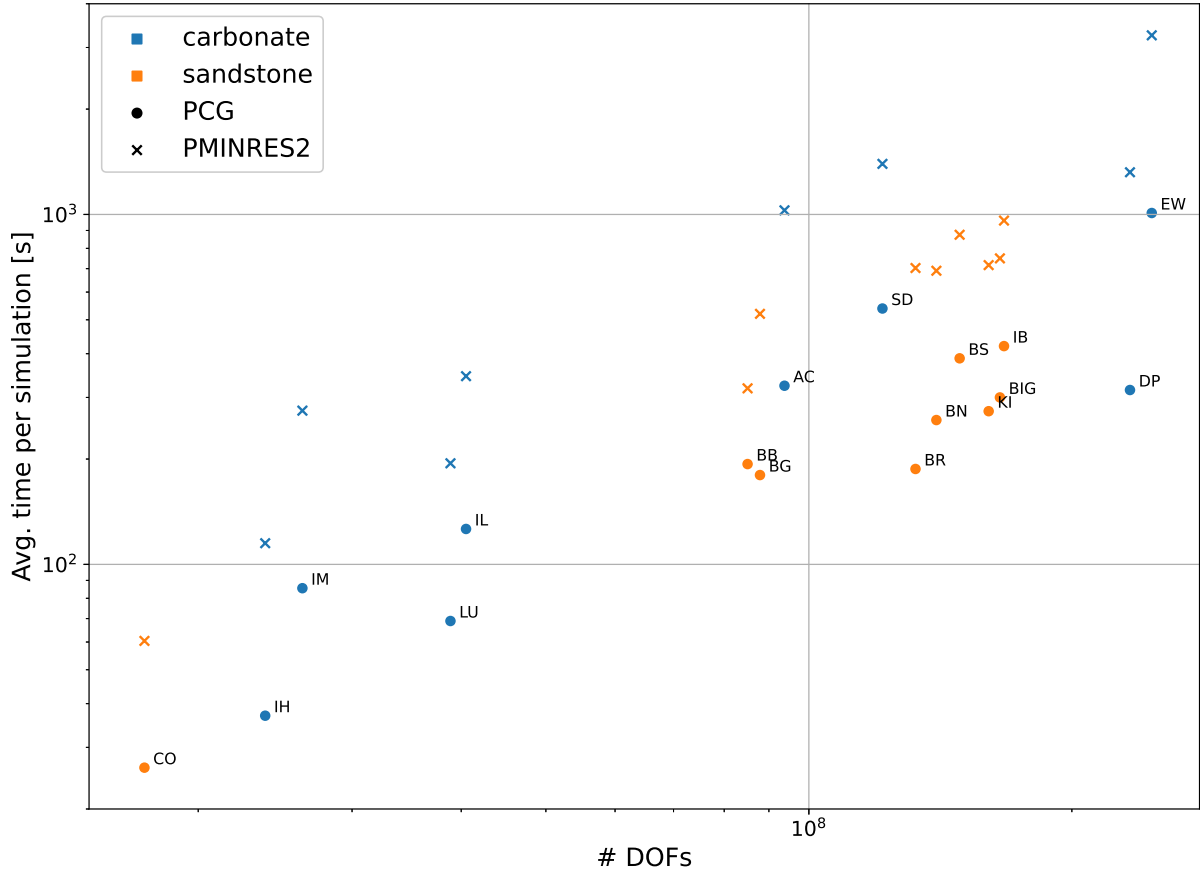


Figure 6.26: Average elapsed time per simulation  $\times$  number of DOFs plot for the batch of reservoir rock samples. All models have  $600^3$  voxels.

From Figure 6.26, it is possible to observe different trends for the PCG and PMINRES2 solvers, as expected. In practice, it can be inferred from this plot that PMINRES2 solutions were roughly  $3\times$  slower than those with the PCG. Recall that it performs extra matrix-vector products to allocate two vectors fewer. Nevertheless, it is important to perceive that similar trends and distributions were obtained between the two considered solvers for sandstones and carbonates, with the PMINRES2 metrics essentially being just shifted up in overall time. Further, on that note, it appears that a separate trend just for carbonates would be a bit higher on the graph (slower) than one just for the sandstones, although not very significantly. All in all, looking at the simulations with 100 million DOFs or more, the measured run times were within a range of 3min to 20min with the PCG solver, while PMINRES2 took 13min to nearly 1h for the same samples. In practical terms, the proposed implementations allowed for permeability analyses of microstructures depicted by  $600^3$  voxels images (216 million voxels) in times of the order of tens of minutes. It is stressed that it is not uncommon to see works in the literature reporting

run times of hours (in CPU) for such models of similar dimensions, as in [47, 97, 144]. Here, considering just the PCG solver, the analyses of all of these 17 samples, in three directions each, took roughly 6 hours to complete in a desktop workstation equipped with a gaming-dedicated GPU (see Table 6.8). Further, the required memory for peak allocation in RAM generally stayed well under 8 GB, especially with the PMINRES2 solver. With the PCG solver (4 vectors), the largest simulations for the EW sample (31.5% porosity, 246 million DOFs) required roughly 9 GB of peak RAM allocation. On that note, memory metrics are plotted in Figure 6.27.

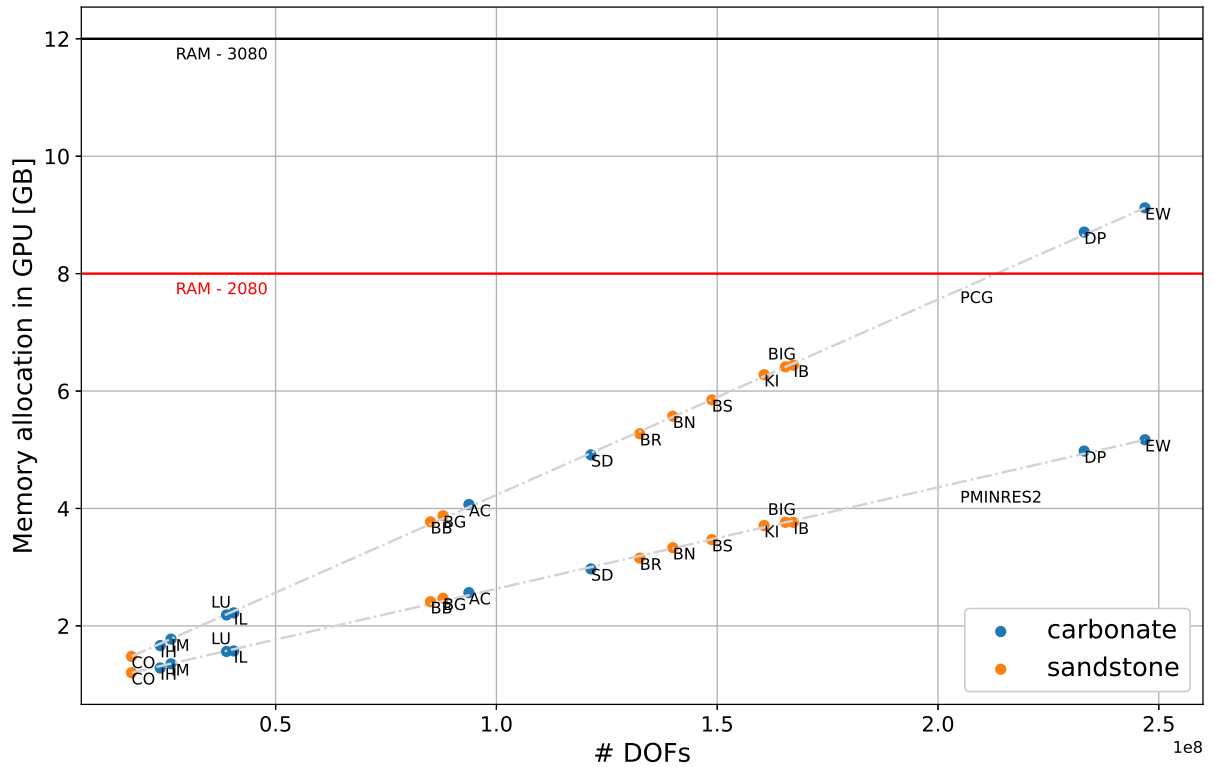


Figure 6.27: Peak memory allocation in GPU per simulation  $\times$  number of DOFs plot for the batch of reservoir rock samples. All models have  $600^3$  voxels. Notice that different trends of allocation were highlighted for the PCG and PMINRES2 solvers.

In Figure 6.27, it is important to observe that a linear tendency of memory allocation vs. number of DOFs was verified. Notice the axis is in linear-linear scale, not log-log. The trends for PCG and PMINRES2 are marked via grey dash-dotted lines, and the memory cap for each of the graphics cards available at the laboratory are highlighted. Clearly, the 2 vector strategy serves as a viable alternative to extend simulation sizes with limited hardware. Finally, a comparison of the achieved memory metrics is made

with the distributed (MPI) matrix-free FEM solution proposed by Yang et al. [151], in Figure 6.28. There, the memory metrics reported in their paper for  $300^3$  and  $400^3$  voxels samples are plotted against the metrics obtained in this Thesis. Notice that the horizontal axis now reports the amount of pore voxels, instead of number of DOFs. Evidently, both of the strategies explored here showed less inclined slopes in the linear tendency of memory allocation. It is important to bear in mind that, besides the implementation, the mathematical modeling in Yang et al. was different from the one shown in this Thesis. They have employed a time-dependent approach to compute the absolute permeability of voxelized microstructures, and that can have an impact in memory requirements. Still, to the best of the author's knowledge, theirs is the work that comes closest to the approach devised here, in terms of exploring structured FE meshes from  $\mu$ CT images without storing the global matrix. This comparison is meant to show that the entire solution proposed in this Thesis, not just the code itself, yielded lower memory allocation.

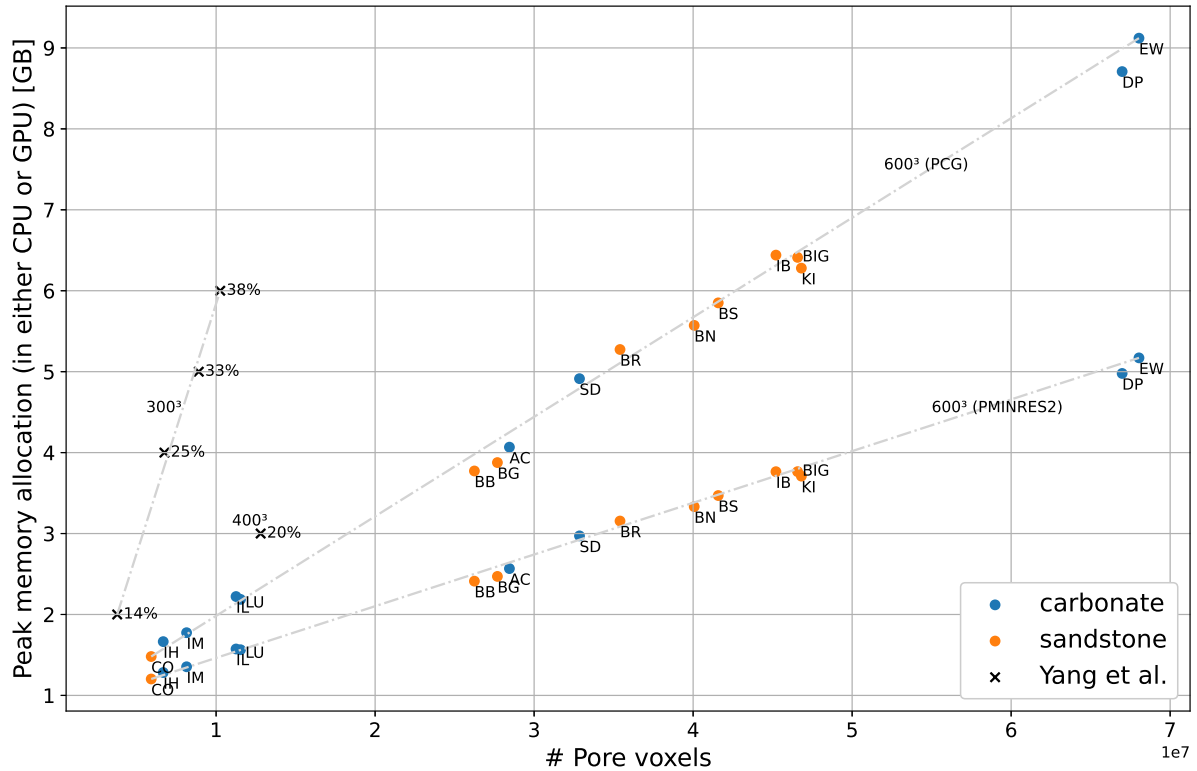


Figure 6.28: Peak memory allocation per simulation  $\times$  number of pore voxels plot for the batch of reservoir rock samples. Notice that memory metrics reported in Yang et al. [151] were plotted as well. The porosity of their samples was written next to their respective markers, for quick reference and comparison.

### 6.3.1 Gigavoxel analyses

At last, to solidify the capabilities of the memory efficient versions of the implemented solvers, namely PCG2 and PMINRES2, *gigavoxel* studies were conducted, i.e. analyses of samples with approximately 1 billion voxels or more. The considered image-based models were constructed from the Austin Chalk (AC) carbonate and the Bandera Brown (BB) sandstone samples from the batch of reservoir rocks. Subvolumes with  $500^3$  voxels were taken from the original  $600^3$  voxels domains, and then were mirrored in all directions to achieve  $1000^3$  voxels “semi-synthetic” samples. It is stressed that, due to a tight memory limitation, the AC sample was shrunk to  $964^3$  voxels, so a nearly gigavoxel model. These two new samples are illustrated in Figures 6.29 and 6.30, and have their details described in Table 6.11. Notice that both have more than 400 million DOFs.

| Rock id. | Type      | voxel [ $\mu\text{m}$ ] | Porosity [%] | Dim. [voxels] | # DOFs      |
|----------|-----------|-------------------------|--------------|---------------|-------------|
| AC       | carbonate | 1.0                     | 15.15        | $964^3$       | 452,034,234 |
| BB       | sandstone | 2.0                     | 12.31        | $1000^3$      | 400,282,542 |

Table 6.11: Characteristics of the gigavoxel samples.

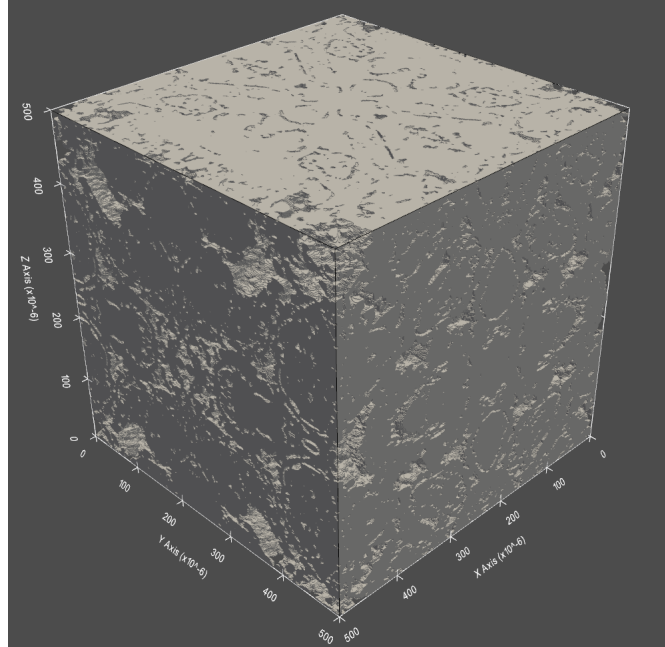


Figure 6.29:  $964^3$  voxels image constructed from an Austin Chalk (AC) carbonate sample. The original  $600^3$  voxels volume from  $\mu\text{CT}$  scans was mirrored to achieve this image.

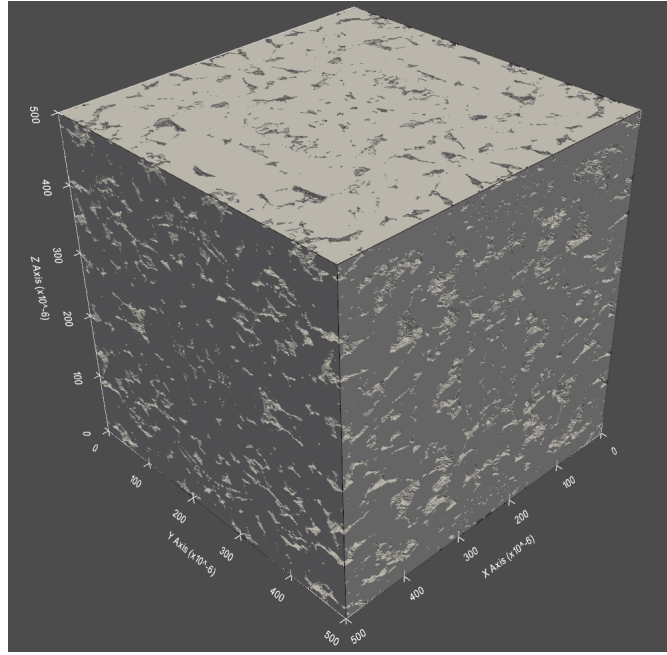


Figure 6.30:  $1000^3$  voxels image constructed from a Bandera Brown (BB) sandstone sample. The original  $600^3$  voxels volume from  $\mu$ CT scans was mirrored to achieve this image.

For these analyses, the PMINRES2 solver was employed, a numerical tolerance of  $10^{-4}$  was admitted, and the  $L2$  norm of the residuals was taken as the stopping criterion. Once more, the Nvidia GeForce 3080 (12 GB) GPU was employed. Simulations were conducted solely in the  $x_3$  (vertical) direction, and so the following reported permeability coefficients are equivalent to

$$\kappa = K_{33}.$$

Then, results and metrics for these experiments are reported below, in Table 6.12.

| Rock id. | $\kappa$ [mD] | Memory [GB] | Time [s] |
|----------|---------------|-------------|----------|
| AC       | 45.424        | 11.78       | 5.29e+03 |
| BB       | 66.467        | 11.27       | 2.32e+03 |

Table 6.12: Permeability results, time and memory metrics for the analyses of the giga-voxel samples. PMINRES2 was employed.

From Table 6.12, notice that both permeability results are in qualitative accordance with the results from the  $600^3$  voxels samples. They are a slightly higher than those previous values, and from reference values for this sort of rock from core sample manufacturers [71], but that can be due to the artificial mirroring of the samples. However, the key observation to be made here is in regards to computational metrics: **Billion voxel samples with 400 million DOFs required 40min (sandstone) and 1h30min (carbonate)**

**to have their permeability computed with a single consumer-grade graphics card, allocating less than 12 GB in memory.** This is a remarkable feat, especially considering that all of the simulations were conducted with a matrix-free FEM approach. In that sense, it is an author's hope that this sort of implementation may be further explored in future developments for the permeability of voxelized microstructures.

# Chapter 7

## Final remarks

In this Thesis, a fast and memory-efficient matrix-free FEM-based scheme was proposed for the computation of the effective absolute permeability of large voxelized porous microstructures. The physical phenomenon of fluid flow through a porous medium was modeled from strong- to weak-form Stokes equations, presenting how these can be approximately solved via an image-based FEM approach. Matrix-free operations were implemented as CUDA kernels, to run in GPU, that can be employed in different solvers. Herein, two Krylov subspace methods were explored: (P)CG and (P)MINRES, both suited for symmetric matrices, as are the global matrices that arise from stabilized quasi-incompressible flow. It stands out, though, that (P)MINRES is generally the most indicated method for such indefinite matrices [136] that arise from a saddle point problem.

It was shown how the NBN strategy can be adapted for meshes that encompass just the pore space within a structured fixed-grid-like domain obtained from  $\mu$ CT scans. It was then extended to a SBS approach that simplifies the computations for nodes within pores in matrix-free operations. In doing so, simulations that would take hours, or perhaps even days, in the CPU of a desktop computer are expected to take minutes, or a few hours at most, by making use of gaming-dedicated GPUs.

The implementation of the massively parallel solvers in a CPU/GPU heterogeneous computing scenario was detailed, including an in-depth discussion of the reasoning behind each decision regarding the actual code. In doing so, two distinct paradigms were considered: (i) one matrix-vector product per iteration; and (ii) fewer vectors stored in memory. (i) strides for peak performance with the matrix-free FEM scheme, but can be optimized in terms of memory usage, while (ii) aims at storing the bare minimum in global memory but has to recompute more floating point values at each iteration. The memory vs. time trade-off is made evident in this context. From (i), the CG, MINRES, PCG and PMINRES

solvers were developed. From (ii), (P)CG3, (P)MINRES3, (P)CG2, (P)MINRES2 were proposed. These were all tested and validated in the conducted numerical experiments reported in this Thesis.

Three validation experiments were performed with voxel-based meshes that model ideal domains for which there are analytical or semi-analytical predictions of absolute permeability: a two-dimensional case, and two three-dimensional problems. All of the proposed analyses showed clear convergent tendencies of permeability to the expected values, with quasi-monotonic error decrease with respect to improvements in image resolution. All of the implemented solvers obtained the same results in all of these experiments, within a specified numerical tolerance, and so they were validated. These first experiments were important for the assessment of performance metrics and the stability of each of the proposed solvers. The MINRES method was shown to achieve much more stable convergence patterns than the CG method, as expected, although it requires more computational resources, either memory or processing time.

After verification against (semi-)analytical benchmarks, the solvers were applied for the characterization of a sandstone and a fibrous material, via digital samples obtained with  $\mu$ CT scans. Sensible results were obtained in comparison with reference values from the literature, and the adopted diagonal preconditioning was shown to be a valuable asset in accelerating the convergence of simulations in digital samples of real materials.

At last, the proposed massively parallel matrix-free FEM scheme was applied for the study of absolute permeability of  $600^3$  voxels images of reservoir rocks in desktop hardware. It was shown that simulations in sandstone and carbonate samples of up to 250 million DOFs can be conducted in a matter of 3min to 20min allocating no more than 9 GB in memory with the PCG solver, while PMINRES2 required from 13min to roughly 1h allocating at most nearly 5 GB. Finally, as a push of simulation size limits with the available hardware (12 GB), gigavoxel analyses were conducted for a carbonate and a sandstone sample. These took from 40min to 1h30min to run.

These reported computational metrics denote an improvement from the usual toll of tens of GB and hours of processing time in CPU that is seen in the literature for simulations of such dimension. Interestingly, this was achieved with a FEM-based approach, a method that is usually avoided in works regarding large-scale voxel-based permeability computation due to its alleged elevated computational cost, mostly in regards to memory requirements. In that sense, this work indicates that, yes, the FEM can be a feasible alternative in this scope as well.

Referring back to the main objective and three key goals stated in Chapter 1, all of the proposed aims were achieved. A set of massively parallel algorithms was devised for memory-efficient absolute permeability computation in consumer-grade graphics cards. It was shown that the proposed implementation is well-suited to handle digital samples of up to  $1000^3$  voxels in desktop computers. The NBN approach, which had been previously employed for thermal conductivity and elasticity characterization, was successfully expanded for the simulation of Stokes flow in porous media, exploring an SBS scheme for inner pore nodes. The results and metrics observed were satisfactory, both in terms of validating correctness and feasibility in reasonable time spans.

## 7.1 Future work

Future efforts that would immediately follow the developments shown in this Thesis include extension of the matrix-free kernels to consider quadratic finite elements, the study of stopping criteria specifically tailored for the permeability analyses, and the search for better preconditioners and starting points for the iterative solvers. In that sense, in the near future, the Author aims to:

- Implement matrix-free kernels for Q2Q1 and Q2P-1 finite elements, still in the framework of a SBS approach for inner pore nodes.
- Test stopping criteria based on the rate of change of the permeability instead of relative norms of residuals.
- Employ similar matrix-free FEM Laplace solvers to handle block subsystems at each iteration, as a hopefully faster preconditioning technique than the diagonal preconditioner.
- Employ such Laplace solver to compute a starting configuration of the pressure fluctuation field, then use it to compute a starting velocity field as well.

In a broader spectrum of possible future endeavors, the methods devised here open doors for many applications. For example, the strategy presented to differentiate phases in a structured grid can be employed in other scenarios, as in multi-physics simulations – where different laws have to be considered in each phase; and problems with domains discretized by block-structured meshes – where elements that are “distorted” similarly, i.e. same Jacobian, may be bundled together. Further, there is an interesting prospect

to apply these solvers for material design, aiming at unit cells with optimal permeability properties. This sort of optimization study has been conducted in 2D, with smaller meshes, but the presented efficient solvers may be employed in this context for large 3D simulations. Finally, referring back to the motivation of this work, the developed software will be applied in the construction of large datasets that include permeability and NMR data, associated with images, to aid in the training of Machine Learning and Artificial Intelligence models that may provide additional insight into permeability prediction for carbonate rocks.

# References

- [1] ADLER, P.; JACQUIN, C.; QUIBLIER, J. Flow in simulated porous media. *International Journal of Multiphase Flow* 16, 4 (1990), 691–712.
- [2] AFFDL, J. C. H.; KARDOS, J. L. The halpin-tsai equations: A review. *Polymer Engineering & Science* 16, 5 (1976), 344–352.
- [3] AKANJI, L. T.; MATTHAI, S. K. Finite element-based characterization of pore-scale geometry and its impact on fluid flow. *Transport in Porous Media* 81, 2 (Jan 2010), 241–259.
- [4] ALCOCER, F. J.; KUMAR, V.; SINGH, P. Permeability of periodic porous media. *Phys. Rev. E* 59 (Jan 1999), 711–714.
- [5] ALCOCER, F. J.; SINGH, P. Permeability of periodic arrays of cylinders for visco-elastic flows. *Physics of Fluids* 14, 7 (2002), 2578–2581.
- [6] ALPAK, F. O.; GRAY, F.; SAXENA, N.; DIETDERICH, J.; HOFMANN, R.; BERG, S. A distributed parallel multiple-relaxation-time lattice boltzmann method on general-purpose graphics processing units for the rapid and scalable computation of absolute permeability from high-resolution 3d micro-ct images. *Computational Geosciences* 22, 3 (Jun 2018), 815–832.
- [7] ANDRÄ, H.; COMBARET, N.; DVORKIN, J.; GLATT, E.; HAN, J.; KABEL, M.; KEEHM, Y.; KRZIKALLA, F.; LEE, M.; MADONNA, C.; MARSH, M.; MUKERJI, T.; SAENGER, E. H.; SAIN, R.; SAXENA, N.; RICKER, S.; WIEGMANN, A.; ZHAN, X. Digital rock physics benchmarks-part i: Imaging and segmentation. *Computers & Geosciences* 50 (2013), 25–32. Benchmark problems, datasets and methodologies for the computational geosciences.
- [8] ANDRÄ, H.; COMBARET, N.; DVORKIN, J.; GLATT, E.; HAN, J.; KABEL, M.; KEEHM, Y.; KRZIKALLA, F.; LEE, M.; MADONNA, C.; MARSH, M.; MUKERJI, T.; SAENGER, E. H.; SAIN, R.; SAXENA, N.; RICKER, S.; WIEGMANN, A.; ZHAN, X. Digital rock physics benchmarks-part ii: Computing effective properties. *Computers & Geosciences* 50 (2013), 33–43. Benchmark problems, datasets and methodologies for the computational geosciences.
- [9] ANDREASSEN, E.; ANDREASEN, C. How to determine composite material properties using numerical homogenization. *Computational Materials Science* 83 (2014), 488–495.
- [10] ARBENZ, P.; FLAIG, C.; KELLENBERGER, D. Bone structure analysis on multiple GPGPUs. *Journal of Parallel and Distributed Computing* 74, 10 (2014), 2941–2950.

- [11] ARBOGAST, T.; BRUNSON, D. S. A computational method for approximating a darcy–stokes system governing a vuggy porous medium. *Computational Geosciences* 11, 3 (Sep 2007), 207–218.
- [12] ARNS, C. H.; BAUGET, F.; LIMAYE, A.; SAKELLARIOU, A.; SENDEN, T. J.; SHEPPARD, A. P.; SOK, R. M.; PINCZEWSKI, W. V.; BAKKE, S.; BERGE, L. I.; ØREN, P. E.; KNACKSTEDT, M. A. Pore-Scale Characterization of Carbonates Using X-Ray Microtomography. *SPE Journal* 10, 04 (12 2005), 475–484.
- [13] ARNS, C. H.; KNACKSTEDT, M. A.; PINCZEWSKI, M. V.; LINDQUIST, W. B. Accurate estimation of transport properties from microtomographic images. *Geophysical Research Letters* 28, 17 (2001), 3361–3364.
- [14] BARRAL, M.; CHATZIGEORGIOU, G.; MERAGHNI, F.; LÉON, R. Homogenization using modified mori-tanaka and tfa framework for elastoplastic-viscoelastic-viscoplastic composites: Theory and numerical validation. *International Journal of Plasticity* 127 (2020), 102632.
- [15] BEN-YAACOV, I.; ROIG, F. Index notation for vector calculus. Tech. rep., New Mexico Tech, Socorro, USA, 2006. Last accessed in March 8th of 2023.
- [16] BENZI, M. Preconditioning techniques for large linear systems: A survey. *Journal of Computational Physics* 182, 2 (2002), 418–477.
- [17] BEZ, L. F.; LEIDERMAN, R.; SOUZA, A.; AZEREDO, R. B. D. V.; PEREIRA, A. M. B. An image-based explicit matrix-free fem implementation with lumped mass for nmr simulations. *Transport in Porous Media* 147, 1 (Mar 2023), 35–57.
- [18] BLUNT, M. J. Flow in porous media - pore-network models and multiphase flow. *Current Opinion in Colloid & Interface Science* 6, 3 (2001), 197–207.
- [19] BLUNT, M. J.; BIJELJIC, B.; DONG, H.; GHARBI, O.; IGLAUER, S.; MOSTAGHIMI, P.; PALUSZNY, A.; PENTLAND, C. Pore-scale imaging and modelling. *Advances in Water Resources* 51 (2013), 197–216. 35th Year Anniversary Issue.
- [20] BROCKENBROUGH, J.; SURESH, S.; WIENECKE, H. Deformation of metal-matrix composites with continuous fibers: geometrical effects of fiber distribution and shape. *Acta Metallurgica et Materialia* 39, 5 (1991), 735–752.
- [21] CAREY, G. F.; JIANG, B.-N. Element-by-element linear and nonlinear solution schemes. *Communications in Applied Numerical Methods* 2, 2 (1986), 145–153.
- [22] CARMAN, P. Fluid flow through granular beds. *Chemical Engineering Research and Design* 75 (1997), S32–S48. Original article submitted in 1937.
- [23] CIORANESCU, D.; DONATO, P.; ENE, H. I. Homogenization of the stokes problem with non-homogeneous slip boundary conditions. *Mathematical Methods in the Applied Sciences* 19, 11 (1996), 857–881.
- [24] CIORANESCU, D.; PAULIN, J. S. J. Homogenization in open sets with holes. *Journal of Mathematical Analysis and Applications* 71, 2 (1979), 590–607.

- [25] CNUDDÉ, V.; BOONE, M.; DEWANCKELE, J.; DIERICK, M.; VAN HOOREBEKE, L.; JACOBS, P. 3D characterization of sandstone by means of X-ray computed tomography. *Geosphere* 7, 1 (02 2011), 54–61.
- [26] CNUDDÉ, V.; MASSCHAELE, B.; DIERICK, M.; VLASSENBROECK, J.; HOOREBEKE, L. V.; JACOBS, P. Recent progress in x-ray ct as a geosciences tool. *Applied Geochemistry* 21, 5 (2006), 826–832. *Frontiers in Analytical Geochemistry-An IGC 2004 Perspective*.
- [27] COOK, R. D. *Finite Element Modeling for Stress Analysis*, 1 ed. John Wiley and Sons, Madison, USA, 1994.
- [28] CORSO, G. M. D.; MENCHI, O.; ROMANI, F. Krylov subspace methods for solving linear systems. Tech. rep., Università di Pisa, Dipartimento di Informatica, Pisa, Italy, 2015. Last accessed in March 31st of 2023.
- [29] DA CRUZ, J. P.; OLIVEIRA, J.; TEIXEIRA-DIAS, F. Asymptotic homogenisation in linear elasticity. part i: Mathematical formulation and finite element modelling. *Computational Materials Science* 45, 4 (2009), 1073–1080.
- [30] DIETRICH, F. Periodic homogenization and fft-based methods for resolving microstructures of linear magneto-elastic problems. Phd Thesis. Technische Universität Kaiserslautern, Germany, August 2021. Last accessed in March 1st of 2023.
- [31] DOS ANJOS, C. E. M.; DE MATOS, T. F.; AVILA, M. R. V.; DE CASTRO VARGAS FERNANDES, J.; SURMAS, R.; EVSUKOFF, A. G. Permeability estimation on raw micro-ct of carbonate rock samples using deep learning. *Geoenergy Science and Engineering* 222 (2023), 211335.
- [32] DRUMMOND, J.; TAHIR, M. Laminar viscous flow through regular arrays of parallel solid cylinders. *International Journal of Multiphase Flow* 10, 5 (1984), 515–540.
- [33] DUARTE, L. S.; CELES, W.; PEREIRA, A.; MENEZES, I. F. M.; PAULINO, G. H. Polytop++: an efficient alternative for serial and parallel topology optimization on cpus and GPUs. *Structural and Multidisciplinary Optimization* 52, 5 (2015), 845–859.
- [34] DVORAK, G. J. Transformation field analysis of inelastic composite materials. *Proceedings: Mathematical and Physical Sciences* 437, 1900 (1992), 311–327.
- [35] DVORKIN, J.; DERZHI, N.; DIAZ, E.; FANG, Q. Relevance of computational rock physics. *Geophysics* 76, 5 (11 2011), E141–E153.
- [36] DVORKIN, J.; DERZHI, N.; FANG, Q.; NUR, A.; NUR, B.; GRADER, A.; BALDWIN, C.; TONO, H.; DIAZ, E. From micro to reservoir scale : Permeability from digital experiments. *The Leading Edge* 28, 12 (12 2009), 1446–1452.
- [37] EPOV, M.; SHURINA, E.; KUTISCHEVA, A. Computation of effective resistivity in materials with microinclusions by a heterogeneous multiscale finite element method. *Physical Mesomechanics* 20, 4 (2017), 407–416.

- [38] ERHEL, J.; TRAYNARD, A.; VIDRASCU, M. An element-by-element preconditioned conjugate gradient method implemented on a vector computer. *Parallel Computing* 17, 9 (1991), 1051–1065.
- [39] FERGUSON, J. C.; PANERAI, F.; BORNER, A.; MANSOUR, N. N. Puma: The porous microstructure analysis software. *SoftwareX* 7 (2018), 81–87.
- [40] FERGUSON, J. C.; SEMERARO, F.; THORNTON, J. M.; PANERAI, F.; BORNER, A.; MANSOUR, N. N. Update 3.0 to “puma: The porous microstructure analysis software”,(pii: S2352711018300281). *SoftwareX* 15 (2021), 100775.
- [41] FERREIRA, L. P.; LOPES, P. C. F.; LEIDERMAN, R.; ARAGÃO, F. T. S.; PEREIRA, A. M. B. An image-based numerical homogenization strategy for the characterization of viscoelastic composites. *International Journal of Solids and Structures* 267 (2023), 112142.
- [42] FISH, J.; BELYTSCHKO, T. *A First Course in Finite Elements*, 1 ed. Wiley, 2007.
- [43] FLAIG, C.; ARBENZ, P. A scalable memory efficient multigrid solver for micro-finite element analyses based on ct images. *Parallel Computing* 37, 12 (2011), 846–854. 6th International Workshop on Parallel Matrix Algorithms and Applications (PMAA’10).
- [44] FONG, D.; SAUNDERS, M. Cg versus minres: An empirical comparison. *SQU Journal for Science* 17 (2012), 44–64.
- [45] GARBOCZI, E.; DAY, A. An algorithm for computing the effective linear elastic properties of heterogeneous materials: Three-dimensional results for composites with equal phase poisson ratios. *Journal of the Mechanics and Physics of Solids* 43, 9 (1995), 1349–1362.
- [46] GEBART, B. Permeability of unidirectional reinforcements for rtm. *Journal of Composite Materials* 26, 8 (1992), 1100–1133.
- [47] GERKE, K. M.; VASILYEV, R. V.; KHIREVICH, S.; COLLINS, D.; KARSANINA, M. V.; SIZONENKO, T. O.; KOROST, D. V.; LAMONTAGNE, S.; MALLANTS, D. Finite-difference method stokes solver (fdmss) for 3d pore geometries: Software development, validation and case studies. *Computers & Geosciences* 114 (2018), 41–58.
- [48] GONÇALVES, E. C.; DA SILVA, P. N.; SILVEIRA, C. S.; CARNEIRO, G.; DOMINGUES, A. B.; MOSS, A.; PRITCHARD, T.; PLASTINO, A.; DE VASCONCELOS AZEREDO, R. B. Prediction of carbonate rock type from nmr responses using data mining techniques. *Journal of Applied Geophysics* 140 (2017), 93–101.
- [49] GUEDES, J. M.; KIKUCHI, N. Preprocessing and postprocessing for materials based on the homogenization method with adaptive finite element methods. *Computer Methods in Applied Mechanics and Engineering* 83, 2 (1990), 143–198.
- [50] GUPTA, P. Cuda refresher: The cuda programming model. Website, Nvidia Developer Blog, 2020. Available at <https://developer.nvidia.com/blog/cuda-refresher-cuda-programming-model/>. Last accessed in June 27th of 2021.

- [51] GUTKNECHT, M. A brief introduction to krylov space methods for solving linear system. Tech. rep., ETH Zurich, Zurich, Switzerland, 2007. Last accessed in March 31st of 2023.
- [52] HALPIN, J. C. *Primer on Composite Materials Analysis (revised)*, 2 ed. Routledge, 1992.
- [53] HASHIN, Z.; SHTRIKMAN, S. On some variational principles in anisotropic and nonhomogeneous elasticity. *Journal of the Mechanics and Physics of Solids* 10, 4 (1962), 335–342.
- [54] HEATH, M. T. *Scientific Computing: An introductory survey*, 2 ed. SIAM, Philadelphia, USA, 2018.
- [55] HELFENSTEIN, R.; KOKO, J. Parallel preconditioned conjugate gradient algorithm on GPU. *Journal of Computational and Applied Mathematics* 236 (2012), 3584–3590.
- [56] HESTENES, M.; STIEFEL, E. Methods of conjugate gradients for solving linear systems. *Journal of Research of the National Bureau of Standards* 49, 6 (12 1952), 409–436.
- [57] HILL, R. Elastic properties of reinforced solids: Some theoretical principles. *Journal of the Mechanics and Physics of Solids* 11, 5 (1963), 357–372.
- [58] HOLLISTER, S. J.; KIKUCHI, N. Homogenization theory and digital imaging: A basis for studying the mechanics and design principles of bone tissue. *Biotechnology and Bioengineering* 43, 7 (1994), 586–596.
- [59] HUGHES, T. J.; FRANCA, L. P.; BALESTRA, M. A new finite element formulation for computational fluid dynamics: V. circumventing the babuška-brezzi condition: a stable petrov-galerkin formulation of the stokes problem accommodating equal-order interpolations. *Computer Methods in Applied Mechanics and Engineering* 59, 1 (1986), 85–99.
- [60] HUGHES, T. J.; FRANCA, L. P.; MALLET, M. A new finite element formulation for computational fluid dynamics: I. symmetric forms of the compressible euler and navier-stokes equations and the second law of thermodynamics. *Computer Methods in Applied Mechanics and Engineering* 54, 2 (1986), 223–234.
- [61] HUGHES, T. J.; LEVIT, I.; WINGET, J. An element-by-element solution algorithm for problems of structural and solid mechanics. *Computer Methods in Applied Mechanics and Engineering* 36, 2 (1983), 241–254.
- [62] HUSSAINI, S. R.; DVORKIN, J. Permeability in the thin section. *Journal of Petroleum Science and Engineering* 218 (2022), 110959.
- [63] IKLASSOV, Z.; MEDVEDEV, D.; NAZAROV, O.; RAZZOKOV, S. Ai for porosity and permeability prediction from geologic core x-ray micro-tomography, 2022. Last accessed in March 15th of 2023.

- [64] KABEL, M.; MERKERT, D.; SCHNEIDER, M. Use of composite voxels in fft-based homogenization. *Computer Methods in Applied Mechanics and Engineering* 294 (2015), 168–188.
- [65] KAPER, T.; MARDAL, K.-A.; WINTHER, R. Unified finite element discretizations of coupled darcy-stokes flow. *Numerical Methods for Partial Differential Equations* 25, 2 (2008), 311–326.
- [66] KEEHM, Y. Computational rock physics: Transport properties in porous media and applications. Phd Thesis. TStanford University, USA, jan 2003. Last accessed in March 15th of 2023.
- [67] KESSLER, A. Matrix-free voxel-based finite element method for materials with heterogeneous microstructures. Phd Thesis. Bauhaus-Universitat Weimar, Germany, December 2017.
- [68] KIRAN, U.; GAUTAM, S.; SHARMA, D. GPU-based matrix-free finite element solver exploiting symmetry of elemental matrices. *Computing* 102 (2020), 1941–1965.
- [69] KIRK, D. B.; MEI W. HWU, W. *Programming Massively Parallel Processors*, 1 ed. Morgan Kaufmann, Burlington, USA, 2010. NVIDIA.
- [70] KÖBLER, J.; SCHNEIDER, M.; OSPALD, F.; ANDRÄ, H.; MÜLLER, R. Fiber orientation interpolation for the multiscale analysis of short fiber reinforced composite parts. *Computational Mechanics* 61, 6 (Jun 2018), 729–750.
- [71] KOCUREK, I. Details for boise sandstone. <https://kocurekindustries.com/our-products/boise-sandstone>. Last accessed in March 30th of 2023.
- [72] KONG, Y.; HUANG, Z.; XIONG, W. Parallelism of the finite-time dynamics method based on gpu. *Computing* 104, 8 (Aug 2022), 1721–1738.
- [73] KOZENY, J. Über kapillare leitung des wassers im boden. *Ber. Wien Akad.* 136 (1927), Abt. II a, P. 277.
- [74] LANCZOS, C. Solution of systems of linear equations by minimized iterations. *Journal of Research of the National Bureau of Standards* 49, 1 (7 1952), 33–53.
- [75] LESUEUR, M.; RATTEZ, H.; COLOMÉS, O.  $\mu$ ct scans permeability computation with an unfitted boundary method to improve coarsening accuracy. *Computers & Geosciences* 166 (2022), 105118.
- [76] LIU, X.; RÉTHORÉ, J.; BAIETTO, M.-C.; SAINOT, P.; LUBRECHT, A. A. An efficient strategy for large scale 3d simulation of heterogeneous materials to predict effective thermal conductivity. *Computational Materials Science* 166 (2019), 265–275.
- [77] LIU, X.; RÉTHORÉ, J.; BAIETTO, M.-C.; SAINOT, P.; LUBRECHT, A. A. An efficient finite element based multigrid method for simulations of the mechanical behavior of heterogeneous materials using ct images. *Computational Mechanics* 66, 6 (12 2020), 1427–1441.

- [78] LOEB, A.; EARLS, C. Analysis of heterogeneous computing approaches to simulating heat transfer in heterogeneous material. *Journal of Parallel and Distributed Computing* 133 (2019), 1–17.
- [79] LOPES, P. Massively parallel implementations of the preconditioned conjugate gradient method applied to image-based numerical homogenization with the assembly-free finite element method. Msc Dissertation. Universidade Federal Fluminense, Brasil, August 2021.
- [80] LOPES, P. C.; VIANNA, R. S.; SAPUCAIA, V. W.; SEMERARO, F.; LEIDERMAN, R.; PEREIRA, A. M. Simulation toolkit for digital material characterization of large image-based microstructures. *Computational Materials Science* 219 (2023), 112021.
- [81] LOPES, P. C. F.; PEREIRA, A. M. B.; CLUA, E. W. G.; LEIDERMAN, R. A gpu implementation of the pcg method for large-scale image-based finite element analysis in heterogeneous periodic media. *Computer Methods in Applied Mechanics and Engineering* 399 (2022), 115276.
- [82] LOPES, P. C. F.; SAPUCAIA, V. W.; PEREIRA, A. M. B.; LEIDERMAN, R. A vectorized assembly-free fem solver for image-based numerical homogenization. *Journal of the Brazilian Society of Mechanical Sciences and Engineering* 44, 8 (Jul 2022), 343.
- [83] LY, H. B.; MONCHIET, V.; GRANDE, D. Computation of permeability with fast fourier transform from 3-d digital images of porous microstructures. *International Journal of Numerical Methods for Heat & Fluid Flow* 26, 5 (Jan 2016), 1328–1345.
- [84] MARFIA, S.; SACCO, E. Tfa-based homogenization for composites subjected to coupled damage-friction effects. *Procedia Engineering* 109 (2015), 113–120. XXIII Italian Group of Fracture Meeting, IGFXIII.
- [85] MARTÍNEZ-FRUTOS, J.; HERRERO-PÉREZ, D. Efficient matrix-free GPU implementation of fixed grid finite element analysis. *Finite Elements in Analysis and Design* 104 (2015), 61–71.
- [86] MARTÍNEZ-FRUTOS, J.; HERRERO-PÉREZ, D. Large-scale robust topology optimization using multi-GPU systems. *Computer Methods in Applied Mechanics and Engineering* 311 (2016), 393–414.
- [87] MARTÍNEZ-FRUTOS, J.; MARTÍNEZ-CASTEJÓN, P. J.; HERRERO-PÉREZ, D. Fine-grained GPU implementation of assembly-free iterative solver for finite element problems. *Computers & Structures* 157 (2015), 9–18.
- [88] MASE, G. T.; SMELSER, R. E.; MASE, G. E. *Continuum Mechanics for Engineers*, 3 ed. CRC Press, Boca Raton, USA, 2010.
- [89] MEHDIKHANI, M.; BREITE, C.; SWOLFS, Y.; WEVERS, M.; LOMOV, S. V.; GORBATIKH, L. Combining digital image correlation with x-ray computed tomography for characterization of fiber orientation in unidirectional composites. *Composites Part A: Applied Science and Manufacturing* 142 (2021), 106234.

- [90] MEHDIKHANI, M.; BREITE, C.; SWOLFS, Y.; WEVERS, M.; LOMOV, S. V.; GORBATIKH, L. A dataset of micro-scale tomograms of unidirectional glass fiber/epoxy and carbon fiber/epoxy composites acquired via synchrotron computed tomography during in-situ tensile loading. *Data in Brief* 34 (2021).
- [91] MESQUITA, P. V. Melhoria da estimativa de permeabilidade através da integração de dados de rmn e micp em rochas análogas a reservatórios de hidrocarbonetos. BSc Dissertation. Universidade Federal Fluminense, Brazil, 2017. Last accessed in March 30th of 2023.
- [92] MEZHOUD, S.; MONCHIET, V.; BORNERT, M.; GRANDE, D. Computation of macroscopic permeability of doubly porous media with fft based numerical homogenization method. *European Journal of Mechanics - B/Fluids* 83 (2020), 141–155.
- [93] MICHEL, J.; MOULINEC, H.; SUQUET, P. Effective properties of composite materials with periodic microstructure: a computational approach. *Comput. Methods Appl. Mech. Engrg.* 172 (1999), 109–143.
- [94] MICHEL, J.; SUQUET, P. Nonuniform transformation field analysis. *International Journal of Solids and Structures* 40, 25 (2003), 6937–6955. Special issue in Honor of George J. Dvorak.
- [95] MIRZENDEHDEL, A. M. Assembly-free structural dynamics on CPU and GPU. Msc Dissertation. University of Wisconsin, Madison, USA, 2014.
- [96] MONCHIET, V.; BONNET, G.; LAURIAT, G. A fft-based method to compute the permeability induced by a stokes slip flow through a porous medium. *Comptes Rendus Mécanique* 337, 4 (2009), 192–197.
- [97] MOSTAGHIMI, P.; BLUNT, M. J.; BIJELJIC, B. Computations of absolute permeability on micro-ct images. *Mathematical Geosciences* 45, 1 (Jan 2013), 103–125.
- [98] MOULINEC, H.; SUQUET, P. A numerical method for computing the overall response of nonlinear composites with complex microstructure. *Computer Methods in Applied Mechanics and Engineering* 157, 1 (1998), 69–94.
- [99] NVIDIA. Cuda toolkit documentation v11.3.0. Website, 2021. Available at <https://docs.nvidia.com/cuda/index.html>. Last accessed in May 19th of 2021.
- [100] OLIVEIRA, J.; DA CRUZ, J. P.; TEIXEIRA-DIAS, F. Asymptotic homogenisation in linear elasticity. part ii: Finite element procedures and multiscale applications. *Computational Materials Science* 45, 4 (2009), 1081–1096.
- [101] ØREN, P.-E.; ANTONSEN, F.; RUESLÅTTEN, H. G.; BAKKE, S. Numerical simulations of nmr responses for improved interpretations of nmr measurements in reservoir rocks. *SPE Annual Technical Conference and Exhibition All Days* (09 2002). SPE-77398-MS.
- [102] ØREN, P.-E.; BAKKE, S. Process based reconstruction of sandstones and prediction of transport properties. *Transport in Porous Media* 46, 2 (Feb 2002), 311–343.
- [103] ØREN, P.-E.; BAKKE, S.; ARNTZEN, O. J. Extending predictive capabilities to network models. *SPE Journal* 3, 04 (12 1998), 324–336.

- [104] PAIGE, C. C.; SAUNDERS, M. A. Solution of sparse indefinite systems of linear equations. *SIAM Journal on Numerical Analysis* 12, 4 (1975), 617–629.
- [105] PANERAI, F.; FERGUSON, J.; LACHAUD, J.; MARTIN, A.; GASCH, M. J.; MANSOUR, N. N. Analysis of fibrous felts for flexible ablators using synchrotron hard x-ray micro-tomography. In *European Symposium on Aerothermodynamics for Space Vehicles* (2015).
- [106] PANERAI, F.; FERGUSON, J. C.; LACHAUD, J.; MARTIN, A.; GASCH, M. J.; MANSOUR, N. N. Micro-tomography based analysis of thermal conductivity, diffusivity and oxidation behavior of rigid and flexible fibrous insulators. *International Journal of Heat and Mass Transfer* 108 (2017), 801–811.
- [107] PANERAI, F.; WHITE, J. D.; COCHELL, T. J.; SCHROEDER, O. M.; MANSOUR, N. N.; WRIGHT, M. J.; MARTIN, A. Experimental measurements of the permeability of fibrous carbon at high-temperature. *International Journal of Heat and Mass Transfer* 101 (2016), 267–273.
- [108] PEARSON, J. W.; PESTANA, J. Preconditioners for krylov subspace methods: An overview. *GAMM-Mitteilungen* 43, 4 (2020), e202000015.
- [109] PEREIRA, A.; COSTA, M.; ANFLOR, C.; PARDAL, J.; LEIDERMAN, R. Estimating the effective elastic parameters of nodular cast iron from micro tomographic imaging and multiscale finite elements: Comparison between numerical and experimental results. *Metals* 695, 8 (2018).
- [110] PEREIRA, A.; TALISCHI, C.; PAULINO, G. H.; M. MENEZES, I. F.; CARVALHO, M. S. Fluid flow topology optimization in polytop: stability and computational implementation. *Structural and Multidisciplinary Optimization* 54, 5 (Nov 2016), 1345–1364.
- [111] PERRINS, W.; MCKENZIE, D.; MCPHEDRAN, R. Transport properties of regular arrays of cylinders. *Proceedings of the Royal Society of London. A. Mathematical and Physical Sciences* 369 (May 1979), 207–225.
- [112] PETERS, J.; REICHELT, V.; REUSKEN, A. Fast iterative solvers for discrete stokes equations. *SIAM Journal on Scientific Computing* 27, 2 (2005), 646–666.
- [113] PETROCORES. Boise. <https://www.petrocores.com/portfolio-item/boise/>. Last accessed in March 30th of 2023.
- [114] PRODANOVIC, M.; LINDQUIST, W.; SERIGHT, R. 3d image-based characterization of fluid displacement in a berea core. *Advances in Water Resources* 30, 2 (2007), 214–226. Pore-scale Modeling.
- [115] QI, M.; WEGNER, J.; FALCO, L.; GANZER, L. Pore-Scale Simulation of Viscoelastic Polymer Flow using a Stabilised Finite Element Method. In *SPE Reservoir Characterisation and Simulation Conference and Exhibition* (09 2013). SPE-165987-MS.

- [116] RATNAKAR, S. K.; SANFUI, S.; SHARMA, D. Graphics Processing Unit-Based Element-by-Element Strategies for Accelerating Topology Optimization of Three-Dimensional Continuum Structures Using Unstructured All-Hexahedral Mesh. *Journal of Computing and Information Science in Engineering* 22, 2 (12 2021). 021013.
- [117] REDDY, J. N. *Introduction to the finite element method*. McGraw-Hill Education, 2019.
- [118] RIBEIRO, M. C. Toward an ultrasonic inspecting method to detect and classify adhesive bonding defects in real time. Msc Dissertation. Universidade Federal Fluminense, Niterói, Brasil, 2019.
- [119] RIBEIRO, M. C.; FILGUEIRAS, J. G.; SOUZA, A.; VIANNA, P. M.; DE AZEREDO, R. B.; LEIDERMAN, R. Image-based simulation of molecular diffusion on nmr pulsed-field gradient experiments: Feasibility to estimate tortuosity and permeability of porous media. *Journal of Petroleum Science and Engineering* 219 (2022), 111064.
- [120] ROUSSETTE, S.; MICHEL, J.; SUQUET, P. Nonuniform transformation field analysis of elastic-viscoplastic composites. *Composites Science and Technology* 69, 1 (2009), 22–27. Mechanical Response of Fibre Reinforced Composites.
- [121] SAEGER, R. B.; SCRIVEN, L. E.; DAVIS, H. T. Transport processes in periodic porous media. *Journal of Fluid Mechanics* 299 (1995), 1–15.
- [122] SANDERS, J.; KANDROT, E. *CUDA by example: An Introduction to General-Purpose GPU Programming*, 1 ed. Addison Wesley, Boston, USA, 2011. NVIDIA.
- [123] SAXENA, N.; HOFMANN, R.; ALPAK, F. O.; DIETDERICH, J.; HUNTER, S.; DAY-STIRRAT, R. J. Effect of image segmentation & voxel size on micro-ct computed effective transport & elastic properties. *Marine and Petroleum Geology* 86 (2017), 972–990.
- [124] SAXENA, N.; HOWS, A.; HOFMANN, R.; ALPAK, F. O.; DIETDERICH, J.; APPEL, M.; FREEMAN, J.; DE JONG, H. Rock properties from micro-ct images: Digital rock transforms for resolution, pore volume, and field of view. *Advances in Water Resources* 134 (2019), 103419.
- [125] SAXENA, N.; HOWS, A.; HOFMANN, R.; O. ALPAK, F.; FREEMAN, J.; HUNTER, S.; APPEL, M. Imaging and computational considerations for image computed permeability: Operating envelope of digital rock physics. *Advances in Water Resources* 116 (2018), 127–144.
- [126] SAXENA, N.; MAVKO, G.; HOFMANN, R.; SRISUTTHIYAKORN, N. Estimating permeability from thin sections without reconstruction: Digital rock study of 3d properties from 2d images. *Computers & Geosciences* 102 (2017), 79–99.
- [127] SCHNEIDER, M. A review of nonlinear fft-based computational homogenization methods. *Acta Mechanica* 232, 6 (Jun 2021), 2051–2100.
- [128] SCHWARTZ, L.; AUZERAIS, F.; DUNSMUIR, J.; MARTYS, N.; BENTZ, D.; TORQUATO, S. Transport and diffusion in three-dimensional composite media. *Physica A: Statistical Mechanics and its Applications* 207, 1 (1994), 28–36.

- [129] SEMERARO, F.; FERGUSON, J. C.; ACIN, M.; PANERAI, F.; MANSOUR, N. N. Anisotropic analysis of fibrous and woven materials part 2: Computation of effective conductivity. *Computational Materials Science* 186 (2021), 109956.
- [130] SEMERARO, F.; FERGUSON, J. C.; PANERAI, F.; KING, R. J.; MANSOUR, N. N. Anisotropic analysis of fibrous and woven materials part 1: Estimation of local orientation. *Computational Materials Science* 178 (2020), 109631.
- [131] SEUFFERT, J.; BITTRICH, L.; CARDOSO DE OLIVEIRA, L.; SPICKENHEUER, A.; KÄRGER, L. Micro-scale permeability characterization of carbon fiber composites using micrograph volume elements. *Frontiers in Materials* 8 (2021).
- [132] SHEWCHUK, J. An introduction to the conjugate gradient method without the agonizing pain. Tech. rep., Carnegie Mellon University, Pittsburgh, USA, 1994. Last accessed in March 8th of 2023.
- [133] SILVA, G.; GINZBURG, I. Stokes–brinkman–darcy solutions of bimodal porous flow across periodic array of permeable cylindrical inclusions: Cell model, lubrication theory and lbm/fem numerical simulations. *Transport in Porous Media* 111, 3 (Feb 2016), 795–825.
- [134] SOGABE, T.; SUGIHARA, M.; ZHANG, S.-L. An extension of the conjugate residual method to nonsymmetric linear systems. *Journal of Computational and Applied Mathematics* 226, 1 (2009), 103–113. Special Issue: The First International Conference on Numerical Algebra and Scientific Computing (NASC06).
- [135] STRANG, G. Krylov subspaces and conjugate gradients. Tech. rep., MIT, Cambridge, USA, 2006. Last accessed in March 23rd of 2023.
- [136] STRANG, G. The saddle point stokes problem. Tech. rep., MIT, Cambridge, USA, 2006. Last accessed in March 22nd of 2023.
- [137] SUN, W. C.; WONG, T.-F. Prediction of permeability and formation factor of sandstone with hybrid lattice Boltzmann/finite element simulation on microtomographic images. *International Journal of Rock Mechanics and Mining Sciences* 106, April (2018), 269–277.
- [138] TALISCHI, C.; PEREIRA, A.; PAULINO, G. H.; MENEZES, I. F.; CARVALHO, M. S. Polygonal finite elements for incompressible fluid flow. *International Journal for Numerical Methods in Fluids* 74, 2 (2014), 134–151.
- [139] TANG, T.; FELICELLI, S. D. Computational evaluation of effective stress relaxation behavior of polymer composites. *International Journal of Engineering Science* 90 (2015), 76–85.
- [140] TEIXEIRA, J. B. Petrografia de rochas reservatório areníticas e relação com diferentes técnicas de obtenção de parâmetros petrofísicos. MSc Dissertation. Universidade Federal Fluminense, Brazil, 2016. Last accessed in March 30th of 2023.
- [141] TEMBELY, M.; ALSUMAITI, A. M.; ALAMERI, W. S. Machine and deep learning for estimating the permeability of complex carbonate rock from x-ray micro-computed tomography. *Energy Reports* 7 (2021), 1460–1472.

- [142] VIANNA, R.; CUNHA, A.; AZEREDO, R.; LEIDERMAN, R.; PEREIRA, A. Computing effective permeability of porous media with fem and micro-ct: An educational approach. *Fluids* 16, 5 (2020).
- [143] VICTOR, R. A. Pore scale modeling of rock transport properties. MSc Dissertation. University of Texas at Austin, USA, August 2014. Last accessed in March 30th of 2023.
- [144] VICTOR, R. A.; MIRABOLGHAEMI, M.; BRYANT, S. L.; PRODANOVIC, M. Minimum divergence viscous flow simulation through finite difference and regularization techniques. *Advances in Water Resources* 95 (2016), 29–45. Pore scale modeling and experiments.
- [145] WANG, Y.; LUO, Z.; ZHANG, N.; KANG, Z. Topological shape optimization of microstructural metamaterials using a level set method. *Computational Materials Science* 87 (2014), 178–186.
- [146] WHITAKER, S. Flow in porous media i: A theoretical derivation of darcy’s law. *Transport in Porous Media* 1, 1 (Mar 1986), 3–25.
- [147] WOLLNER, U.; KERIMOV, A.; MAVKO, G. Scale and boundary effects on the effective elastic properties of 2-d and 3-d non-rev heterogeneous porous media. *Journal of Geophysical Research: Solid Earth* 123, 7 (2018), 5451–5465.
- [148] WU, Y.; LI, J.; YANG, Z.; MA, Z.; GUO, Y.; TAO, D.; YANG, T.; LIANG, M. Computational assessment of thermal conductivity of compacted graphite cast iron. *Advances in Materials Science and Engineering* 2019 (2019).
- [149] XIE, X.; XU, J.; XUE, G. Uniformly-stable finite element methods for darcy-stokes-brinkman models. *Journal of Computational Mathematics* 26, 3 (2008), 437–455.
- [150] XIONG, Q.; BAYCHEV, T. G.; JIVKOV, A. P. Review of pore network modeling of porous media: Experimental characterisations, network constructions and applications to reactive transport. *Journal of Contaminant Hydrology* 192 (2016), 101–117.
- [151] YANG, L.; YANG, J.; BOEK, E.; SAKAI, M.; PAIN, C. Image-based simulations of absolute permeability with massively parallel pseudo-compressible stabilised finite element solver. *Computational Geosciences* 23, 5 (Oct 2019), 881–893.
- [152] YU-LING LAI; WEN-WEI LIN; PIERCE, D. Conjugate gradient and minimal residual methods for solving symmetric indefinite systems. *Journal of Computational and Applied Mathematics* 84, 2 (1997), 243–256.
- [153] ZHANG, H.; YU, H.; YUAN, X.; XU, H.; MICHEAL, M.; ZHANG, J.; SHU, H.; WANG, G.; WU, H. Permeability prediction of low-resolution porous media images using autoencoder-based convolutional neural network. *Journal of Petroleum Science and Engineering* 208 (2022), 109589.
- [154] ZICK, A.; HOMSY, G. Stokes flow through periodic arrays of spheres. *Journal of Fluid Mechanics* 115 (1982), 13–26.

- 
- [155] ZIENKIEWICZ, O.; TAYLOR, R. L. *The Finite Element Method: Its Basis and Fundamentals*, 7th ed. Elsevier, 2013.